

MWORKS.Sysplorer

2026a 基础功能



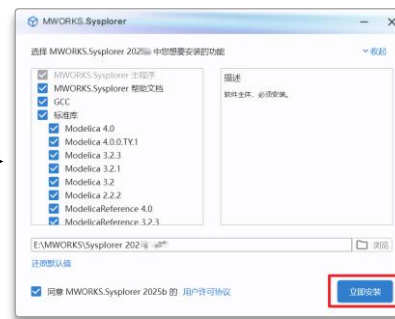
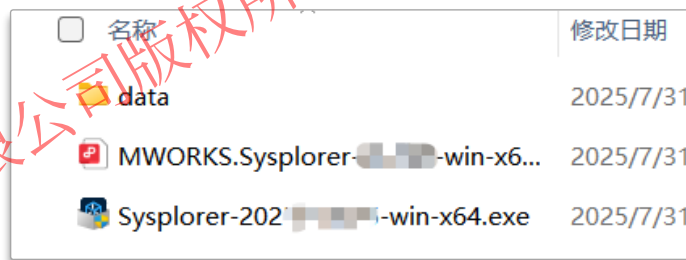
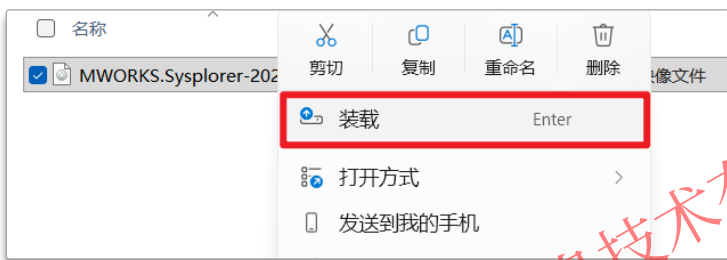
软件下载与注册



安装包为iso光盘映像文件。其中：

- data文件夹为相关资源文件
- .exe文件为MWORKS.Sysplorer安装程序
- PDF文件为安装与配置说明书

安装步骤



- 若已安装 Sysplorer 旧版本软件，可直接在 Sysplorer 软件内收到更新通知，实现**热更新**，无需单独前往官网下载

软件下载与注册

- 高校师生可使用**学校邮箱**直接注册账号，申请教育版使用许可，教育版许可每180天需要重新激活
- 个人版仅限个人非商业用途使用，且功能有所欠缺



国产科学计算与系统建模仿真赋能专项行动

MWORKS 2026

——星火计划

“星星之火，足以燎原。”

国产自主不只是一个愿景，更是一场可以由每一位工程师共同参与的实证之旅。为加速这一进程，同元软控正式启动MWORKS 免费升级与试用活动，期待联合广大用户、生态伙伴，在真实场景中打磨国产工业软件实战性能。

星火已燃，静待同路人。我们诚邀您加入“MWORKS星火计划”，共同见证并推动中国工业软件的自强历程！

★ 活动详情：请关注“同元软控” / “MWORKS” 官方公众号

★ 2026年2月7日-12月31日，为企业用户提供以下专属权益：



零成本试用

全功能版本免费开放



升级支持

老用户免费版本升级



线上培训

免费参加线上培训



产品资料包

全套产品资料包



技术支持

线上/线下技术支持

国产科学计算与系统建模仿真赋能专项行动

MWORKS 2026

——星火计划

“星星之火，足以燎原。”

国产自主不只是一个愿景，更是一场可以由每一位工程师共同参与的实证之旅。为加速这一进程，同元软控正式启动MWORKS 免费升级与试用活动，期待联合广大用户、生态伙伴，在真实场景中打磨国产工业软件实战性能。

星火已燃，静待同路人。我们诚邀您加入“MWORKS星火计划”，共同见证并推动中国工业软件的自强历程！

★ 活动详情：请关注“同元软控” / “MWORKS” 官方公众号

★ 面向高校用户，长期提供以下权益



校级免费授权
全校教育版使用



课程教材评优
奖金/资源/荣誉



数智英才培养
对接企业优先就业



高影响力赛事
荣誉/奖金/就业



应用赋能培训
季度性专业辅导



百行百业共建
拉通产业产品共创



产品资料包
全套产品资料包



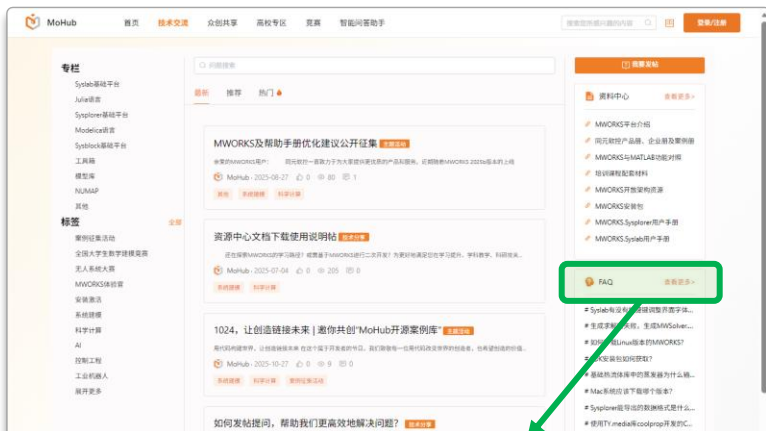
技术支持
线上/线下技术支持

MoHub社区-技术交流、智能助手、在线技术支持

- MoHub提供MWORKS常见FAQ、同时为MWORKS用户提供了相互交流的社区平台
- 智能问答助手内嵌的大模型深度融合了MWORKS相关知识和运行机制，能够为用户提供更为精准的回复
- 在线技术支持由同元技术工程师团队轮值坐席，为用户提供技术支持保障



MoHub首页: www.mohub.net



在线技术支持，及时解决用户问题

CONTENTS

目录 ➔



MWORKS.Sysplorer软件概览



模型库加载



拖拽建模：电子阀流量控制系统



文本建模：电阻



模型发布



MWORKS.Sysplorer与Simulink之间的差异



Modelica语法概览

PART 01 ➔

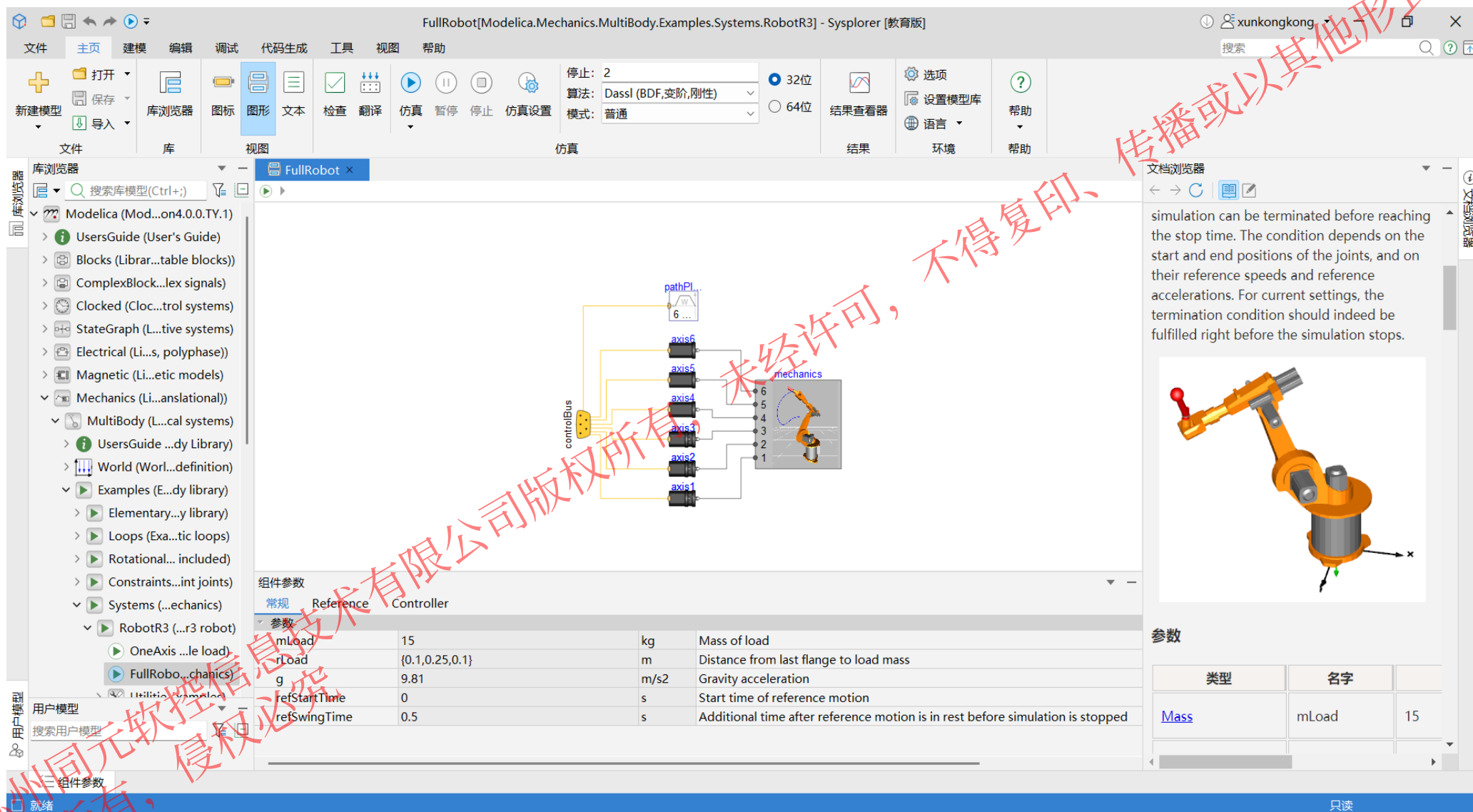
MWORKS.Sysplorer软件概览

@苏州同元软控信息技术有限公司
版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

01

1. MWORKS.Sysplorer软件概览



六自由度机械手模型路径为：

Modelica.Mechanics.MultiBody.Examples.Systems.RobotR3.fullRobot

1. MWORKS.Sysplorer软件概览

快速工具访问栏

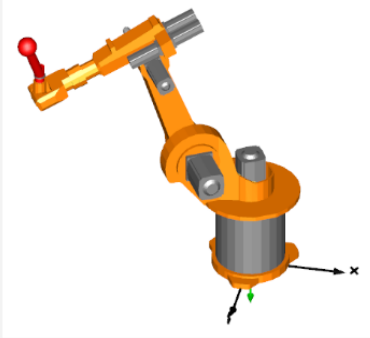
文件 主页 建模 编辑 调试 代码生成 工具 视图 帮助

新建模型 打开 保存 导入 库浏览器 图标 图形 文本 检查 翻译 仿真 暂停 停止 仿真设置 停止: 2 算法: Dassl (BDF,变阶,刚性) 模式: 普通 32位 64位 结果查看器 选项 设置模型库 语言 帮助

功能区

文档浏览器

simulation can be terminated before reaching the stop time. The condition depends on the start and end positions of the joints, and on their reference speeds and reference accelerations. For current settings, the termination condition should indeed be fulfilled right before the simulation stops.



参数

类型	名字	
Mass	mLoad	15

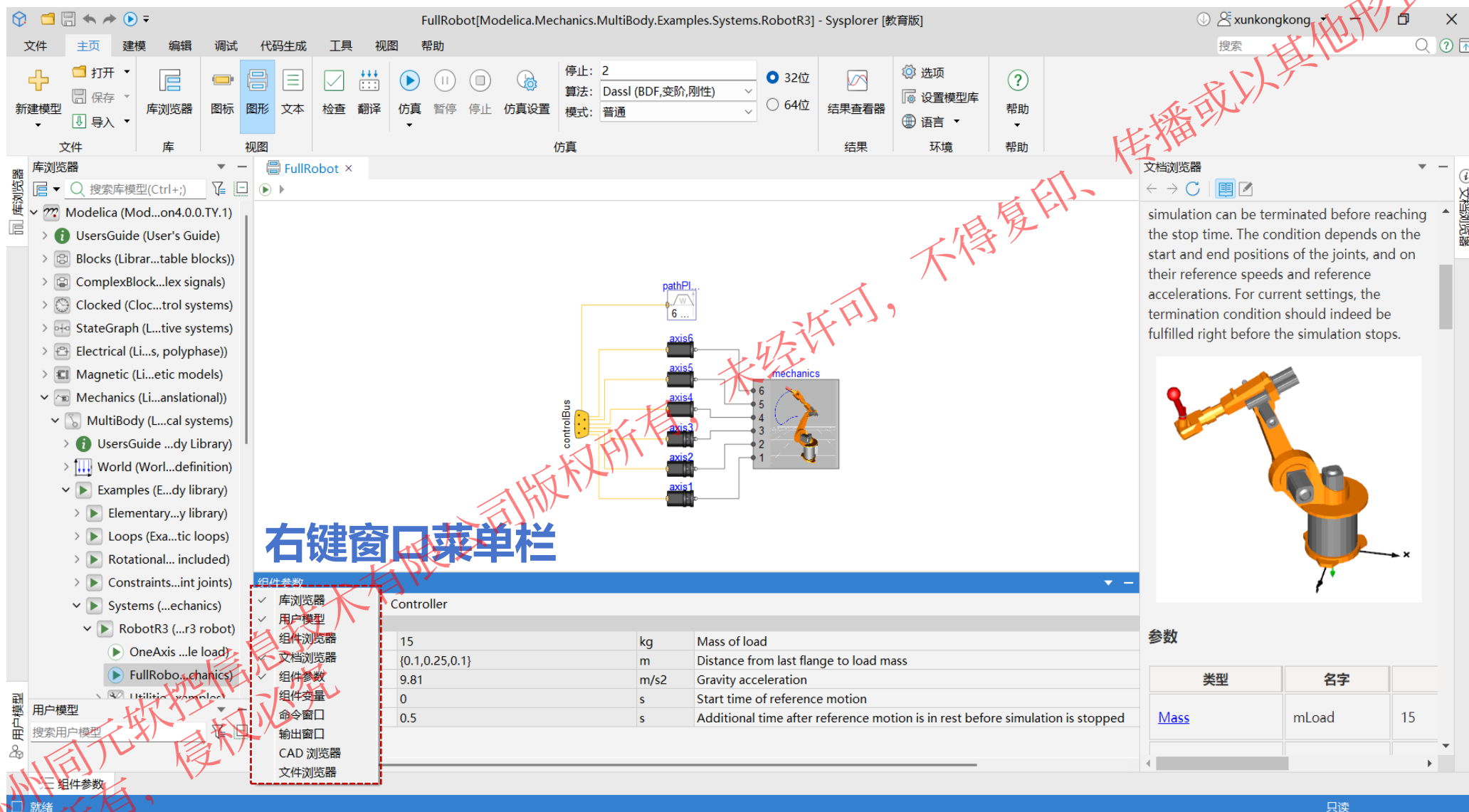
组件参数

常规	Reference	Controller	
参数			
mLoad	15	kg	Mass of load
rLoad	{0.1,0.25,0.1}	m	Distance from last flange to load mass
g	9.81	m/s2	Gravity acceleration
refStartTime	0	s	Start time of reference motion
refSwingTime	0.5	s	Additional time after reference motion is in rest before simulation is stopped

就绪 只读

@苏州同元软控信息技术有限公司版权所有，侵权必究

1. MWORKS.Sysplorer软件概览

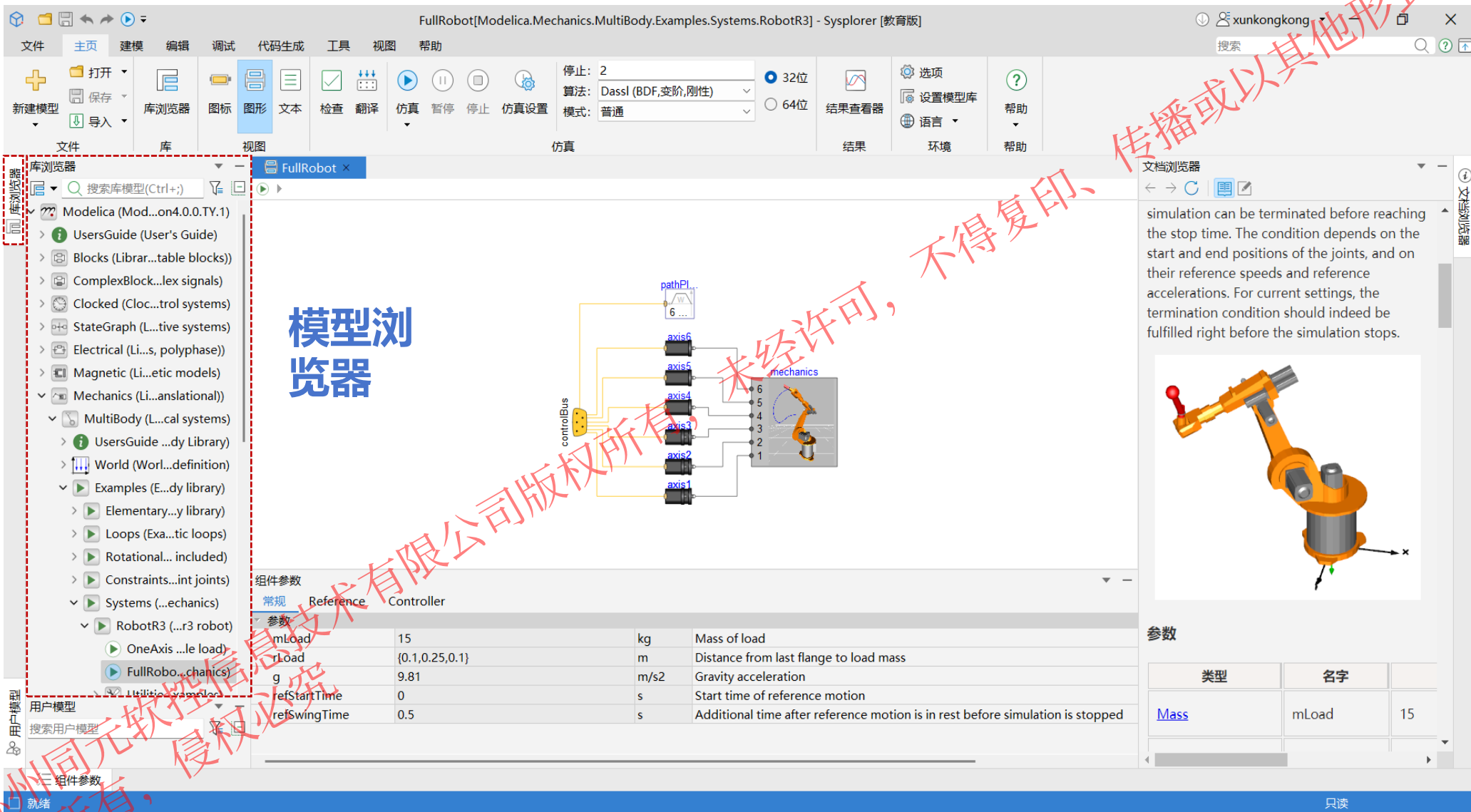


传播或以其他形式使用

未经许可，不得复印、

苏州同元软控信息技术有限公司版权所有，侵权必究

1. MWORKS.Sysplorer软件概览



1. MWORKS.Sysplorer软件概览

FullRobot[Modelica.Mechanics.MultiBody.Examples.Systems.RobotR3] - Sysplorer [教育版]

文件 主页 建模 编辑 调试 代码生成 工具 视图 帮助

新建模型 打开 保存 库浏览器 图标 图形 文本 检查 翻译 仿真 暂停 停止 仿真设置

停止: 2
算法: Dassl (BDF,变阶,刚性)
模式: 普通

32位 64位

结果查看器 选项 设置模型库 语言 帮助

库浏览器

Modelica (Mode...ion4.0.0.TY.1)
Sysblock

搜索库模型(Ctrl+;)

组件浏览器

Modelica.Mecha...otr3.FullRobot
extends Modelica.Icons.Example
mechanics
pathPlanning
axis1
axis2
axis3
axis4
axis5
axis6
controlBus

组件参数

常规	Reference	Controller	
参数			
mLoad	15	kg	Mass of load
rLoad	{0.1,0.25,0.1}	m	Distance from last flange to load mass
g	9.81	m/s2	Gravity acceleration
refStartTime	0	s	Start time of reference motion
refSwingTime	0.5	s	Additional time after reference motion is in rest before simulation is stopped

仿真

结果

文档浏览器

simulation can be terminated before reaching the stop time. The condition depends on the start and end positions of the joints, and on their reference speeds and reference accelerations. For current settings, the termination condition should indeed be fulfilled right before the simulation stops.

参数

类型	名字	值
Mass	mLoad	15

只读

组件浏览器

组件参数

仿真

结果

文档浏览器

simulation can be terminated before reaching the stop time. The condition depends on the start and end positions of the joints, and on their reference speeds and reference accelerations. For current settings, the termination condition should indeed be fulfilled right before the simulation stops.

参数

类型	名字	值
Mass	mLoad	15

只读

传播或以其他形式使用

未经许可，不得复印、

@苏州同元软控信息技术有限公司版权所有，侵权必究

1. MWORKS.Sysplorer软件概览

FullRobot[Modelica.Mechanics.MultiBody.Examples.Systems.RobotR3] - Sysplorer [教育版]

文件 主页 建模 编辑 调试 代码生成 工具 视图 帮助

新建模型 打开 保存 库浏览器 图标 图形 文本 检查 翻译 仿真 暂停 停止 仿真设置 停止: 2 算法: Dassl (BDF,变阶,刚性) 模式: 普通 32位 64位 结果查看器 选项 设置模型库 语言 帮助

库浏览器 搜索库模型(Ctrl+;) Modelica (Mode...ion4.0.0.TY.1) Sysblock

图形建模视图

组件浏览器

- Modelica.Mecha...otr3.FullRobot
 - extends Modelica.Icons.Example
 - mechanics
 - pathPlanning
 - axis1
 - axis2
 - axis3
 - axis4
 - axis5
 - axis6
 - controlBus

组件参数

常规	Reference	Controller	
参数			
mLoad	15	kg	Mass of load
rLoad	{0.1,0.25,0.1}	m	Distance from last flange to load mass
g	9.81	m/s2	Gravity acceleration
refStartTime	0	s	Start time of reference motion
refSwingTime	0.5	s	Additional time after reference motion is in rest before simulation is stopped

文档浏览器

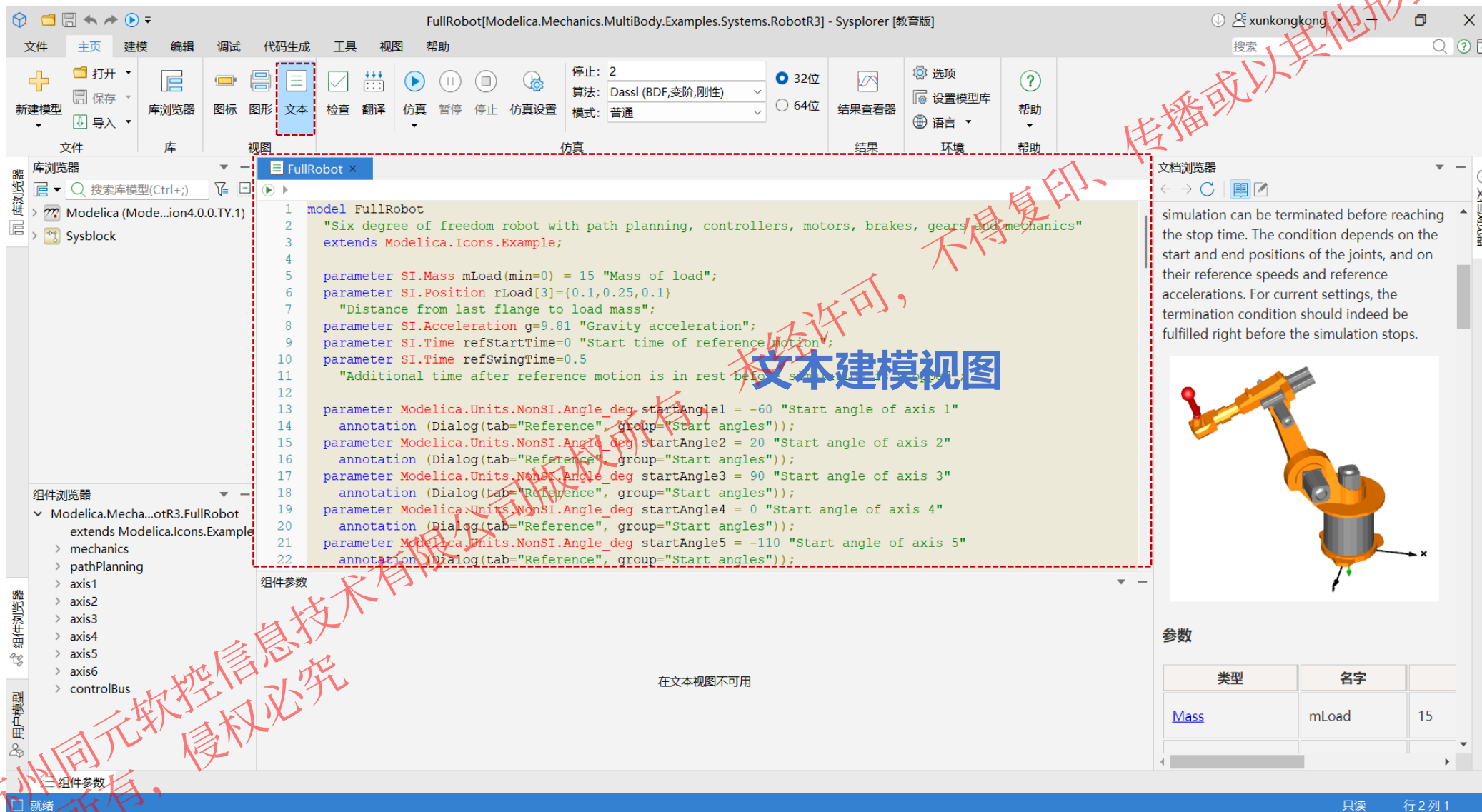
simulation can be terminated before reaching the stop time. The condition depends on the start and end positions of the joints, and on their reference speeds and reference accelerations. For current settings, the termination condition should indeed be fulfilled right before the simulation stops.

参数

类型	名字	
Mass	mLoad	15

就绪 只读

1. MWORKS.Sysplorer软件概览



1. MWORKS.Sysplorer软件概览

FullRobot[Modelica.Mechanics.MultiBody.Examples.Systems.RobotR3] - Sysplorer [教育版]

文件 主页 建模 编辑 调试 代码生成 工具 视图 帮助

新建模型 打开 保存 导入 库浏览器 图标 图形 文本 检查 翻译 仿真 暂停 停止 仿真设置 停止: 2 算法: Dassl (BDF,变阶,刚性) 模式: 普通 32位 64位 结果查看器 选项 设置模型库 语言 帮助

库浏览器: Modelica (Mode...ion4.0.0.TY.1) Sysblock

组件浏览器: Modelica.Mecha...otr3.FullRobot extends Modelica.Icons.Example > mechanics > pathPlanning > axis1 > axis2 > axis3 > axis4 > axis5 > axis6 > controlBus

组件参数窗口

常规	Reference	Controller	
参数			
mLoad	15	kg	Mass of load
rLoad	{0.1,0.25,0.1}	m	Distance from last flange to load mass
g	9.81	m/s2	Gravity acceleration
refStartTime	0	s	Start time of reference motion
refSwingTime	0.5	s	Additional time after reference motion is in rest before simulation is stopped

文档浏览器: simulation can be terminated before reaching the stop time. The condition depends on the start and end positions of the joints, and on their reference speeds and reference accelerations. For current settings, the termination condition should indeed be fulfilled right before the simulation stops.

参数

类型	名字	
Mass	mLoad	15

就绪 只读

1. MWORKS.Sysplorer软件概览

FullRobot[Modelica.Mechanics.MultiBody.Examples.Systems.RobotR3] - Sysplorer [教育版]

文件 主页 建模 编辑 调试 代码生成 工具 视图 帮助

新建模型 打开 保存 库浏览器 图标 图形 文本 检查 翻译 仿真 暂停 停止 仿真设置 停止: 2 算法: Dassl (BDF,变阶,刚性) 模式: 普通 32位 64位 结果查看器 选项 设置模型库 语言 帮助

库浏览器

Modelica (Mode...ion4.0.0.TY.1)

Sysblock

组件浏览器

- Modelica.Mecha...otR3.FullRobot
 - extends Modelica.Icons.Example
 - mechanics
 - pathPlanning
 - axis1
 - axis2
 - axis3
 - axis4
 - axis5
 - axis6
 - controlBus

组件参数

常规	Reference	Controller	
参数			
mLoad	15	kg	Mass of load
rLoad	{0.1,0.25,0.1}	m	Distance from last flange to load mass
g	9.81	m/s2	Gravity acceleration
refStartTime	0	s	Start time of reference motion
refSwingTime	0.5	s	Additional time after reference motion is in rest before simulation is stopped

文档浏览器

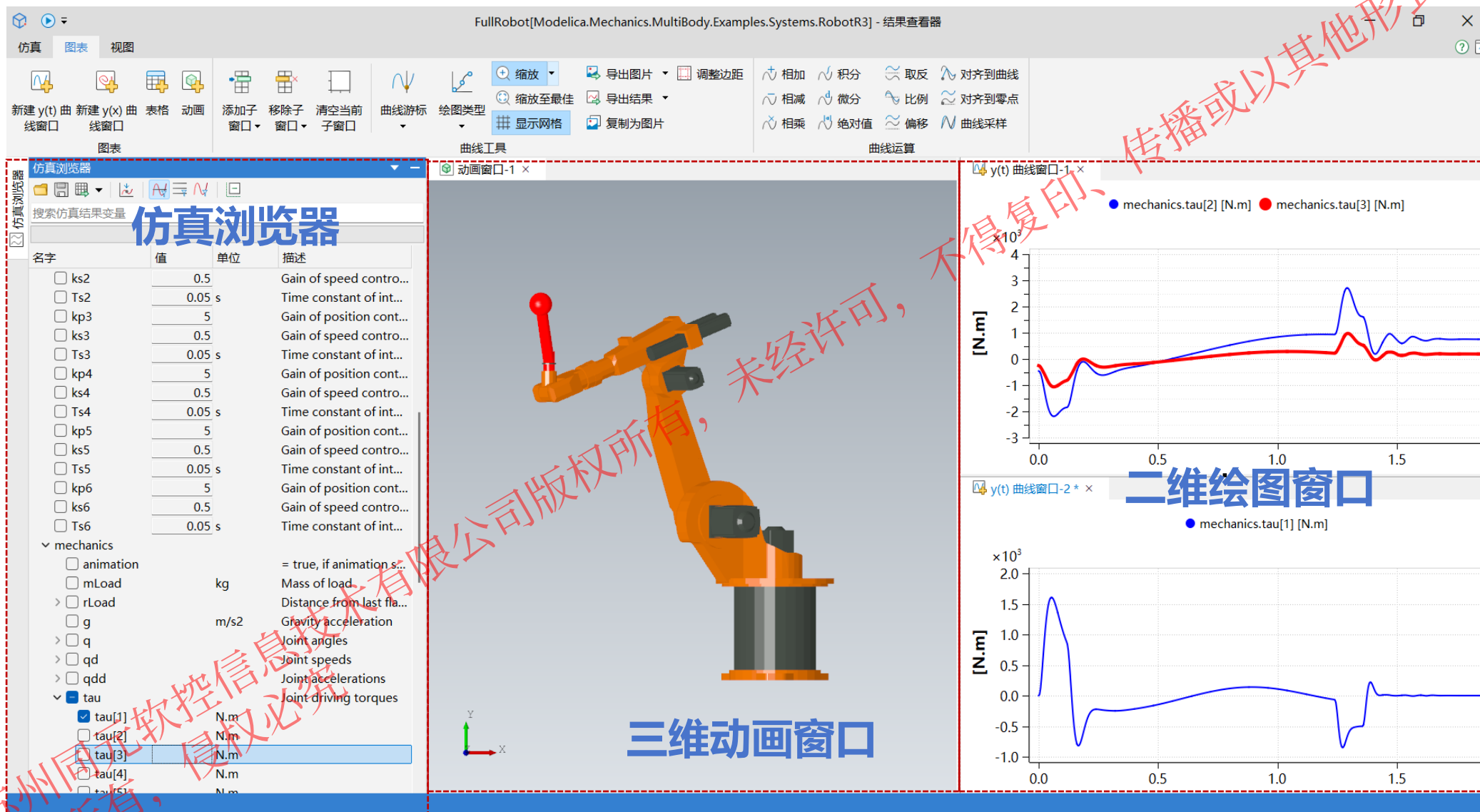
simulation can be terminated before reaching the stop time. The condition depends on the start and end positions of the joints, and on their reference speeds and reference accelerations. For current settings, the termination condition should indeed be fulfilled right before the simulation stops.

参数

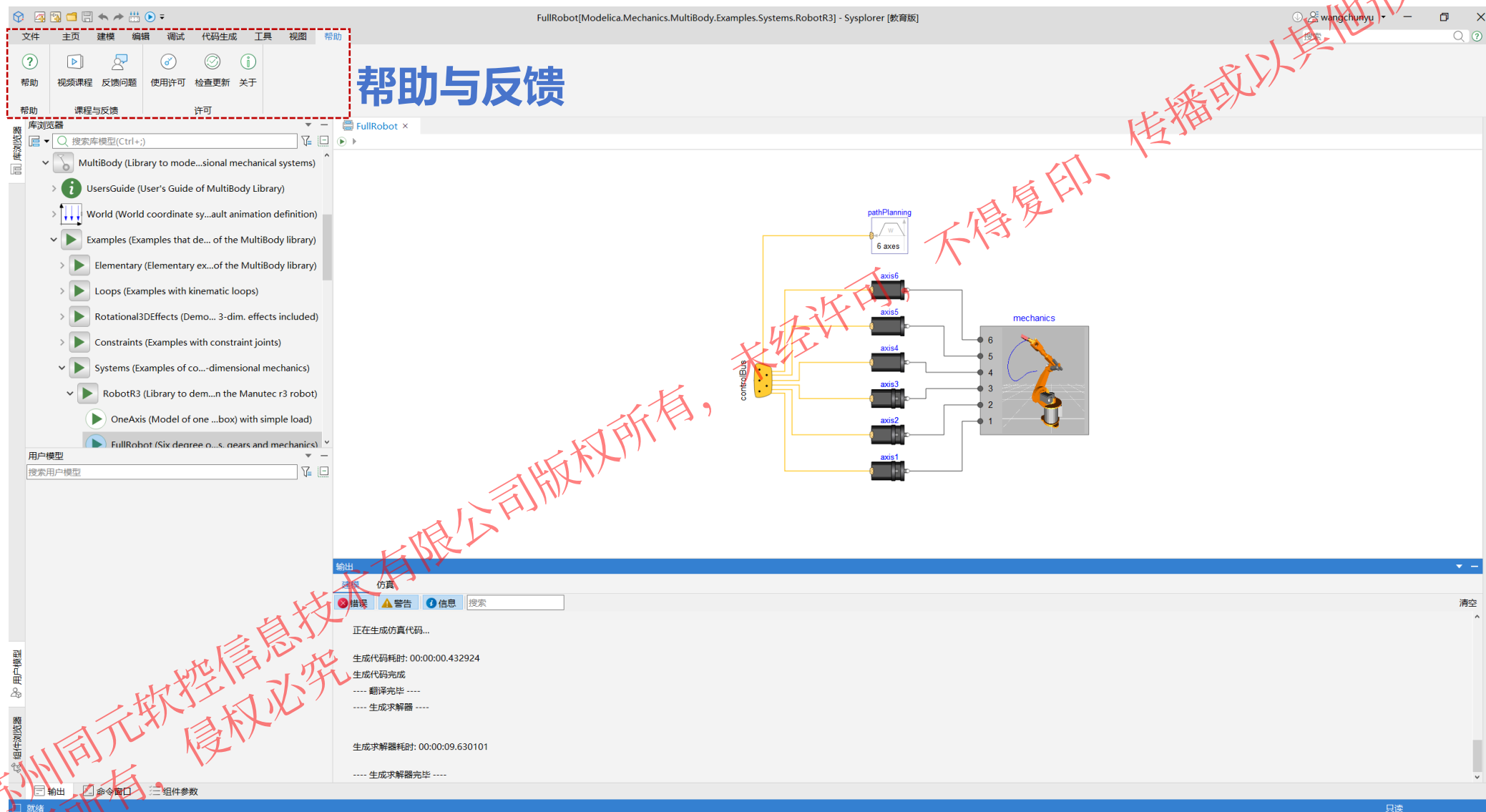
类型	名字	
Mass	mLoad	15

就绪 只读

1. MWORKS.Sysplorer软件概览



1. MWORKS.Sysplorer软件概览



1. MWORKS.Sysplorer软件概览

视频课程：提供Sysplorer软件使用视频课程



1. MWORKS.Sysplorer软件概览

反馈问题：可针对不同产品模块，反馈产品缺陷和提出功能建议等



建议类型

- 产品缺陷
- 功能建议
- 用户体验
- 一般咨询

产品模块

- 基本环境
- 外部C/C++接口
- 模型加密
- 二维动画
- 三维动画工具
- 导入导出FMU
- 导出S-function
- 导出Veristand
-

The image shows the '新建工单' (New Ticket) form in the MWORKS.Sysplorer software. The form is titled '我想解决的问题' (The problem I want to solve). It includes a dropdown menu for '建议类型' (Suggestion Type) with '1. 产品缺陷' (1. Product Defect) selected. Below this are input fields for '* 产品名称' (Product Name) with 'MWORKS.Sysplorer' and '* 产品模块' (Product Module) with '请选择模块' (Please select a module). There is a large text area for '* 问题描述' (Problem Description) with a placeholder '请输入' (Please enter) and a character count '0/500'. Below the text area are links for '产品信息' (Product Information) and '系统信息' (System Information). There is a button for '上传附件' (Upload Attachment). At the bottom, there is a section titled '通过以下方式提醒我工单进展' (Remind me of my ticket progress in the following ways) with an input field for '* 邮箱' (Email) with a placeholder '请输入' (Please enter) and a character count '0/50'. A '提交' (Submit) button is at the bottom right. A large red watermark is overlaid on the form, reading '未经授权，不得复印、传播或以其他方式使用' (Unauthorized, no reproduction, distribution or use in any other form).

@苏州同元软件信息技术有限公司
版权所有，侵权必究

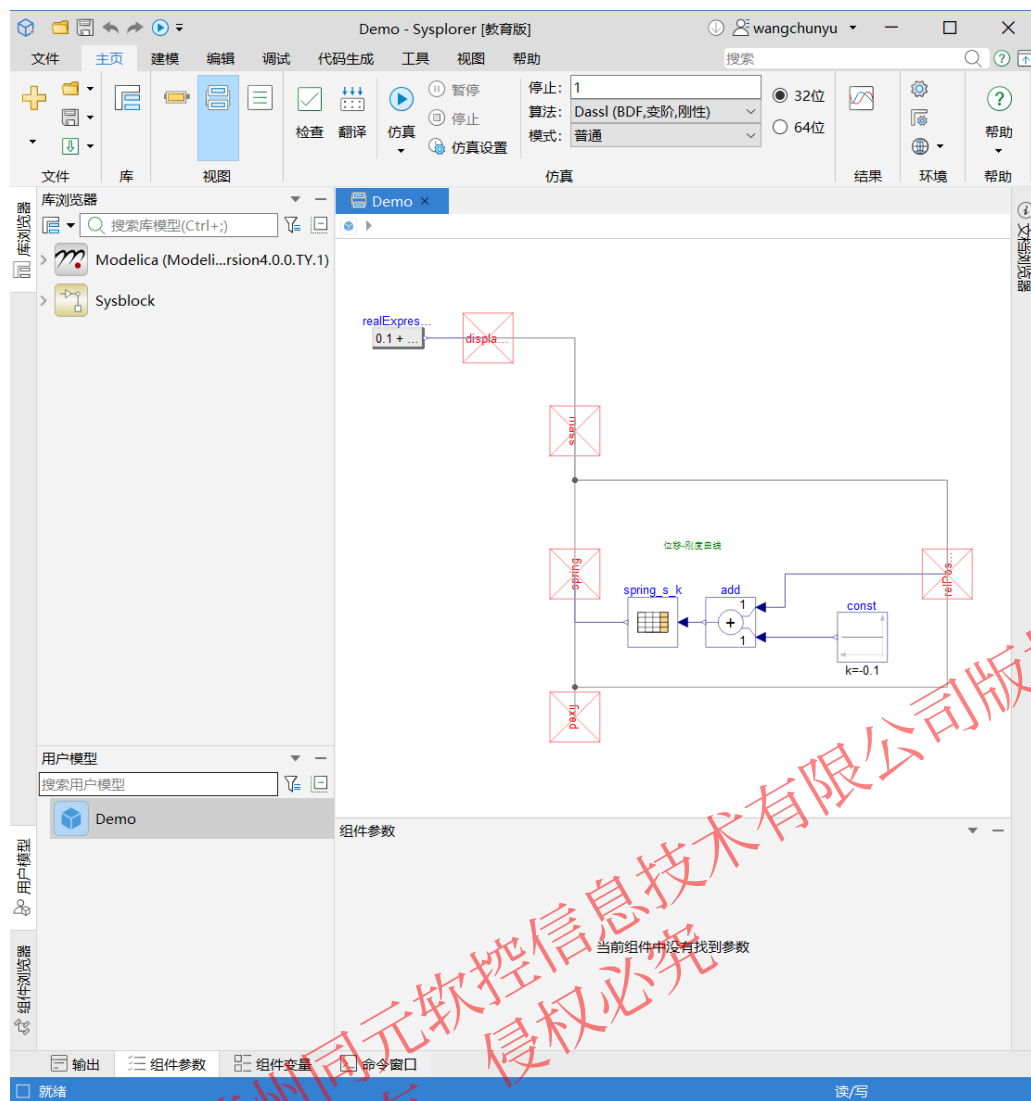
02

PART 02 ➔

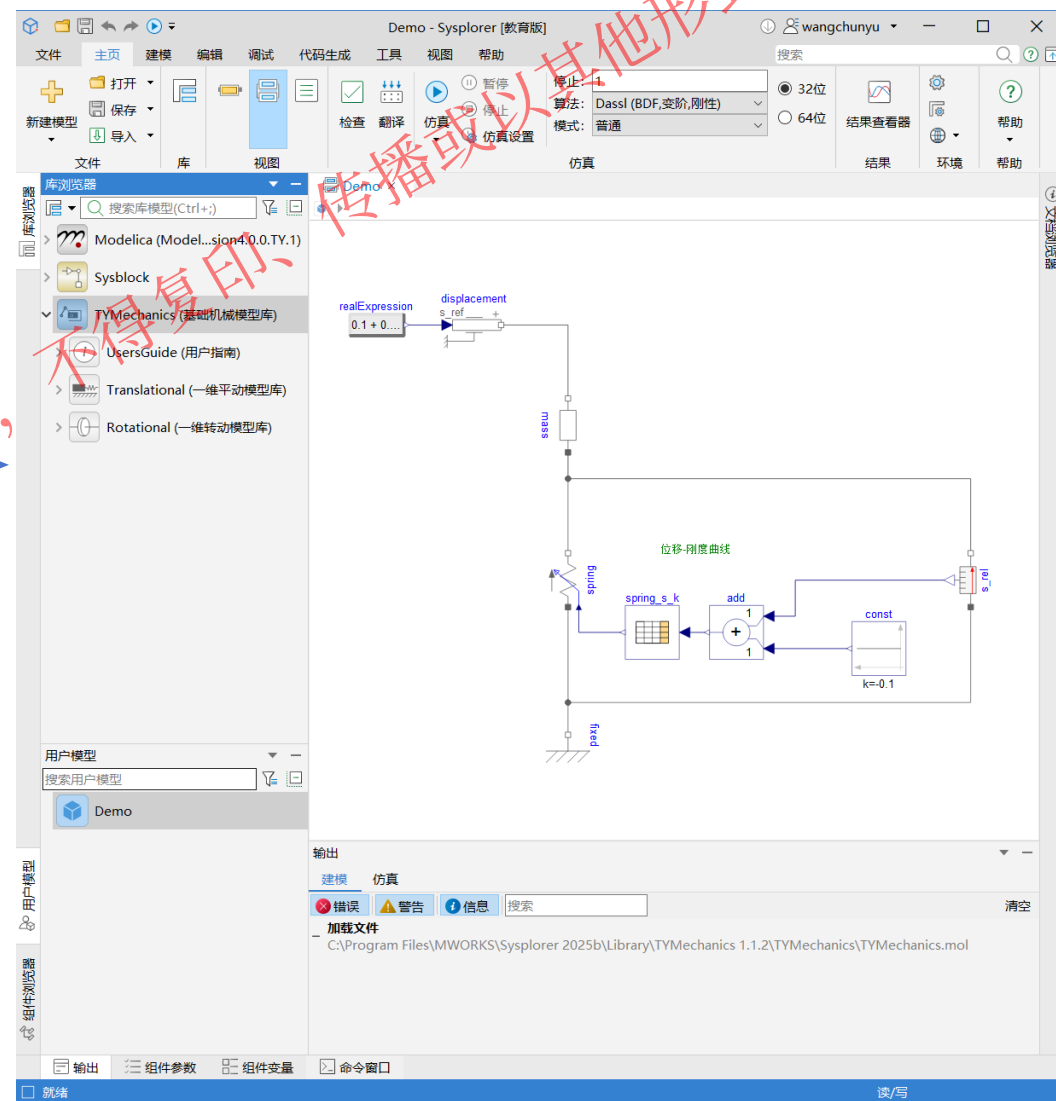
模型库加载

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

2. 模型库加载



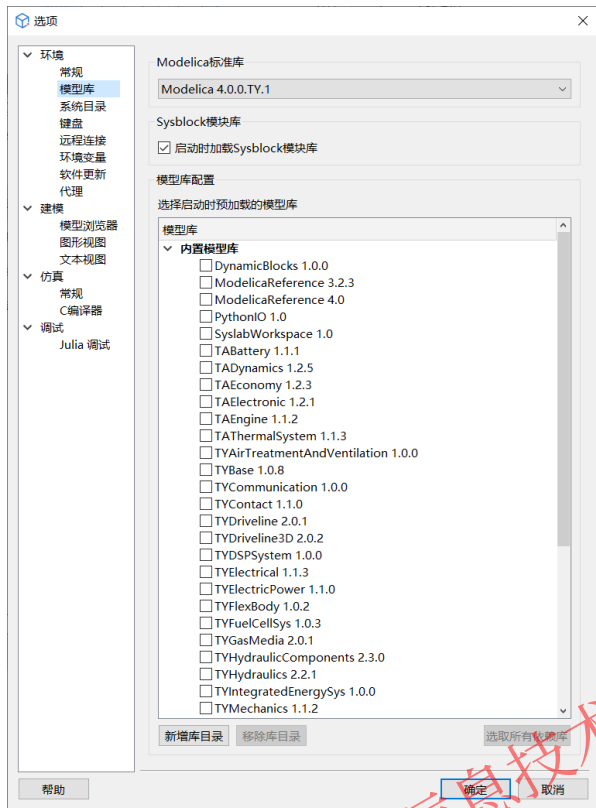
这是因为：
没有加载配套模型库



有哪些方式可以加载模型库？

2. 模型库加载

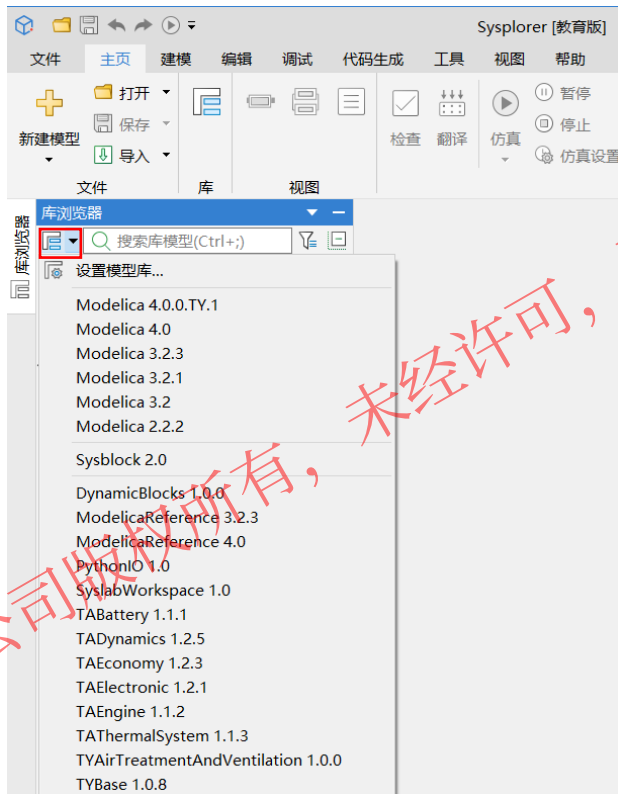
三种方式：



预加载模型库

每次打开软件自动打开预加载模型库

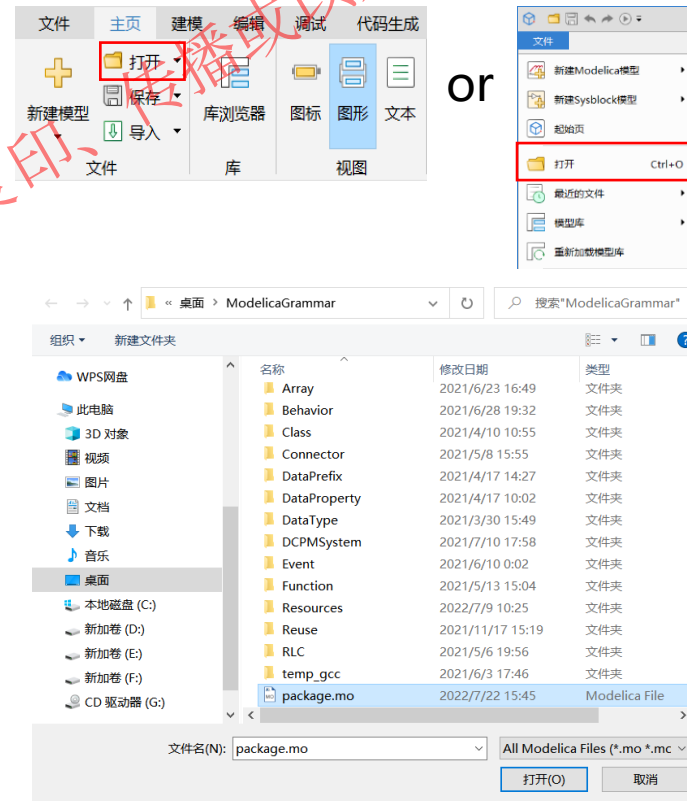
模型库不可修改



打开模型库

临时打开内置模型库

模型库不可修改



打开模型库

临时打开自定义模型库

模型库可修改

CONTENTS

目录 ➔



01 MWORKS.Sysplorer软件概览



02 模型库加载



03 **拖拽建模：电子阀流量控制系统**



04 文本建模：电阻



05 模型发布



06 MWORKS.Sysplorer与Simulink之间的差异



07 Modelica语法概览

03

PART 03 ➔

拖拽建模：电子阀流量控制系统

- 图形化建模
- 仿真求解设置
- 结果后处理

@苏州工业园区信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

3.拖拽建模：电子阀流量控制系统



理论分析

需求分析、机理分析、
系统分解、组件分析



模型构建

新建模型、文本建模、
拖拽组件、接口连接、
参数设置、图标绘制



仿真求解

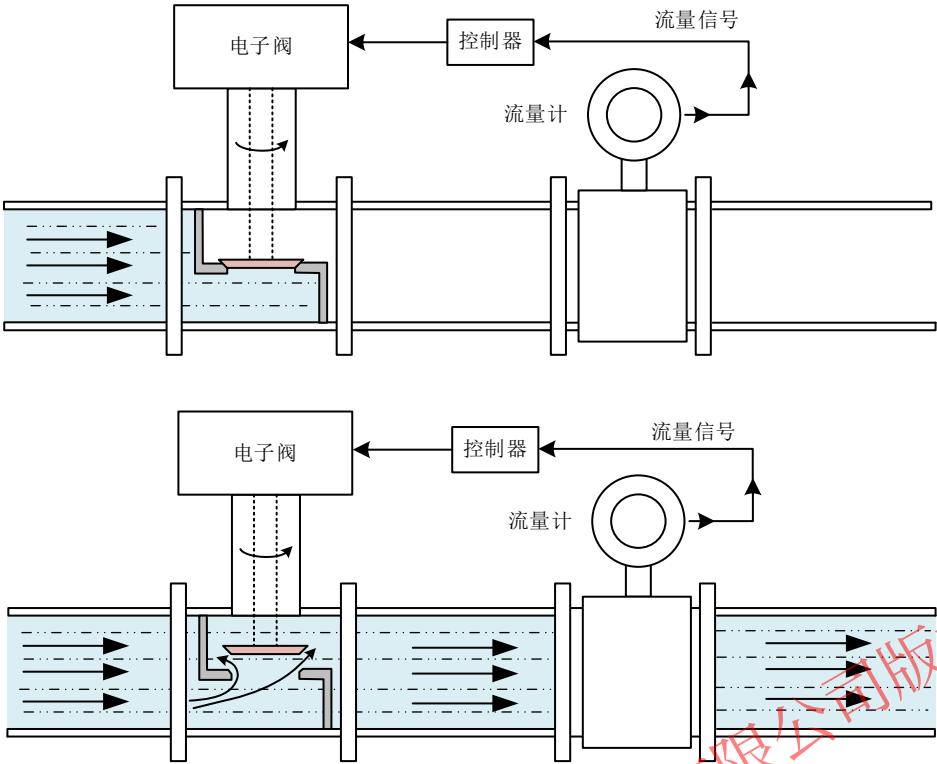
模型检查、模型翻译、
仿真设置、运行仿真



结果处理

曲线查看、后处理、
二维动画、三维动画

3.拖拽建模：电子阀流量控制系统-理论分析



电子阀的打开/关闭可以控制管道中的液体通断，流量计可以检测管道中的流量信号并反馈给控制器来控制电子阀开口的大小，从而实现对管路流量的控制。

简化原理：
阀口开度通过电子阀带动转动的螺杆控制，螺杆转动范围为0~360°，对应管道的流量为0~50L/min。

系统组成	组成	组件需求
控制器	PID控制	比例模块、积分模块、微分模块
电子阀	电路部分	电学组件
	机械结构	机械组件
流量计	插值表	插值表

@苏州工业园区
版权所有，

3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

架构设计

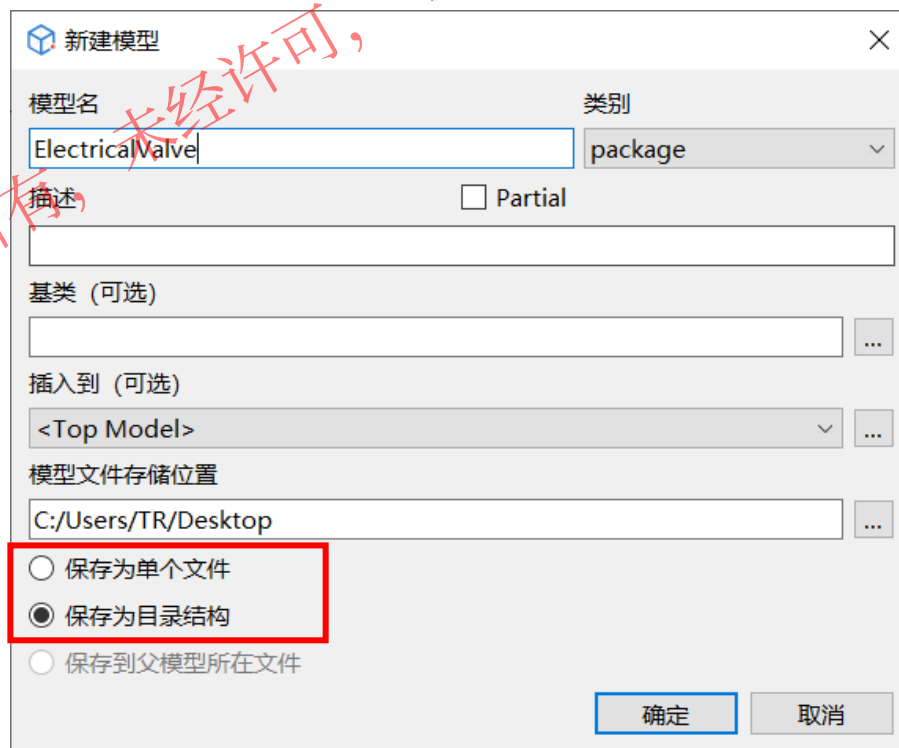
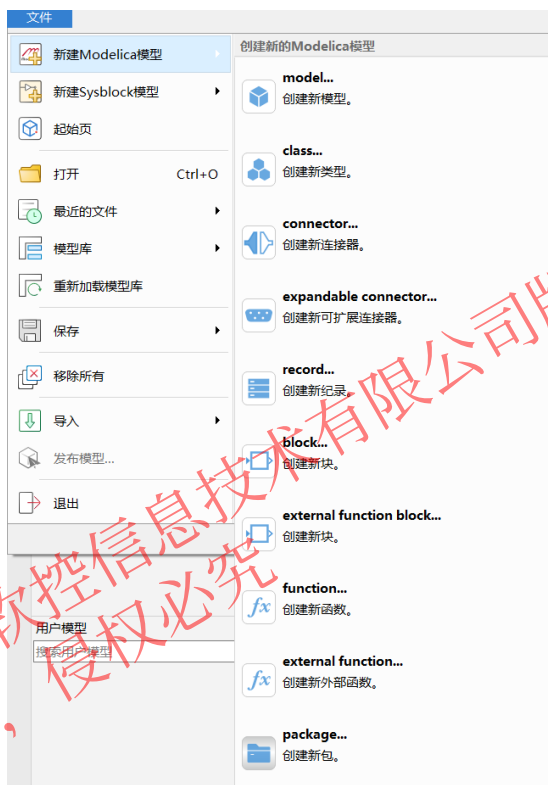
组件建模

系统构建

设置参数

操作步骤：

- 点击“文件>新建>package...”，弹出“新建模型”对话框
- 填写模型名为“ElectricalValve”，描述为“电子阀模型库”
- 点击确定，即可完成模型创建



@苏州同元软控信息技术有限公司版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

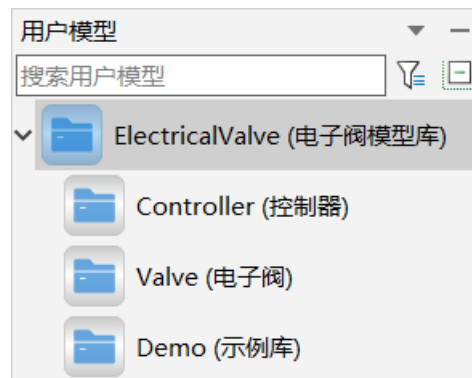
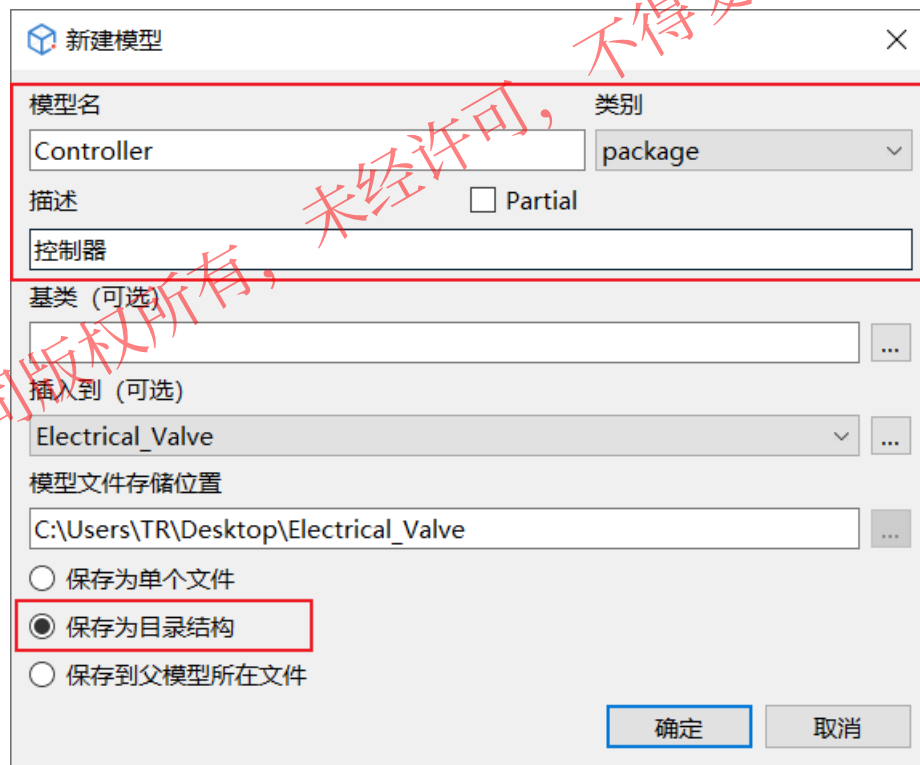
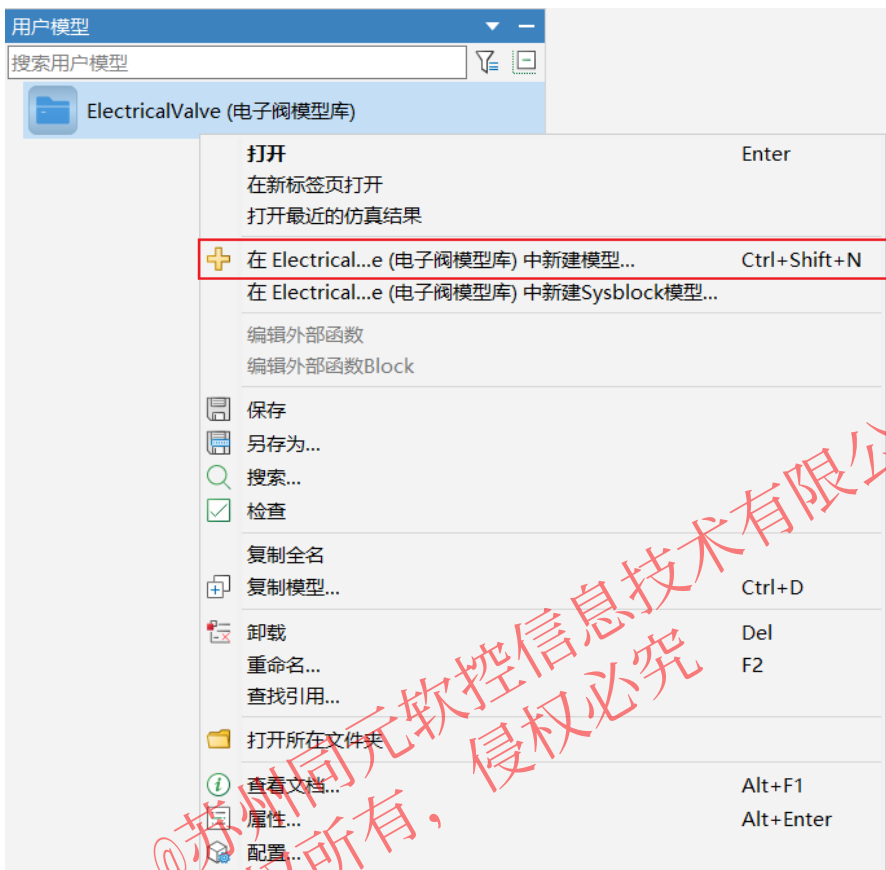
架构设计

组件建模

系统构建

设置参数

- 操作步骤：**
- 右键点击所建立的ElectricalValve(电子阀模型库)，在弹出的对话框中点击新建模型；
 - 更改类别为“package”并定义模型名及相应描述，选择保存为目录结构，点击确定。



模型库架构

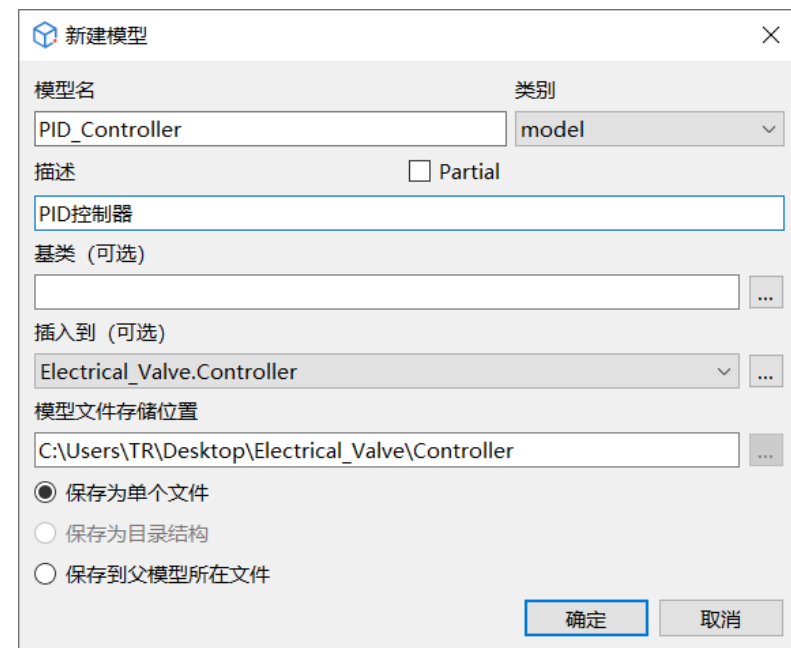
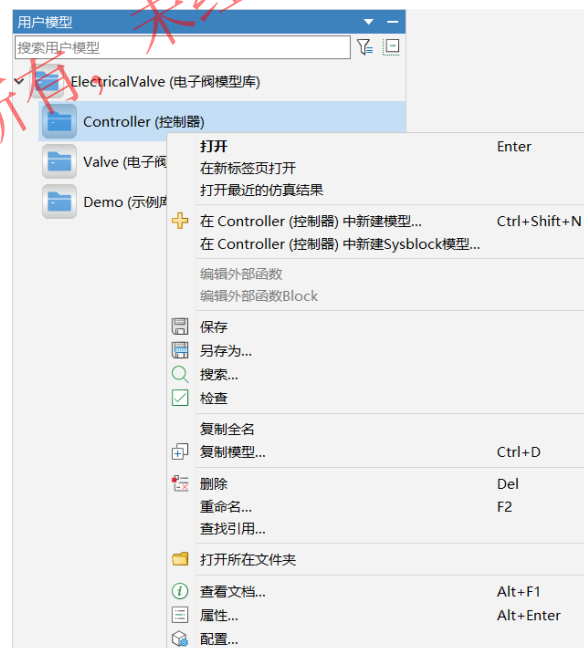
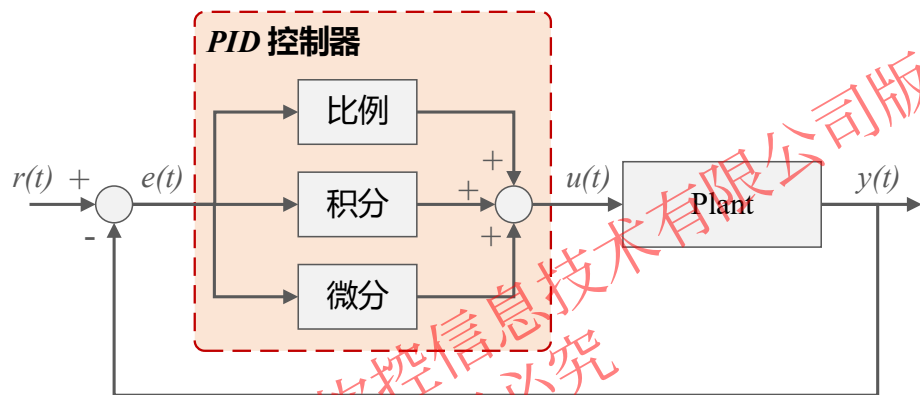
3.拖拽建模：电子阀流量控制系统-模型构建



PID控制器

在当今应用的工业控制器中，有半数以上采用了PID或变形PID控制器。PID控制的价值取决于它们对大多数控制系统的广泛适用性，体现在：

- **原理简单**，应用方便，参数整定灵活
- **适用性强**，特别是当被控对象的数学模型未知，而不能使用解析设计方法时，PID控制就显得特别有用
- **鲁棒性强**，控制品质对受控对象的变化不太敏感，如受控对象受外界扰动时，无需经常改变控制器参数或结构

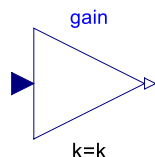
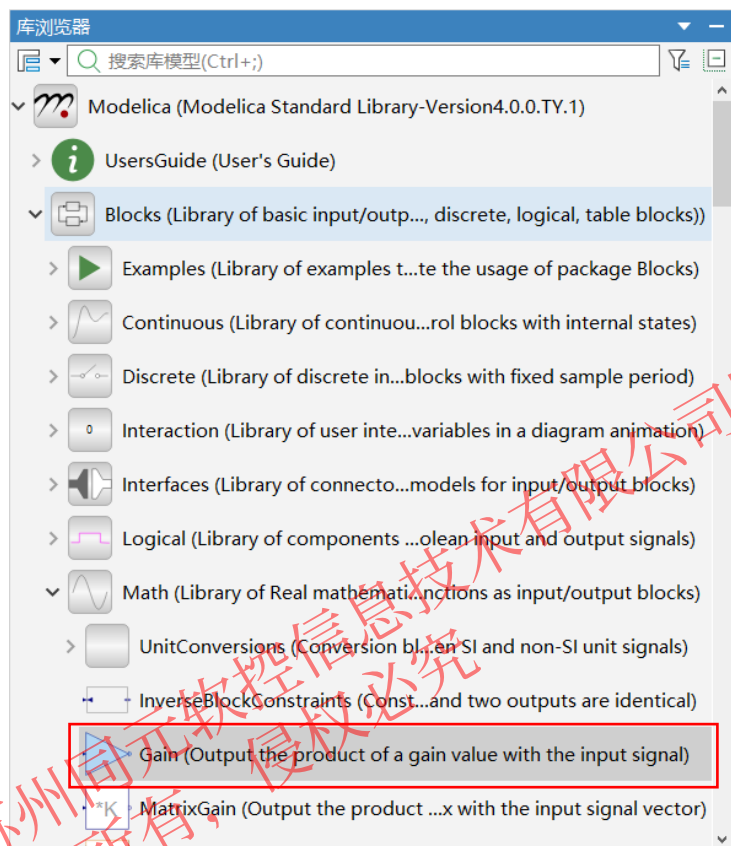


@苏州同元软控信息技术有限公司版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建



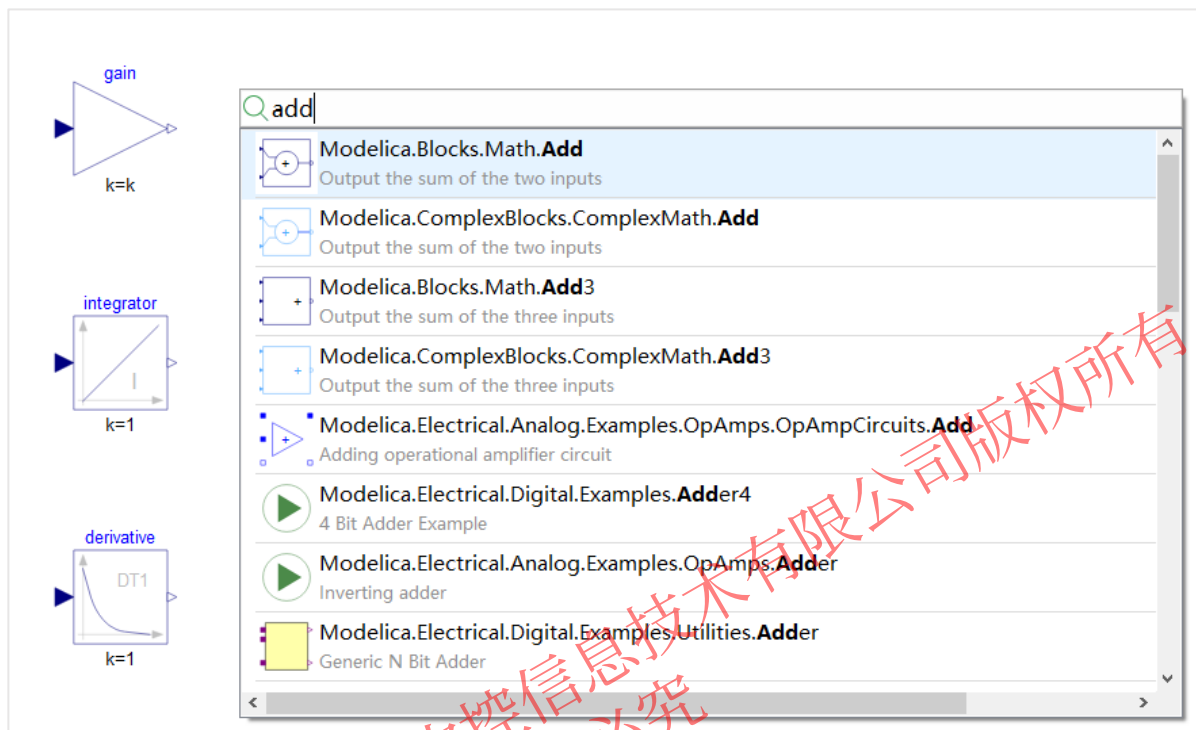
- 操作步骤：**
- 选中Modelica标准库模型Modelica.Blocks.Math.Gain
 - 左键长按拖拽至右侧图形视图中



3.拖拽建模：电子阀流量控制系统-模型构建



操作步骤： 双击视图创建模块功能



双击搜索组件.mp4

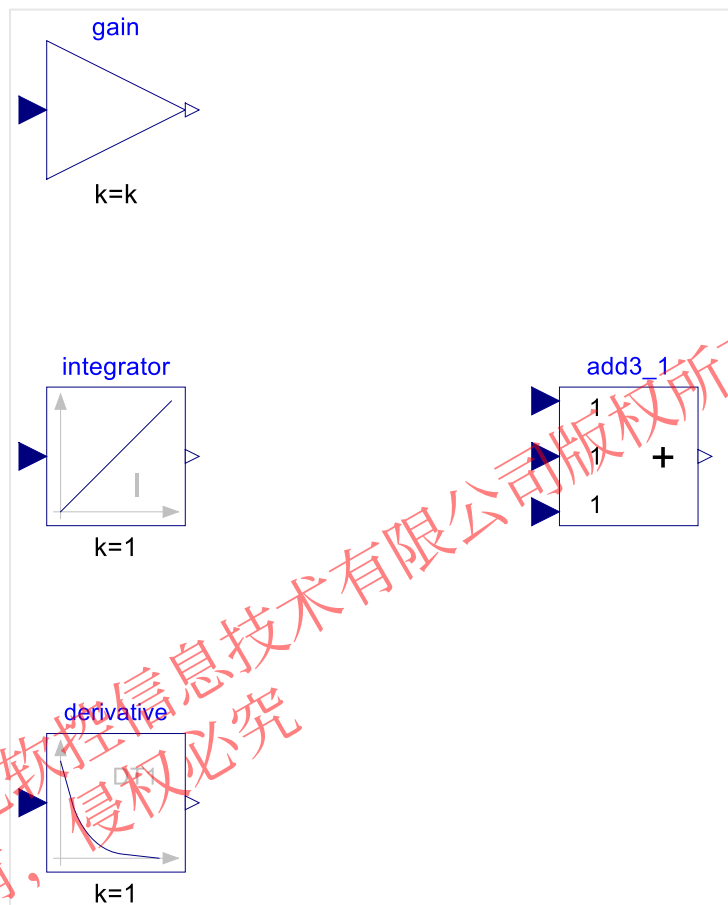
在图形视图界面的任意空白位置鼠标左键双击，弹出该功能的输入框。用户在输入框中输入要创建的模型或者注解。

注：该功能不适应于不可编辑模型

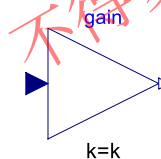
3.拖拽建模：电子阀流量控制系统-模型构建



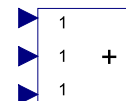
操作步骤： 同 “add3”模型，拖入PID模型所需要的其他组件，并根据物理拓扑关系适当排布。



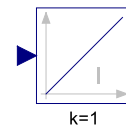
组件路径如下：



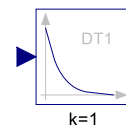
Modelica.Blocks.Math.Gain



Modelica.Blocks.Math.Add3



Modelica.Blocks.Continuous.Integrator



Modelica.Blocks.Continuous.Derivative

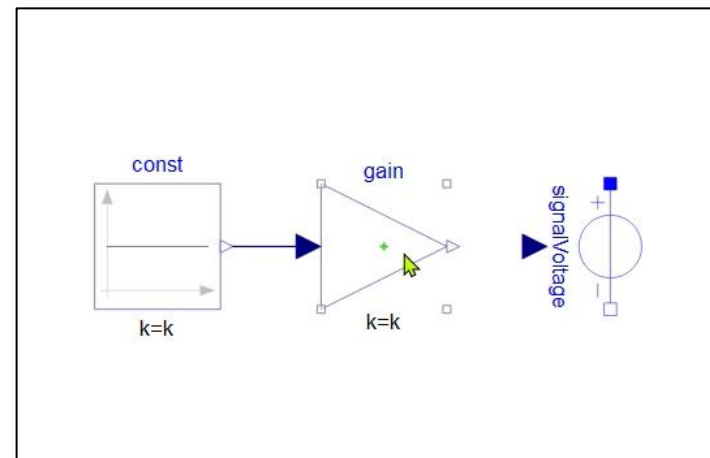
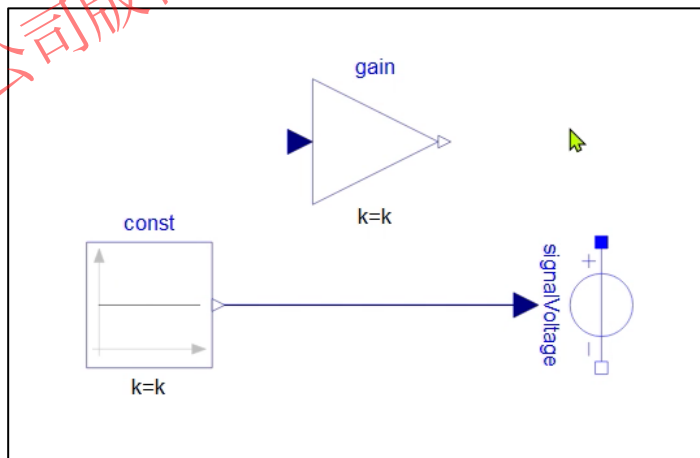
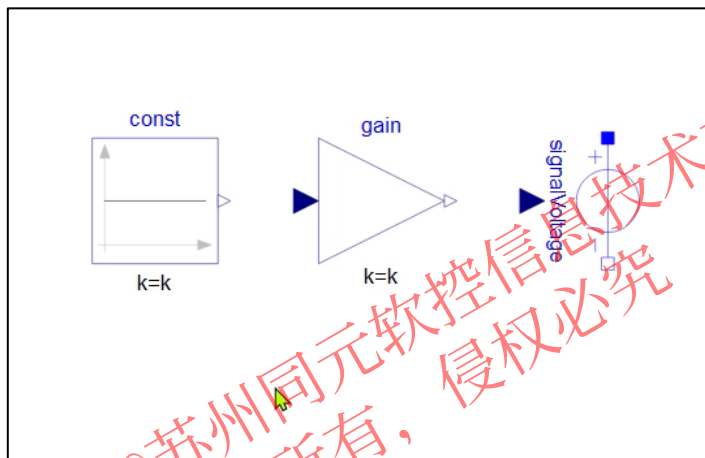
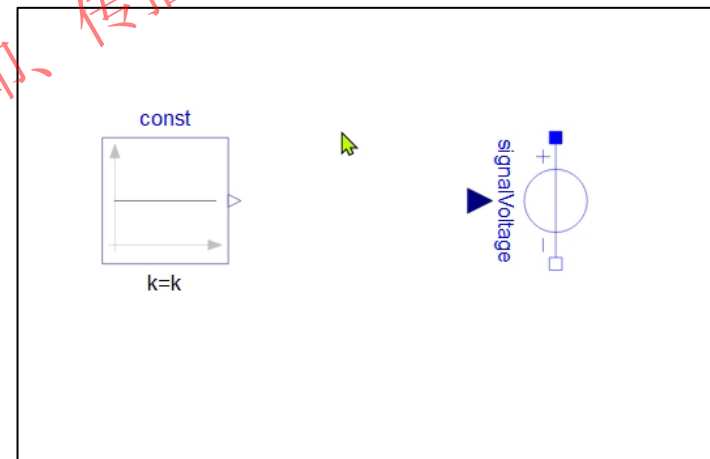
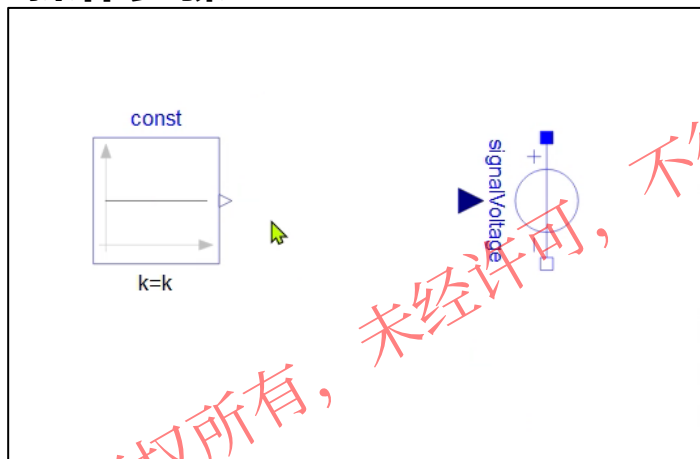
@苏州同元软控信息技术有限公司版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建



操作步骤：

- 拖拽连接组件；
- 点击端口连接组件；
- 连接线自动布局；
- 移动过程中自动连接；
- 保留连接线及快速连接。



@苏州同元软控信息技术有限公司
版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

新建模型

架构设计

拖拽组件

组件建模

连接组件

系统构建

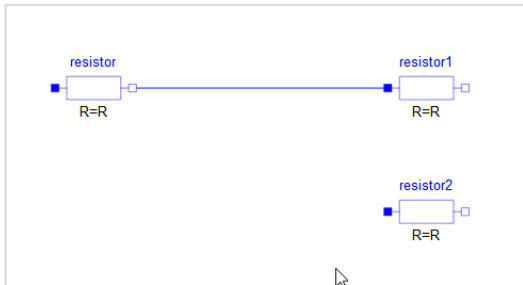
设置参数

设置参数

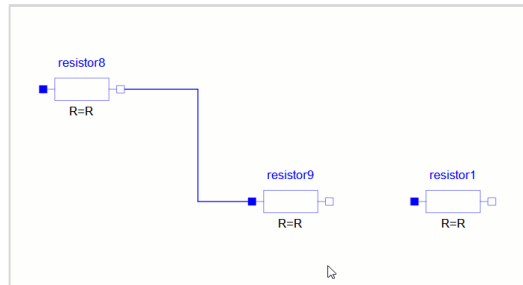
绘制图标

编辑文档

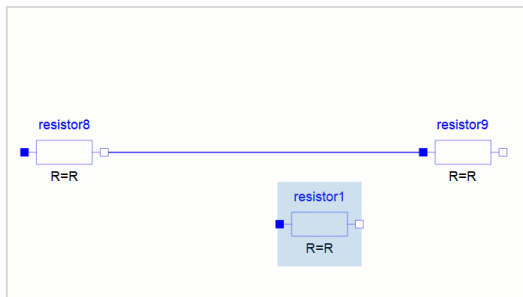
Sysplorer 2026a 更新：优化物理建模连线机制



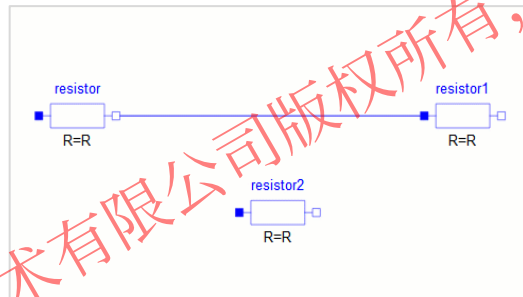
创建连线显示汇合点



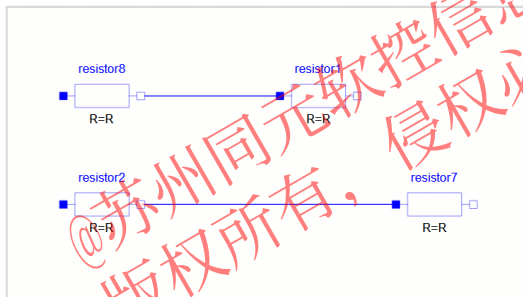
连线悬空&重连到端口



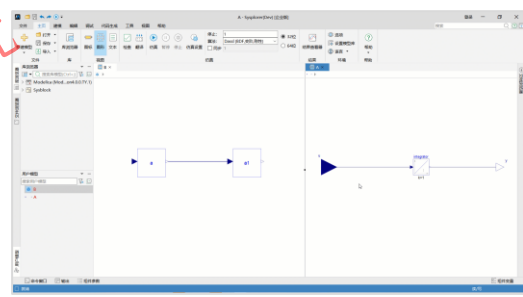
插入组件&删除重连



移动组件连线避障

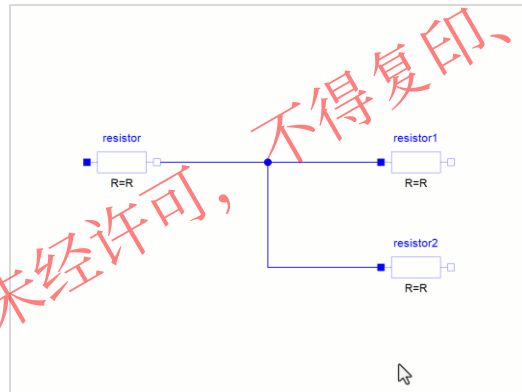


悬空连线连接到连线

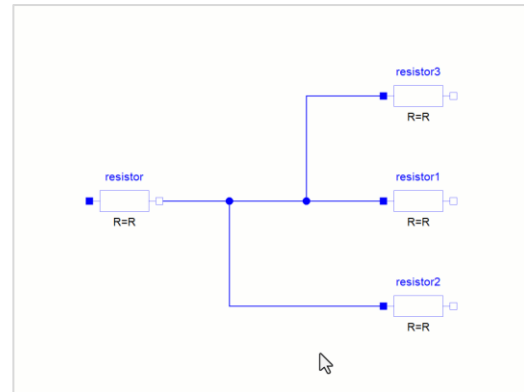


组件重命名引用连线同步

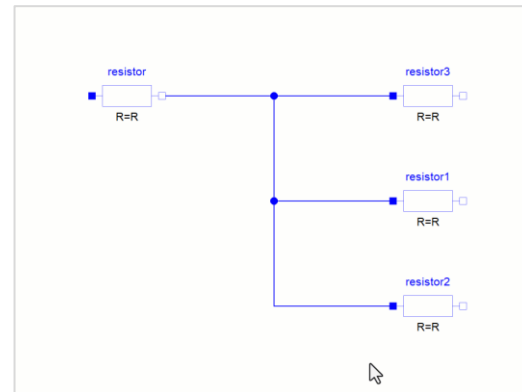
选项 -> 图形视图 -> 勾选 使用Sysblock 连线方式



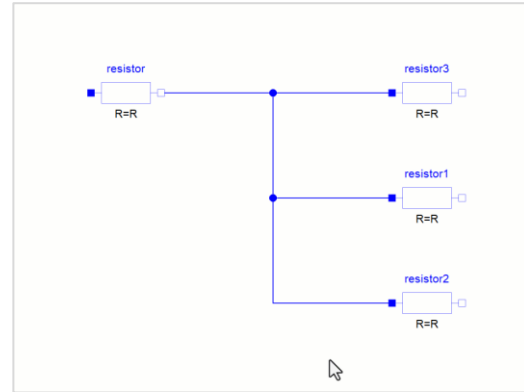
分段选中连线



分段删除连线



分段移动连线

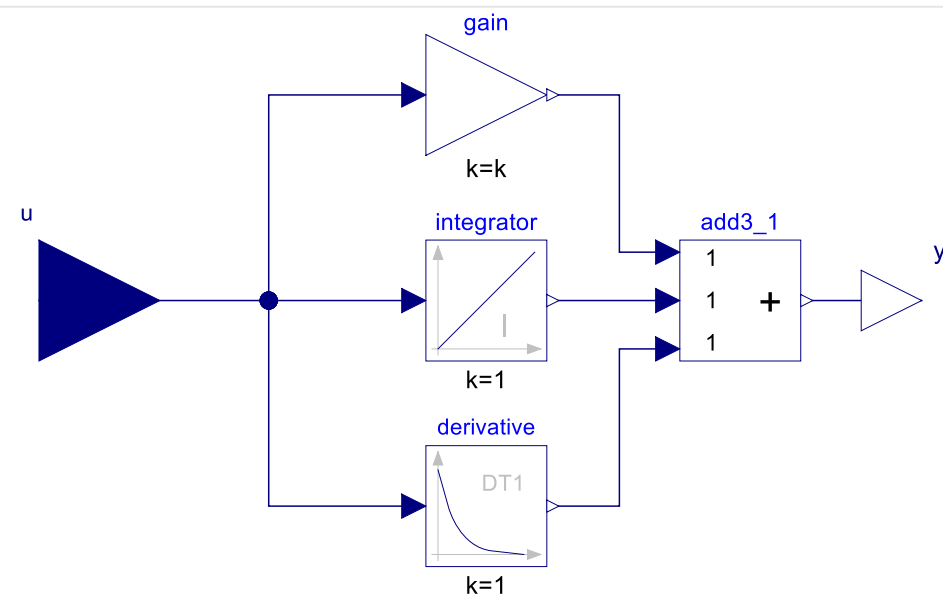
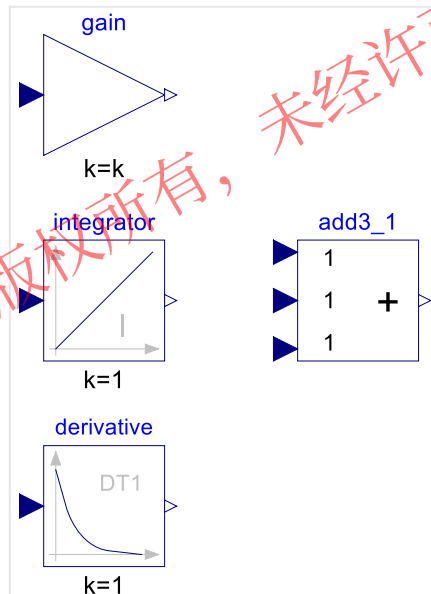
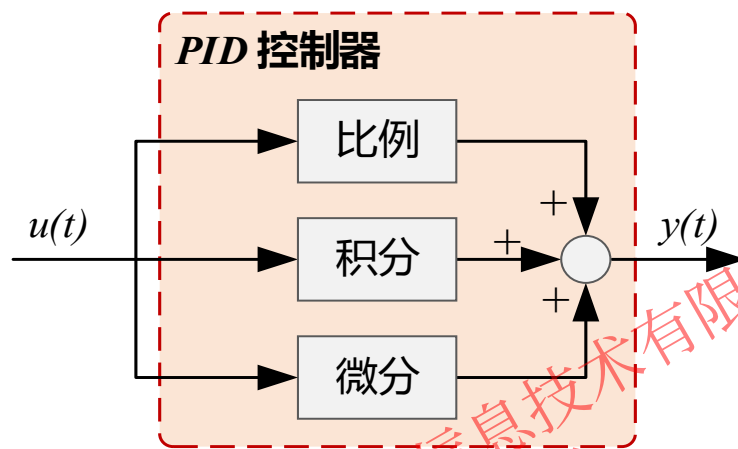


移动组件仅影响关联分段

3.拖拽建模：电子阀流量控制系统-模型构建



- PID控制器-模型构建**
- 在标准库中寻找增益、积分、微分模块和加法器模块，并按照原理连接；
 - 封装组件，创建外部接口，使得该组件可被复用。

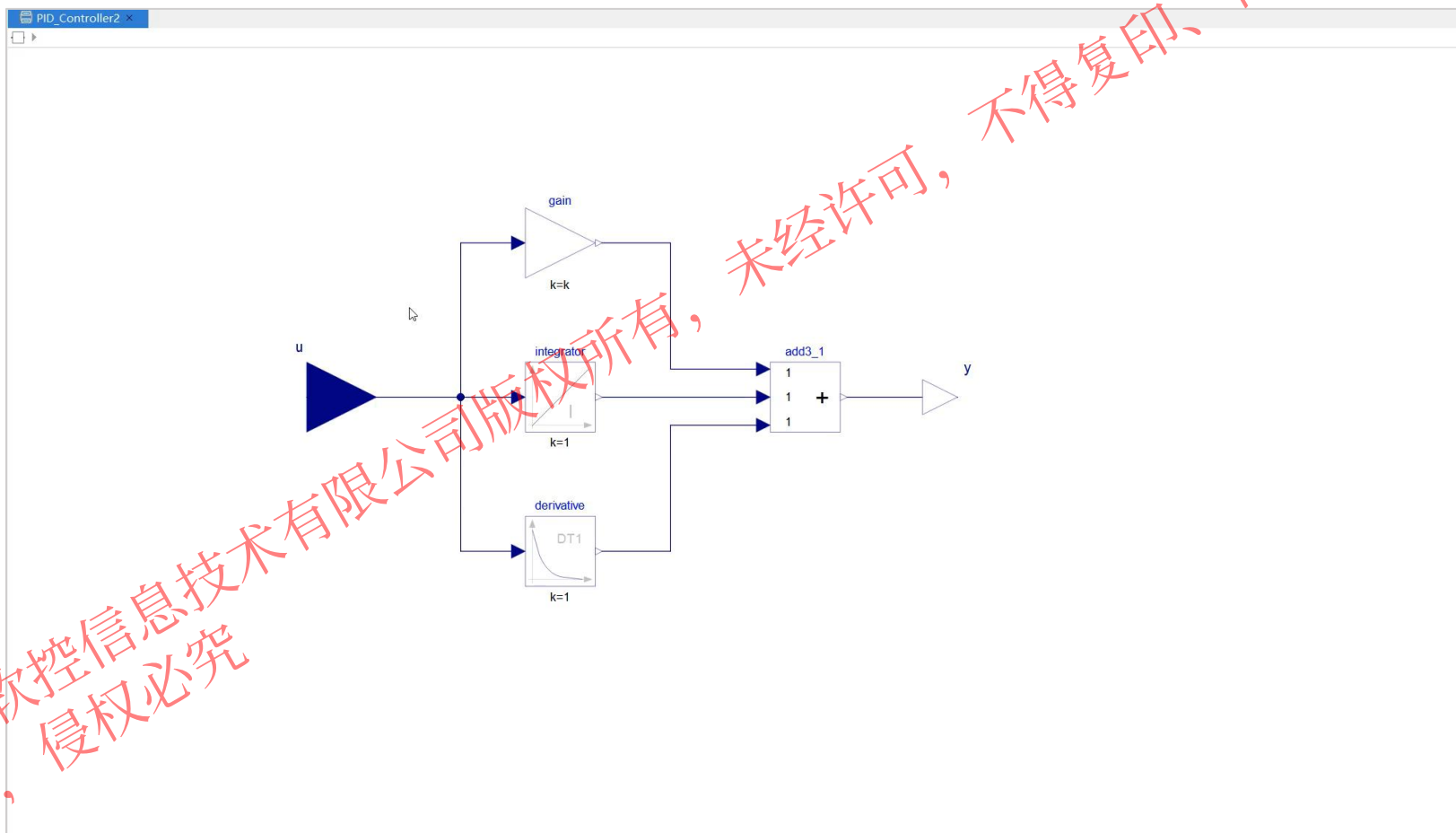


@苏州同元软控信息技术有限公司版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建



Sysplorer 2026a 更新：支持子系统封装和逆封装



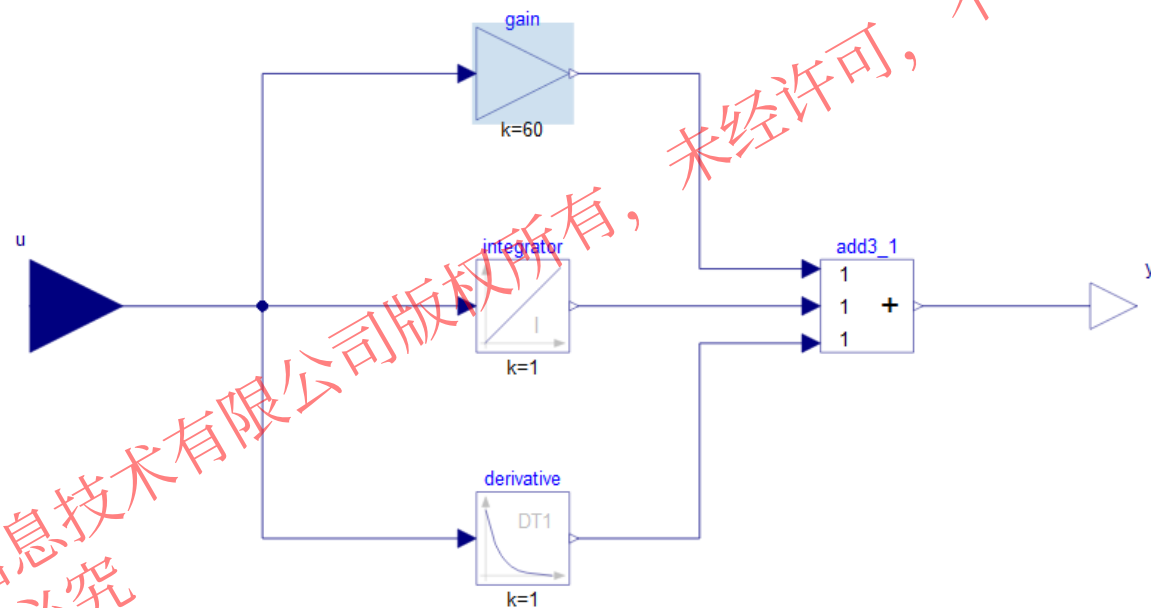
@苏州同元软控信息技术有限公司版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建



操作步骤：根据系统属性，设置模型参数。

如：PID控制模块的增益系数为60，选中组件gain，在组件参数面板中选中参数“k”的值，修改为60。



组件参数			
常规			
参数			
k	60	1	Gain value multiplied with input signal

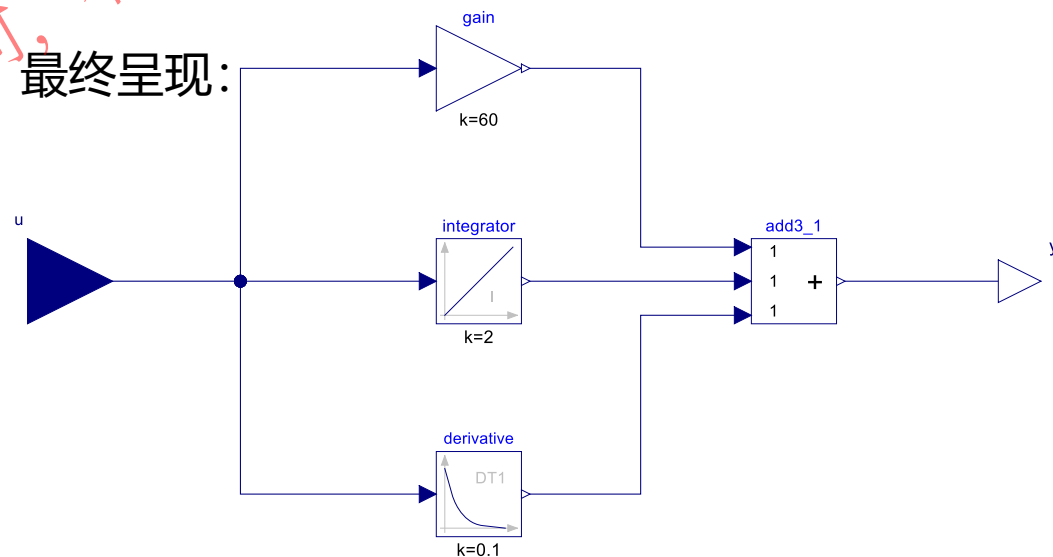
3.拖拽建模：电子阀流量控制系统-模型构建



操作步骤：参考组件“gain”的参数“k”的修改方法，修改其余组件参数

组件	参数	参数值	量纲
gain	k(常值大小)	60	/
integrator	k(常值大小)	2	/
derivative	k(常值大小)	0.1	/

最终呈现：

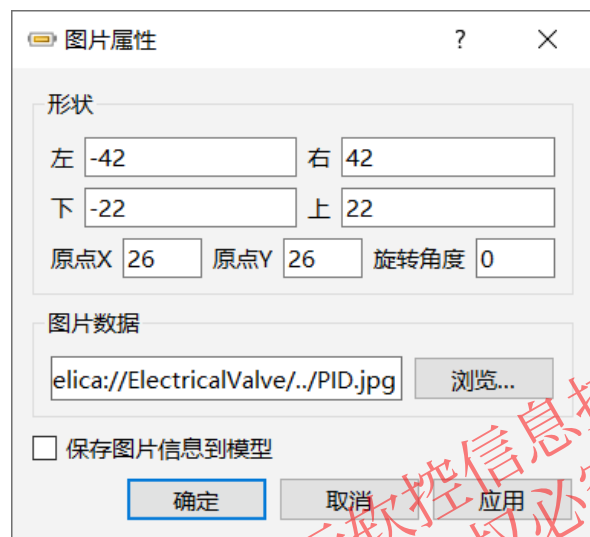


@苏州同元软控信息技术有限公司版权所有，侵权必究

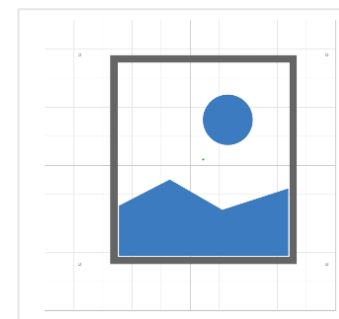
3.拖拽建模：电子阀流量控制系统-模型构建



模型构造完成之后，可以直接插入本地图片作为模型图标，以满足更为丰富的外观样式

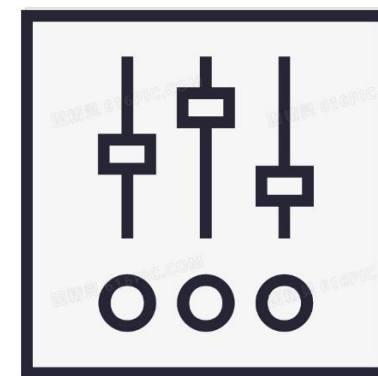


➤ **未勾选**“保存图片信息到模型”，更改模型/图片位置后，图标不显示



图片查找不到

➤ **勾选**“保存图片信息到模型”，更改模型/图片位置后，图标正常显示



正常显示

@苏州同元软控信息技术有限公司版权所有，侵权必究

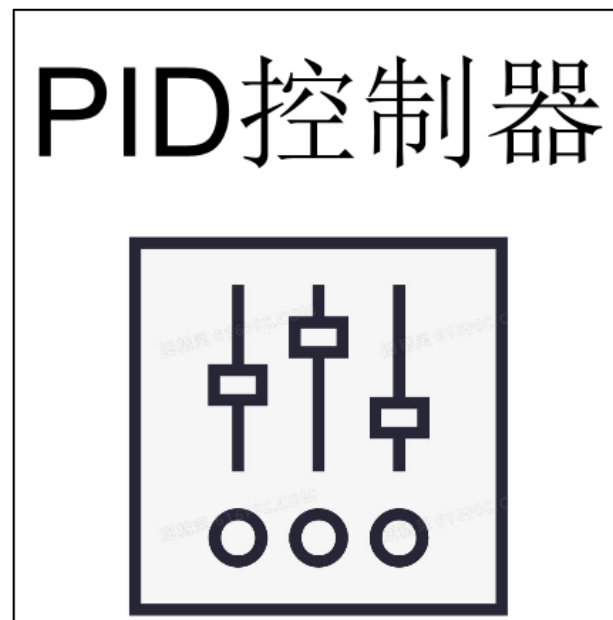
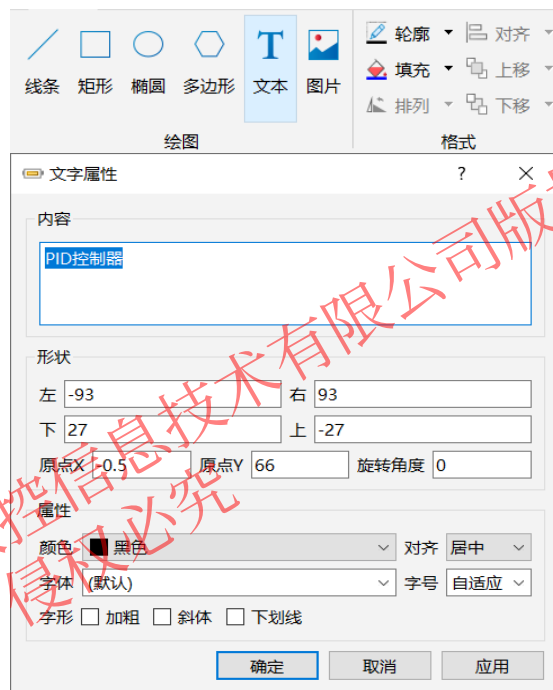
3.拖拽建模：电子阀流量控制系统-模型构建



除直接插入图片外，还可通过图标绘制可以让模型更加**直观**

操作步骤：

- 选择**图标**界面
- 使用 “编辑” 中的基本图元进行绘制
- 直接添加文本，可以修改文本字体等

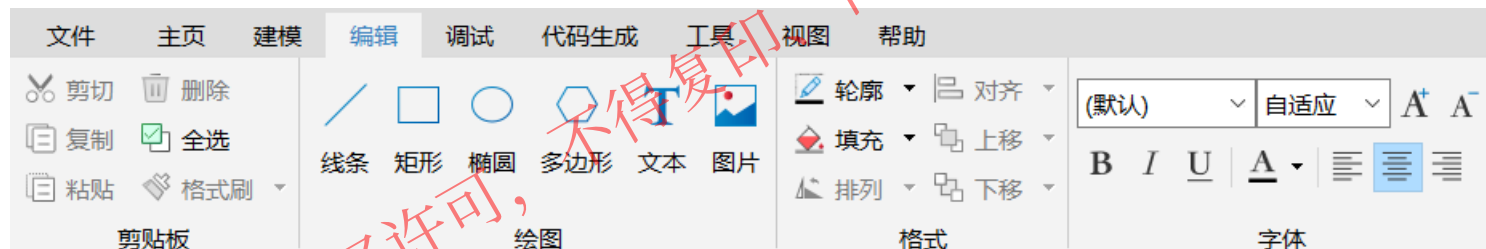
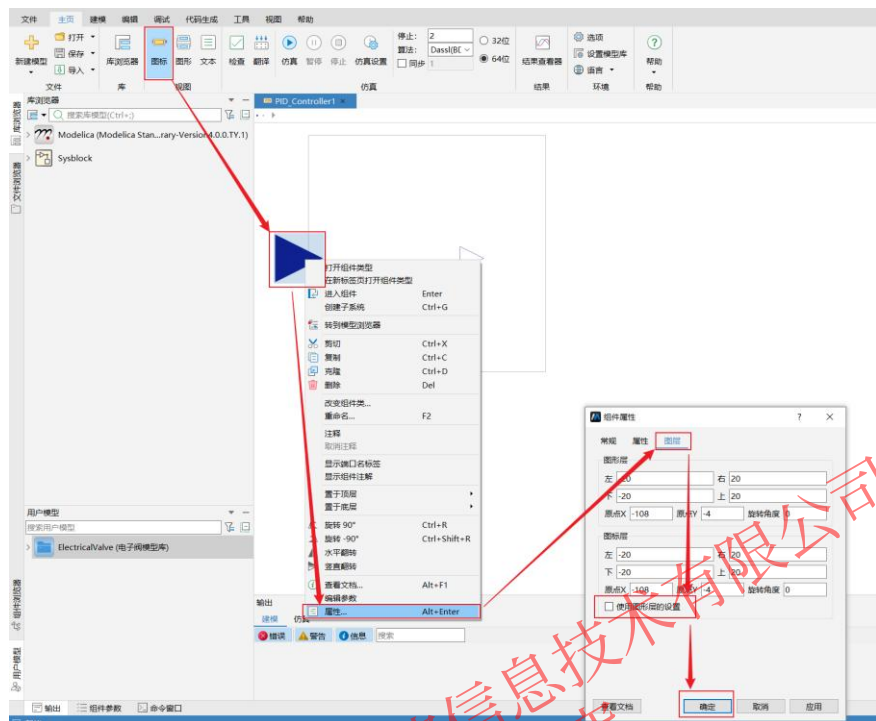


给模型穿上“漂亮的衣服”

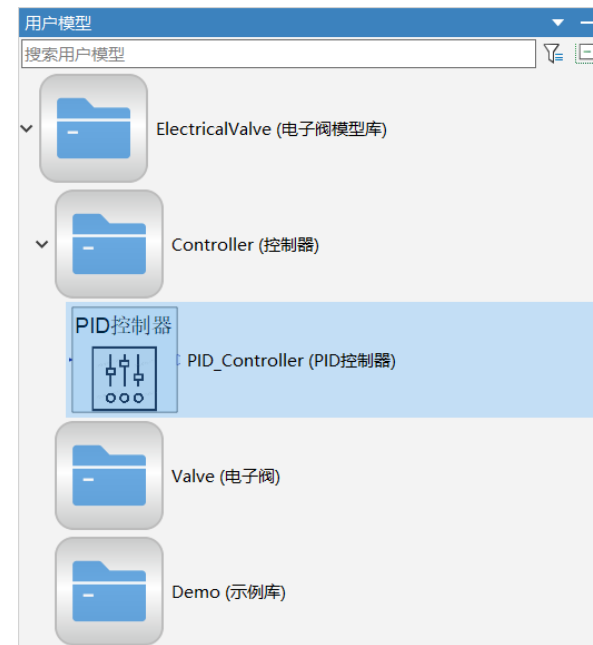
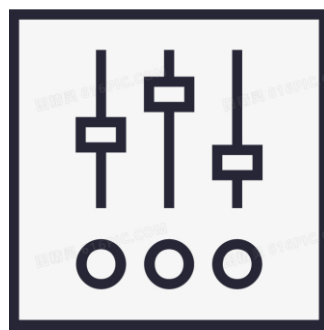
3.拖拽建模：电子阀流量控制系统-模型构建



PID控制器-图标设计



PID控制器



@苏州同元软控信息技术有限公司版权所有，侵权必究

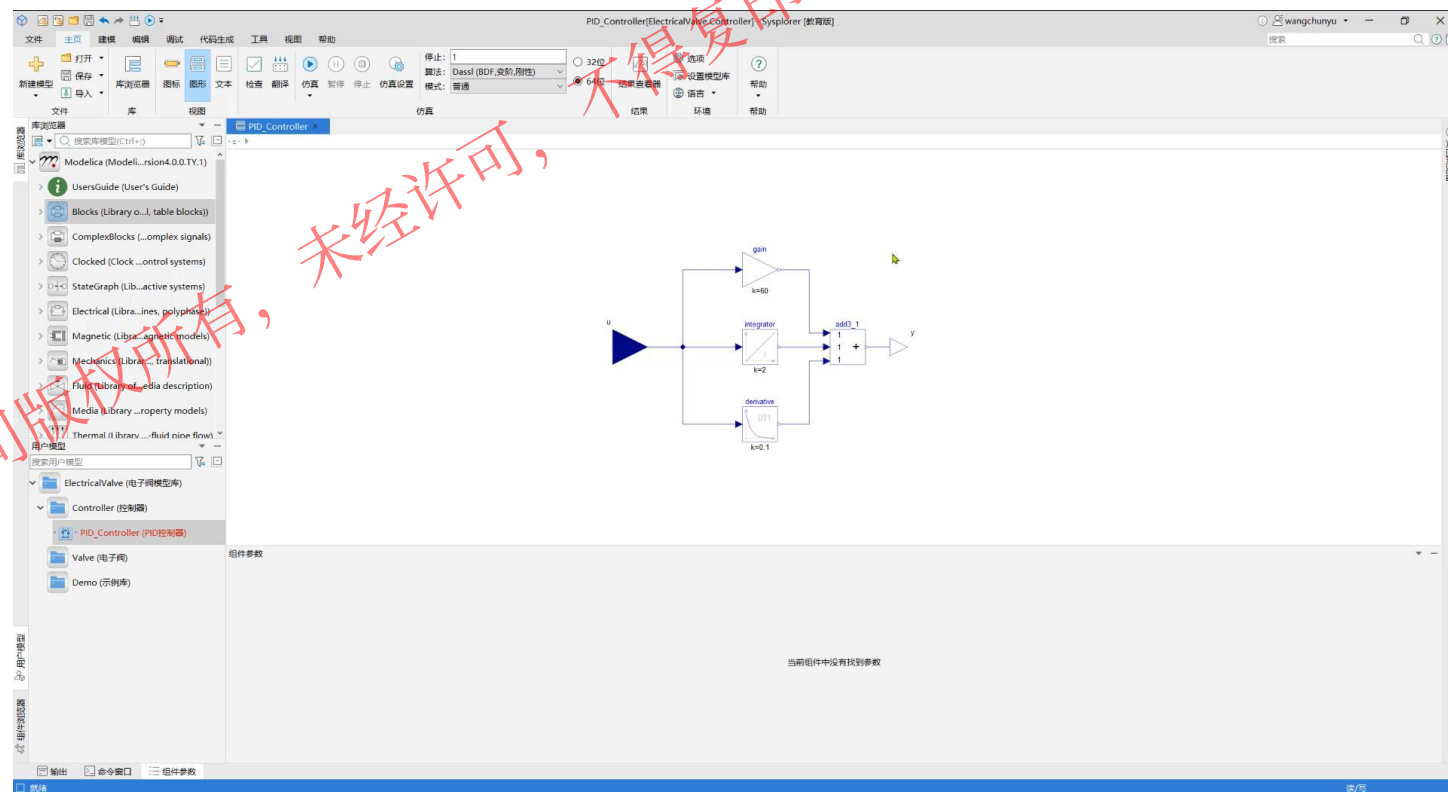
3.拖拽建模：电子阀流量控制系统-模型构建



模型构造完成之后，通过编辑文档可以让模型更加易用

操作步骤：

- 在模型对应的文档浏览器中进入编辑模式。
- 在文档浏览器中插入文字。
- 在文档浏览器中插入图片。
- 在文档浏览器中插入链接。



文档浏览器编辑.mp4

@苏州同元软控信息技术有限公司
版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

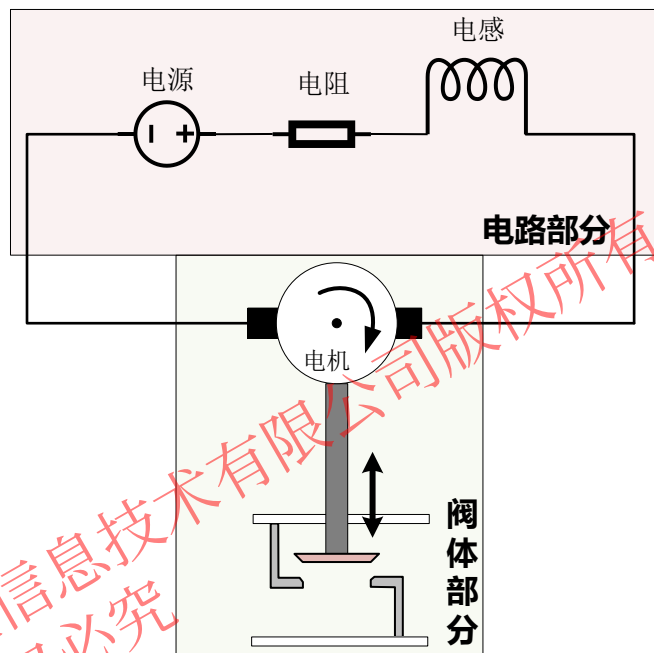
架构设计

组件建模

系统构建

设置参数

- 简化电子阀：**
- 简化后的电子阀可以分为电路部分和阀体部分；
 - 在Sysplorer中可以分为两个组件分别构建组件模型。



阀芯转动范围为0~360°

模型分析：

➤ 电路部分

需要电源、电阻及电感组件来构建电路整体的物理关系；

➤ 阀体部分

需要电机驱动阀芯部分转动；
阀体部分需要转动惯量组件模拟；
阀芯转动需要限位0~360°满足物理要求。

3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

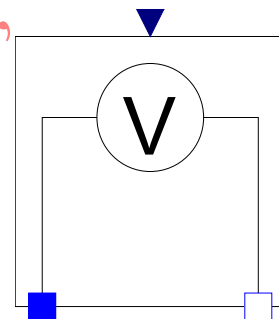
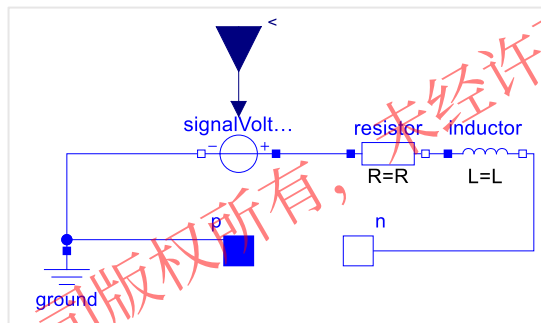
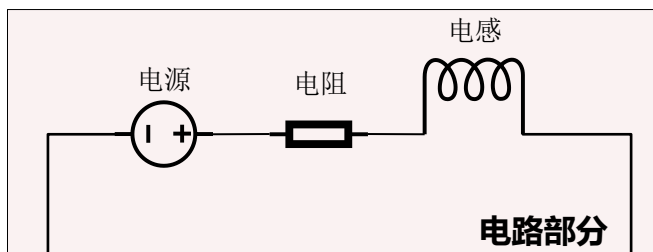
架构设计

组件建模

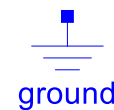
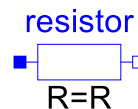
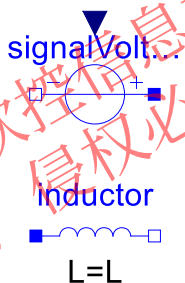
系统构建

设置参数

- 电路部分：**
- 在前述步骤中建立好的Valve子库中新建组件并修改名称及描述信息；
 - 拖拽组件：电源、电阻和电感并按照关系依次连接；
 - 绘制电路部分图标样式。



组件路径如下：



3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

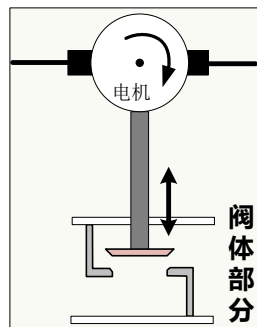
架构设计

组件建模

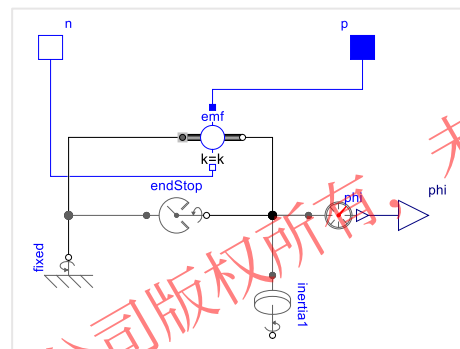
系统构建

设置参数

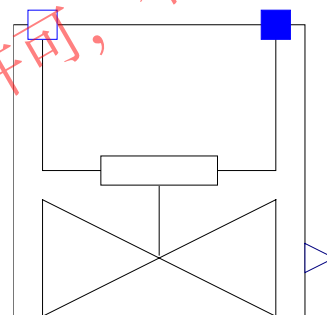
- 阀体部分：**
- 在前述步骤中建立好的Valve子库中新建组件并修改名称及描述信息；
 - 拖拽组件：EMF、转动惯量、限位器、和传感器并按照关系依次连接；
 - 绘制阀体部分图标样式。



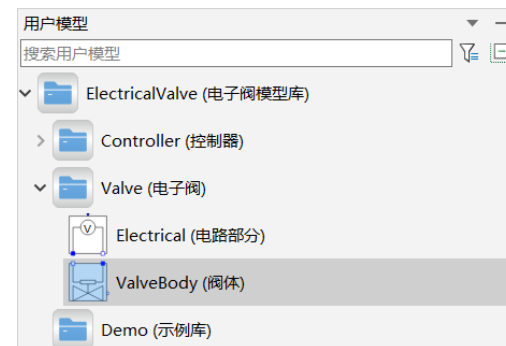
简化原理



组件建模



图标绘制



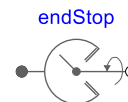
展示样式

组件路径如下：

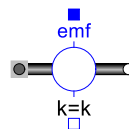
TYMechanics.Rotational.Sensors.AngleSensor



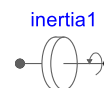
TYMechanics.Rotational.Components.EndStop



Modelica.Electrical.Analog.Basic.RotationalEMF



TYMechanics.Rotational.Components.Inertia



3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

架构设计

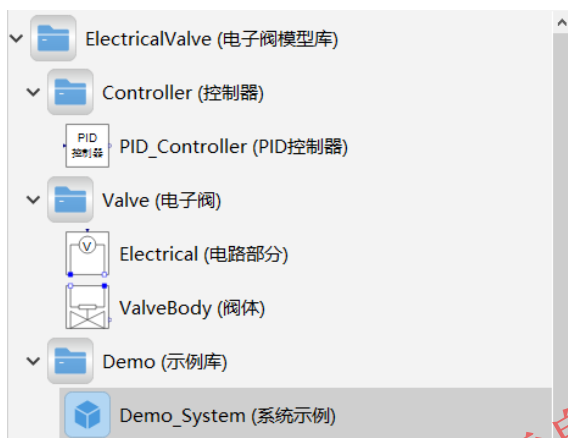
组件建模

系统构建

设置参数

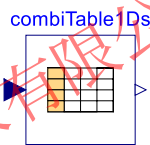
操作步骤：

- 在前述步骤中建立好的Demo子库中新建示例模型
- 根据阀芯旋转角度与流量对应关系，利用插值表替代简易流量计；
- 根据系统物理关系，构建电子阀的流量控制系统模型

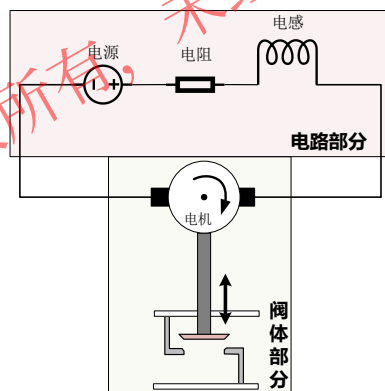


新建模型文件

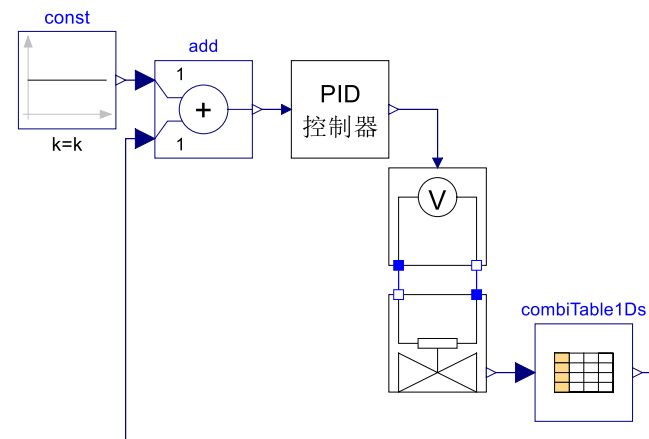
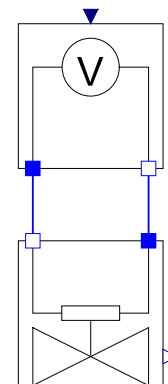
阀芯转动0~360°
对应流量0~50L/min



流量插值表



电子阀部分



系统模型

@苏州同元软控信息技术有限公司
版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-模型构建

创建新包

架构设计

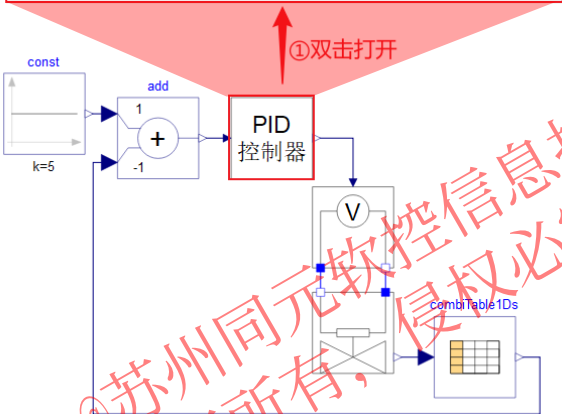
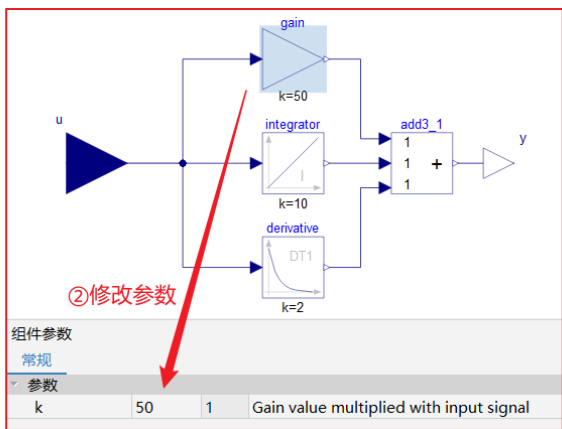
组件建模

系统构建

设置参数

操作步骤：

- 单独组件可直接修改参数，如const模块；
- 封装组件可双击打开组件内部，找到对应模块进行参数修改



组件名称	物理含义	参数名称	数值	单位
cosnt	期望流量	k	5	\
PID_Controller	比例增益	gain.k	60	\
	积分增益	intrgrator.k	2	\
	微分增益	derivative.k	0.1	\
Electrical	电阻阻值	resistor.R	50	Ω
	电感量	inductor.L	1.2e-4	H
ValveBody	电机变换系数	emf	10	N*m/A
	转动惯量	inertia	0.1	Kg*m ²
	角度限位	endStop.g_F	360	°
combiTable1Ds	流量插值表	table	{{0.0, 0}, {2*3.14, 50}}	\

03

PART 03 ➔

拖拽建模：电子阀流量控制系统

- 图形化建模
- 仿真求解设置
- 结果后处理

@苏州工业园区信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

3.拖拽建模：电子阀流量控制系统-仿真求解

模型检查

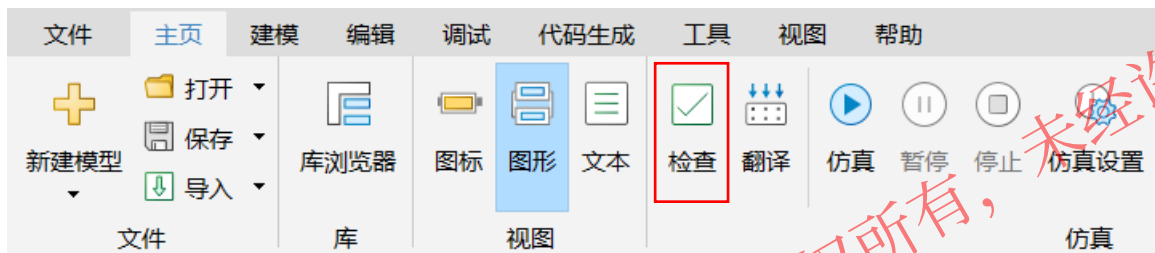
仿真设置

模型翻译

运行仿真

通过模型检查，分析模型是否存在错误

操作步骤：点击建模菜单的“检查”



检查模型是否可以仿真的条件:

- 判断模型中是否存在语法或其他错误，错误信息必须修改，警告仅起提示作用，可不修改。
- 检查模型是否完备(变量数和方程数一定要相等)，只有模型完备才有可能进行仿真。



3.拖拽建模：电子阀流量控制系统-仿真求解

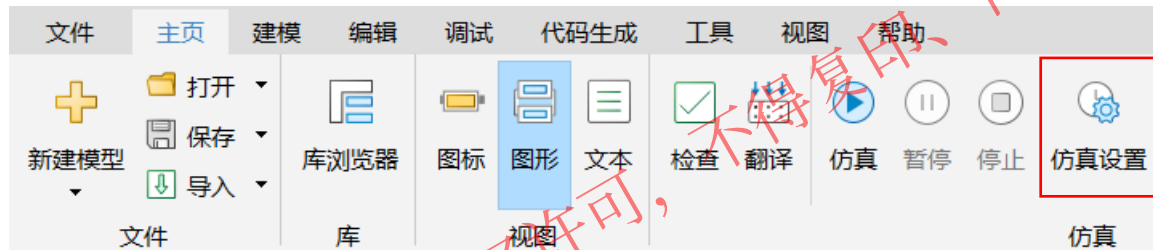
模型检查

仿真设置

模型翻译

运行仿真

操作步骤：点击主页（仿真）菜单的“仿真设置”



- **仿真区间**：仿真开始/停止的时间，此处设置为0~2s。
- **步长**：仿真输出点之间的间隔长度，此处设置步长为0.001。
- **步数**：仿真生成的输出间隔的数目。
- **算法**：MWORKS提供24种不同的仿真算法进行选择，并且提供自定义算法，此处使用Dassl算法。
- **精度**：指定每个仿真步长的局部精度。
- **积分步长**：选择变步长算法时为初始积分步长，选择定步长算法时为固定积分步长。
- **确定**：对于可修改的模型，可以将仿真设置中的常规设置保存到本次模型中。
- **确定并保存到模型**：对于可修改的模型，可以将仿真设置中的常规设置保存到模型中。

3.拖拽建模：电子阀流量控制系统-仿真求解

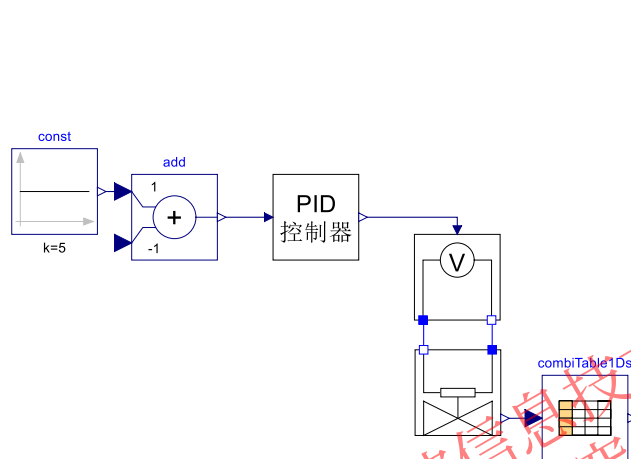
模型检查

仿真设置

模型翻译

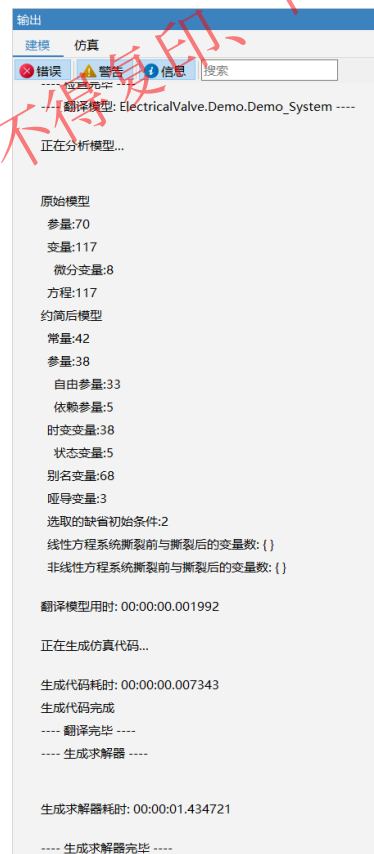
运行仿真

操作步骤：点击建模（仿真）菜单的“翻译”



系统模型

注意：模型存在错误，翻译失败终止。



翻译信息：

模型内变量数=方程数

常量

参量

变量

微分变量

方程

.....

翻译完成输出页面

3.拖拽建模：电子阀流量控制系统-仿真求解

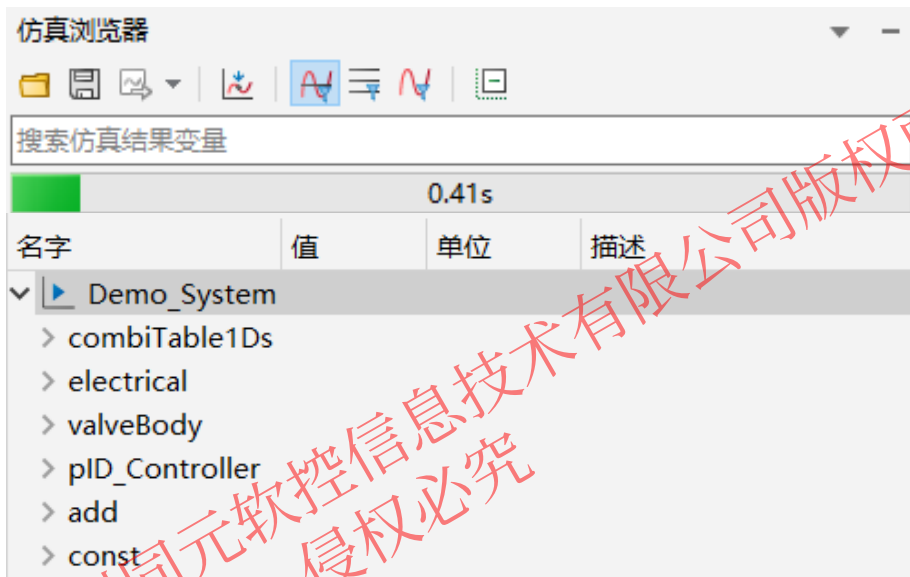
模型检查

仿真设置

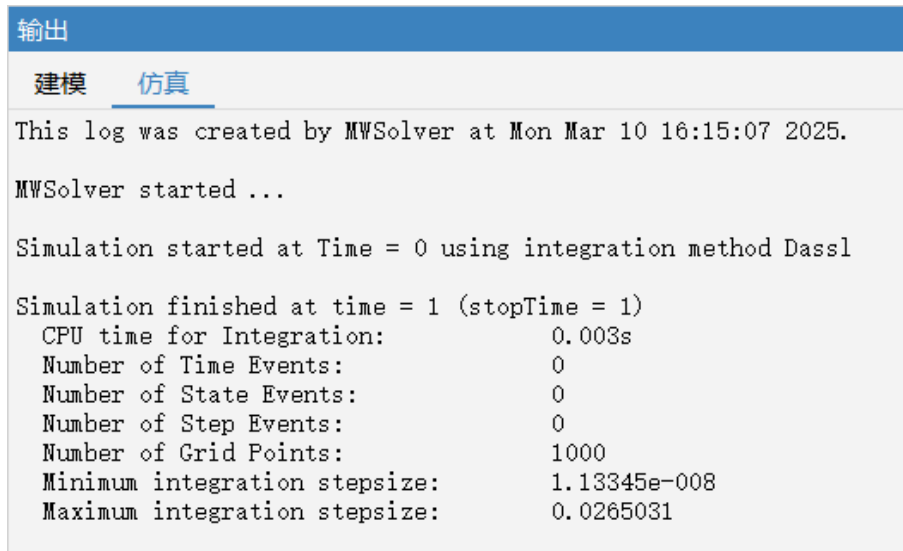
模型翻译

运行仿真

操作步骤：点击建模（仿真）菜单的“仿真”



仿真浏览器可查看模型仿真进度



仿真结束后可查看模型输出信息

03

PART 03 ➔

拖拽建模：电子阀流量控制系统

- 图形化建模
- 仿真求解设置
- 结果后处理

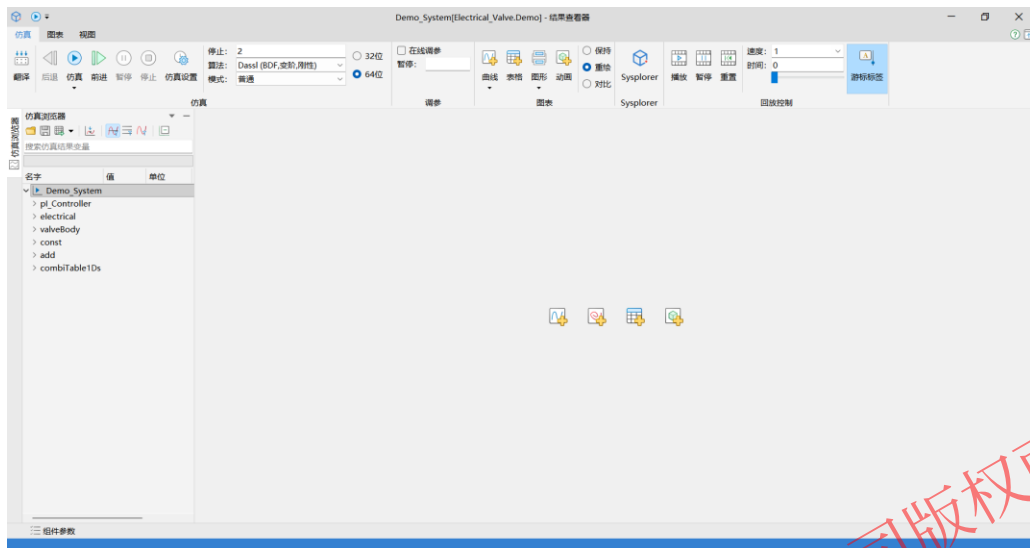
@苏州工业园区信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

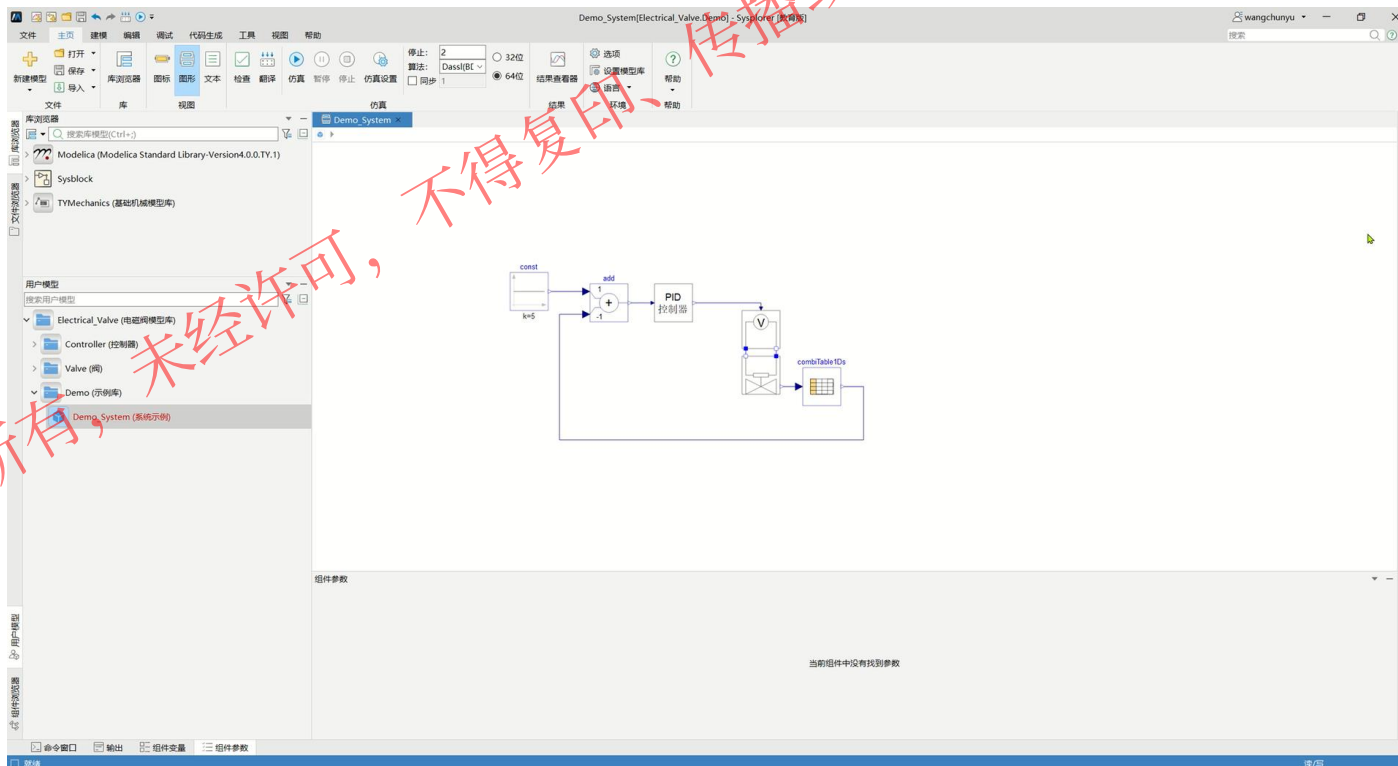
3.拖拽建模：电子阀流量控制系统-结果处理

结果查看器

- 自动弹出



- 通过按钮打开



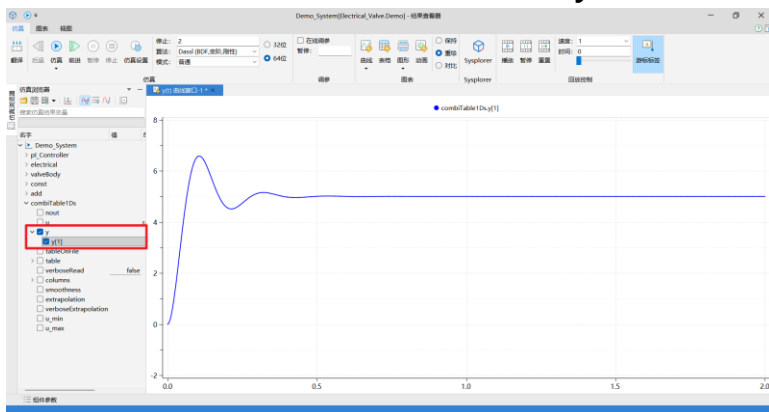
打开结果查看器.mp4

@苏州同元软控信息技术有限公司
版权所有，侵权必究

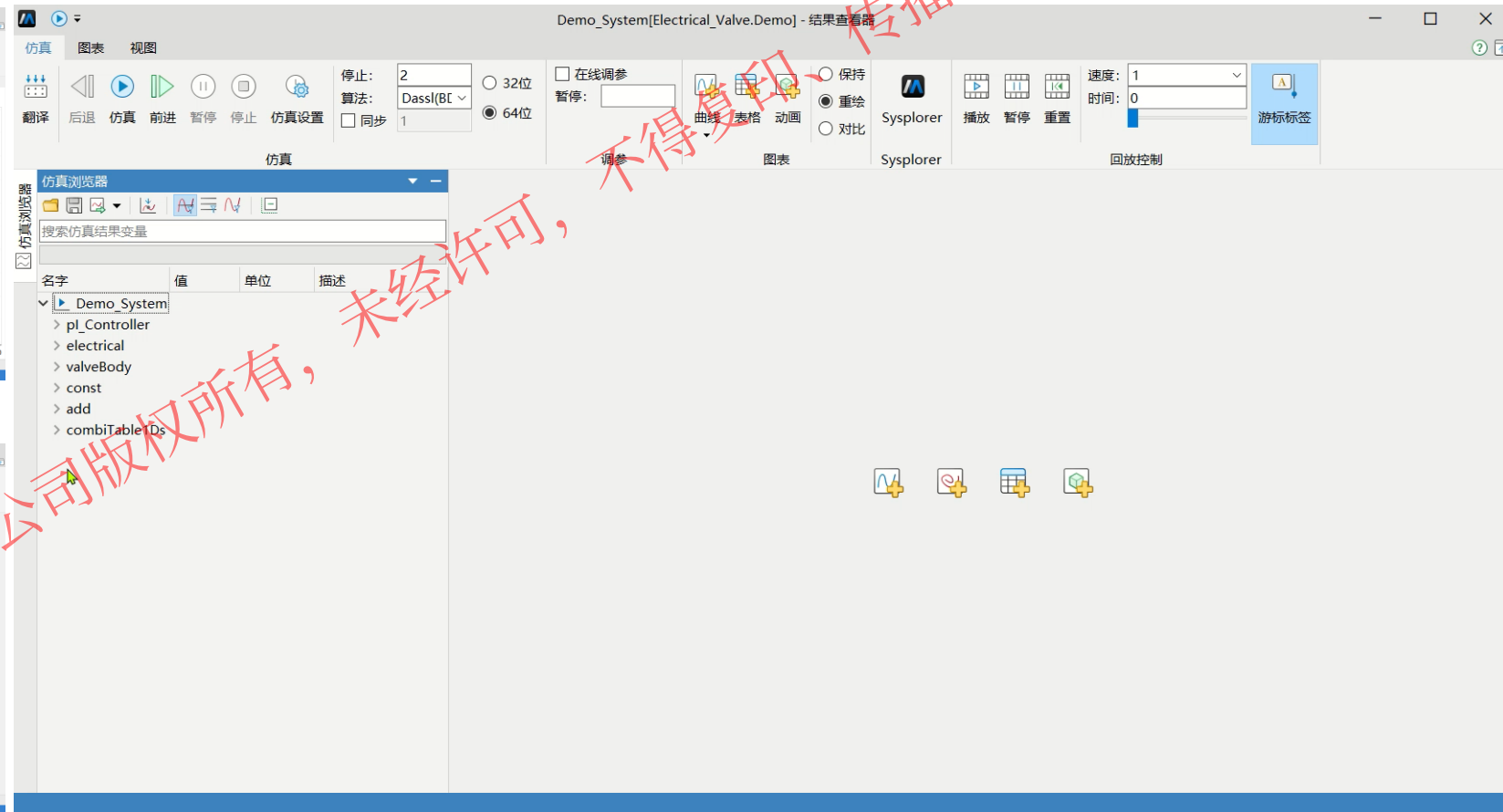
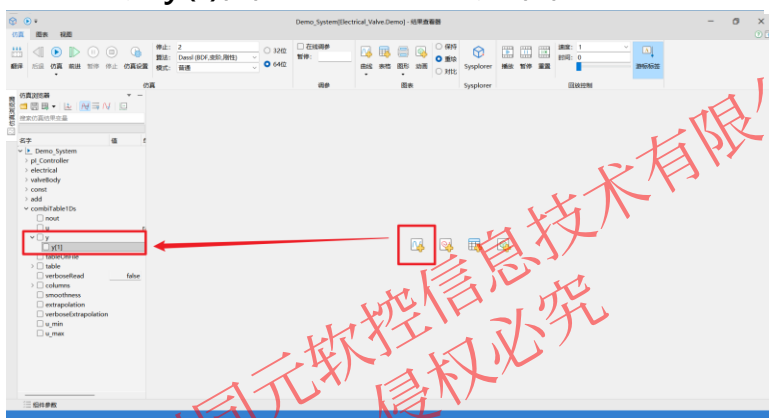
3.拖拽建模：电子阀流量控制系统-结果处理

绘制y(t)曲线

- 直接勾选结果变量，默认自动生成y(t)曲线



- 点击新建y(t)图窗按钮后再绘制曲线

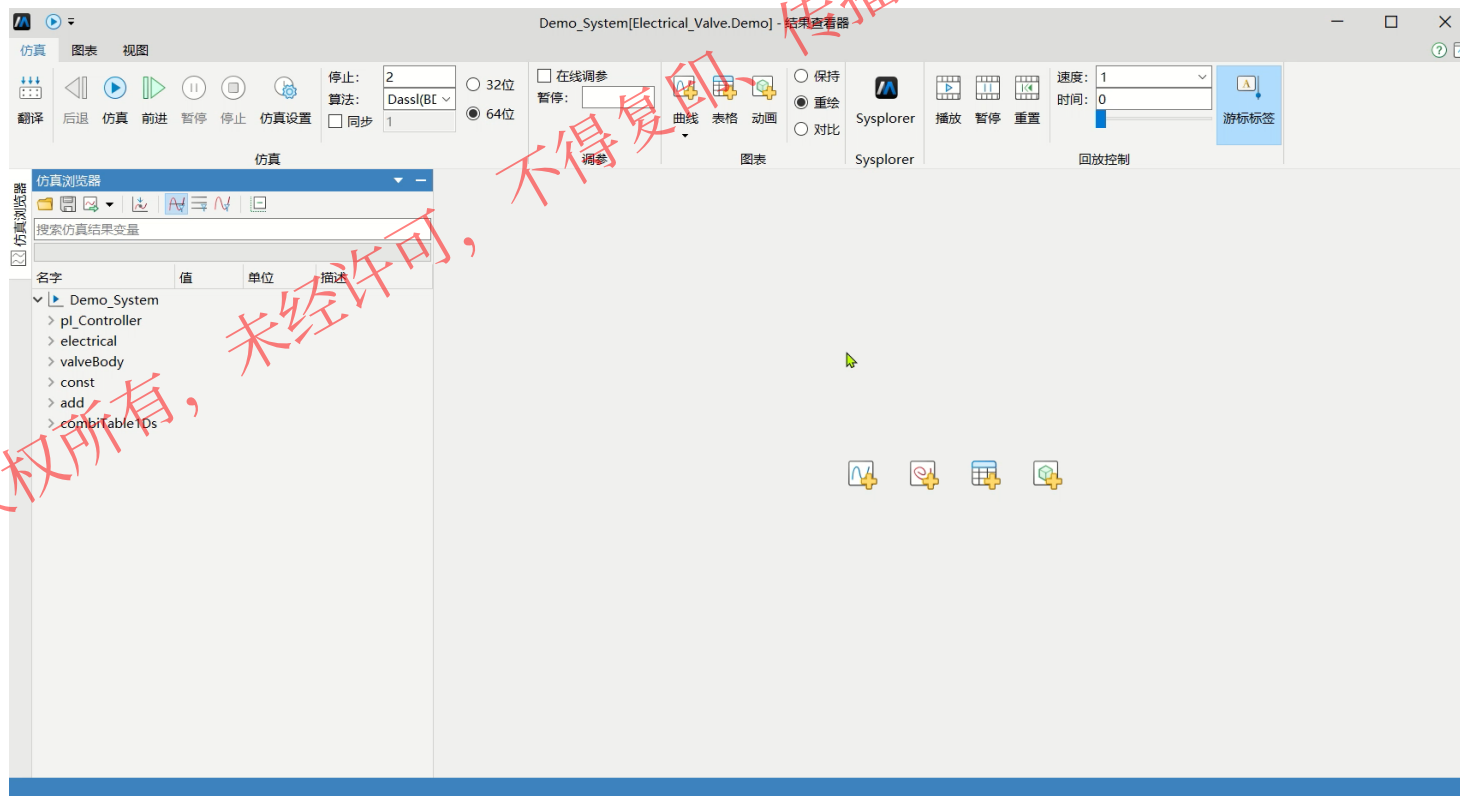
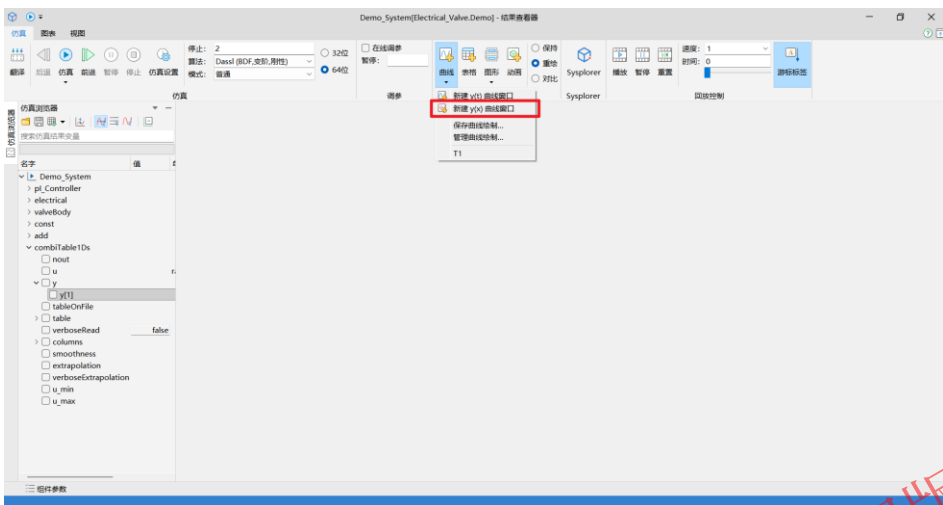


绘制y(t)曲线.mp4

3.拖拽建模：电子阀流量控制系统-结果处理

绘制y(x)曲线

- 点击新建y(x)图窗按钮后再绘制曲线



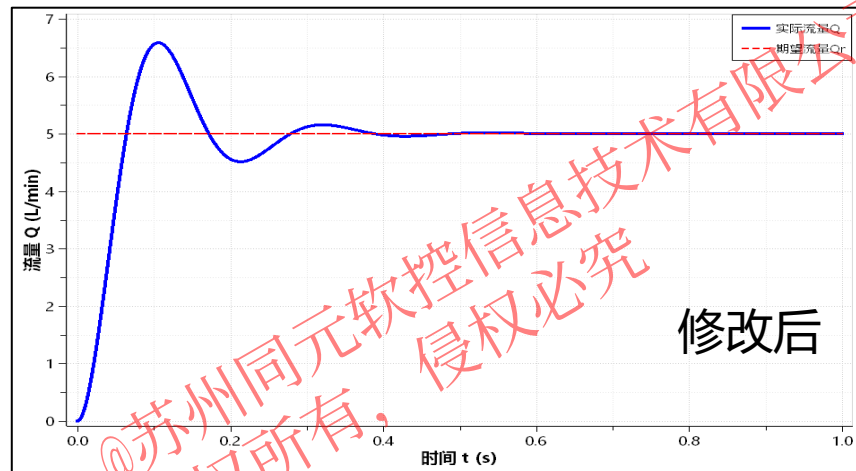
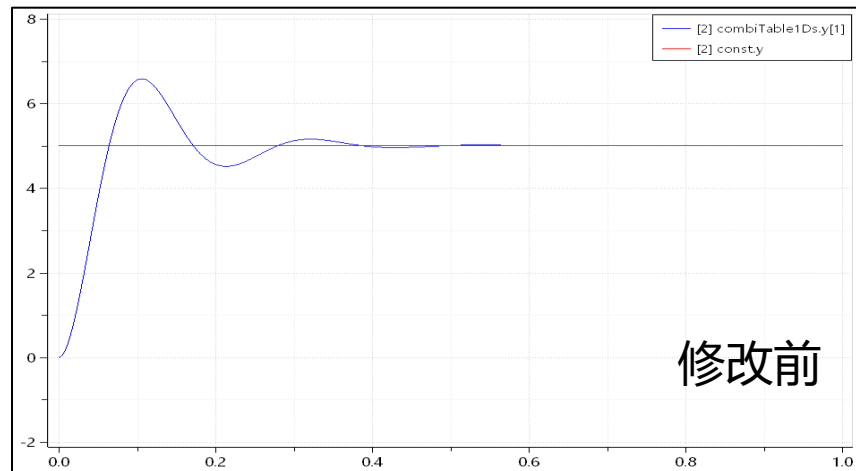
绘制y(x)曲线.mp4

@苏州同元软控信息技术有限公司版权所有，侵权必究

3.拖拽建模：电子阀流量控制系统-结果处理

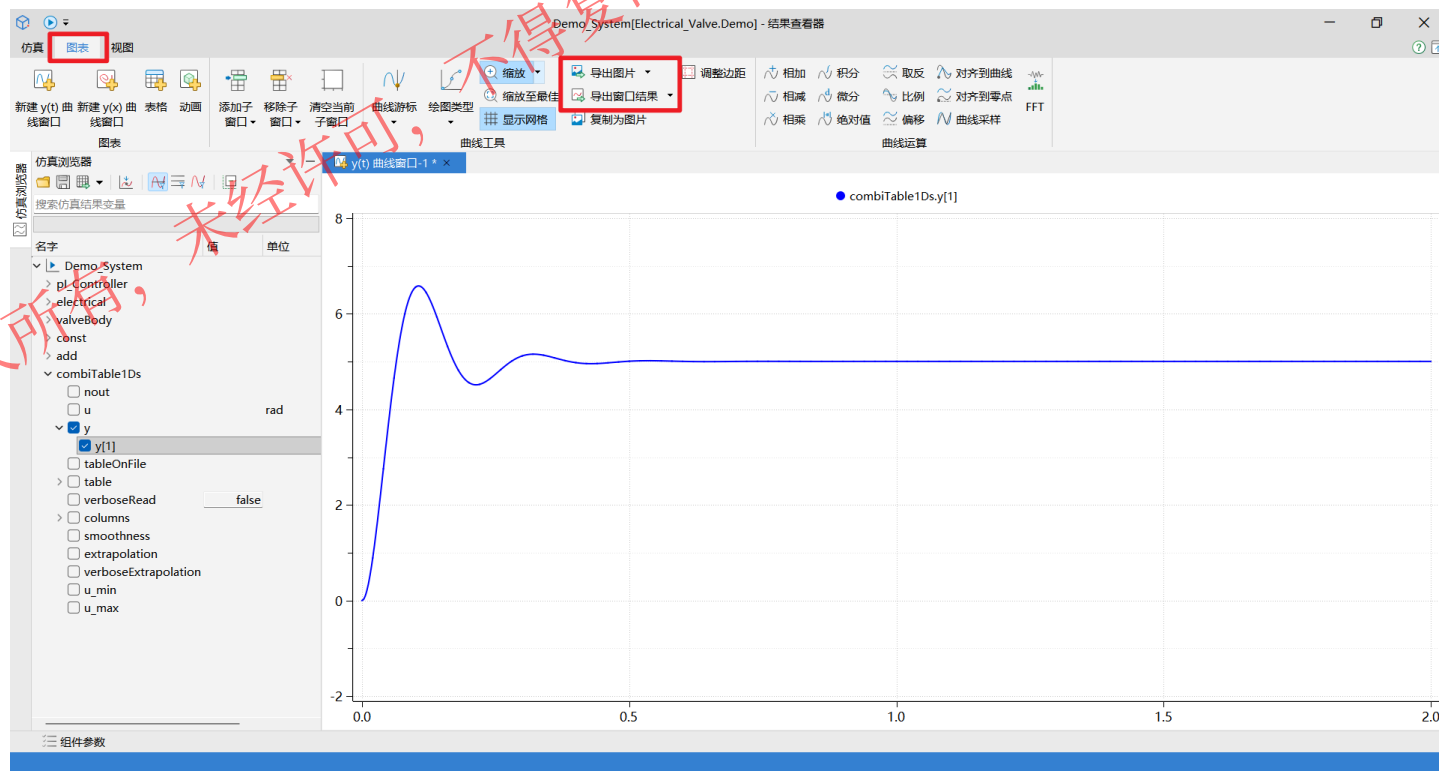
曲线处理

- 右键曲线图例位置，可以进行曲线样式修改



曲线导出

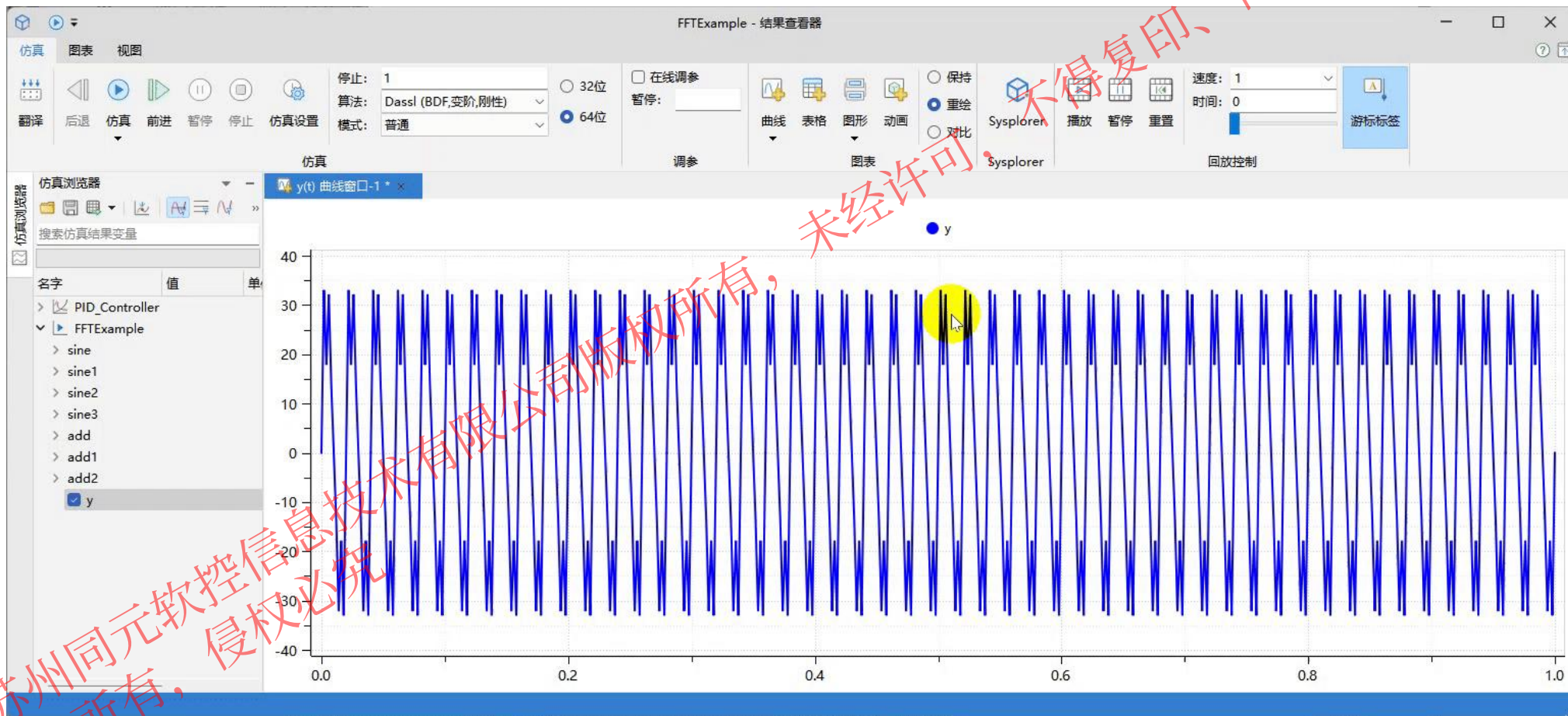
- 点击图窗栏，选择导出图片或导出结果
 - 导出图片支持png、svg、jpg等多种格式；
 - 导出结果支持csv、txt格式。



3.拖拽建模：电子阀流量控制系统-结果处理

曲线处理

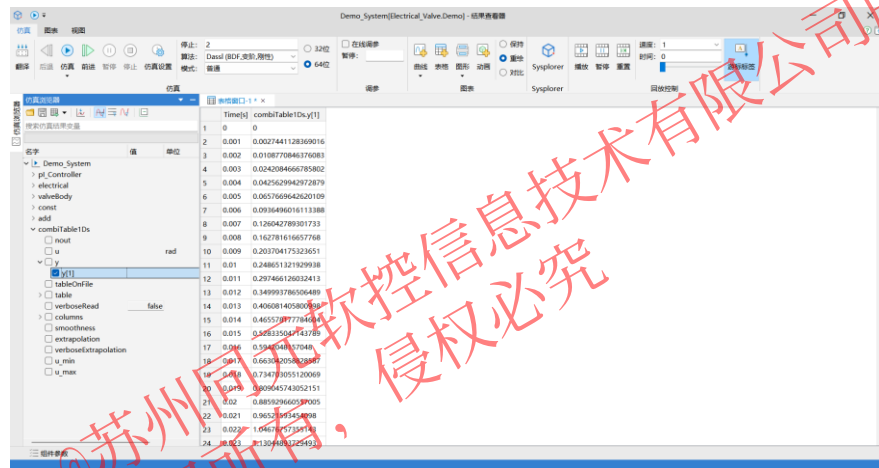
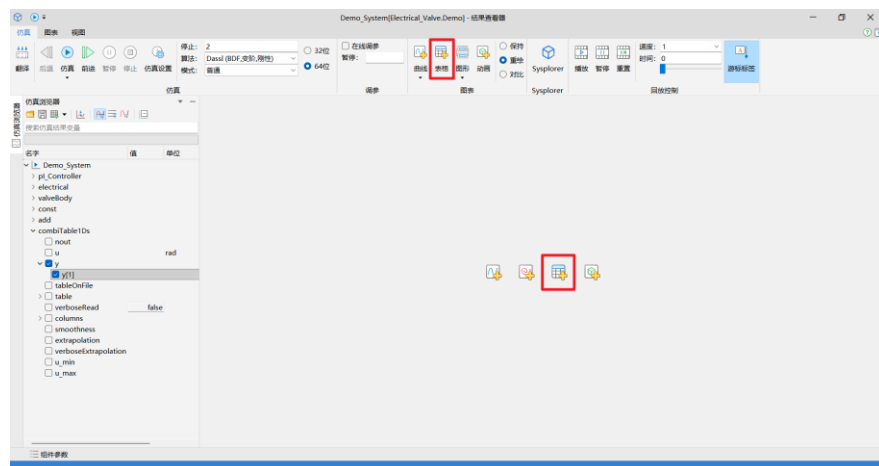
Sysplorer 2026a 更新：曲线支持傅里叶变换，快速对数据进行频域分析



3.拖拽建模：电子阀流量控制系统-结果处理

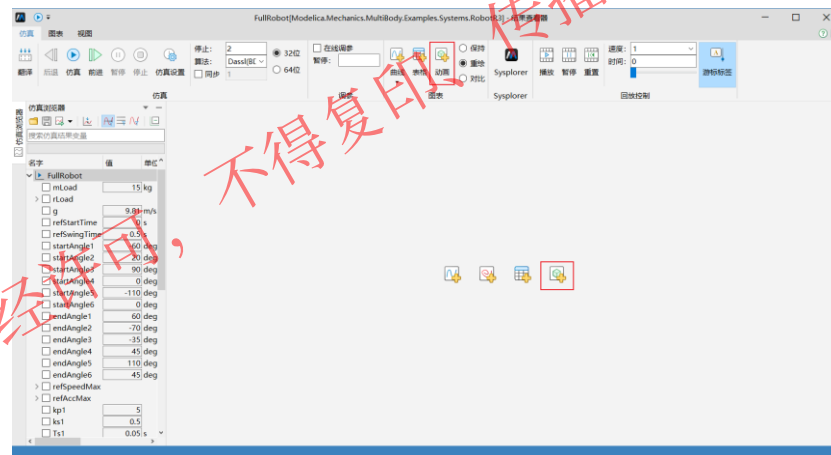
获取表格数据

- 新建表格窗口后选择对应变量可生成对应结果

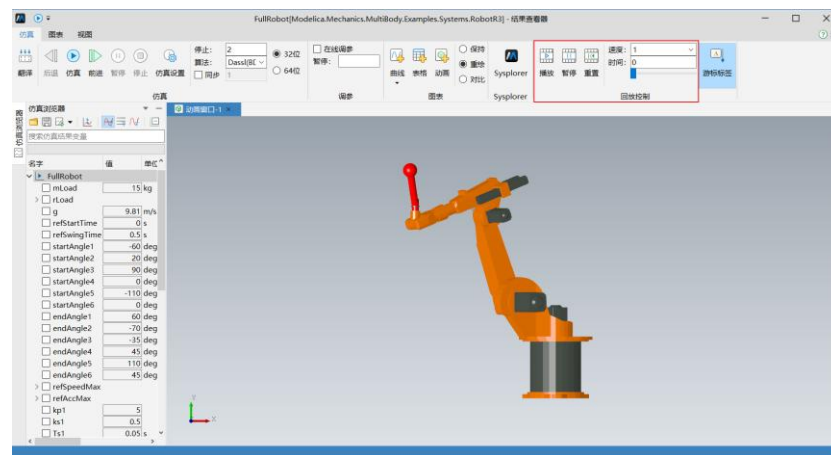


展示多体动画

- 新建多体动画窗口(注：仿真模型中需具备多体模型才能展示)

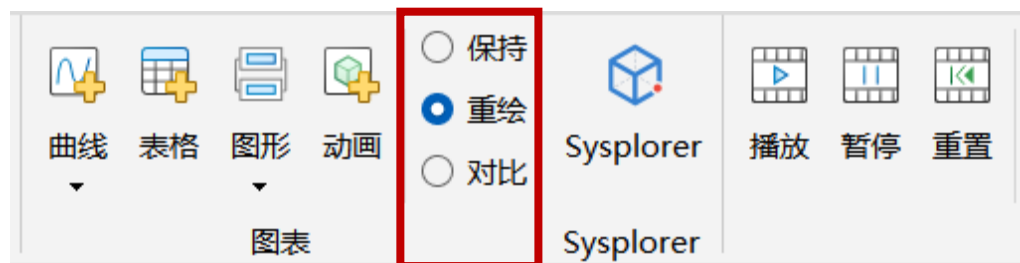


- 可通过功能按钮实现多体动画的播放调整



3.拖拽建模：电子阀流量控制系统-结果处理

• 曲线的“保持”、“重绘”和“对比”



➤ 保持

保持当前绘图窗口的结果，重新仿真不会改变；

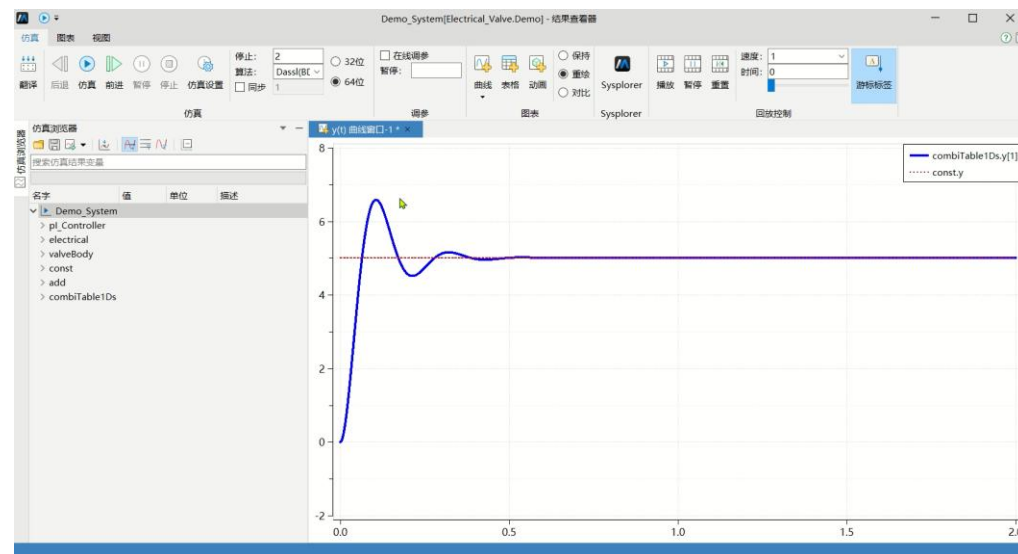
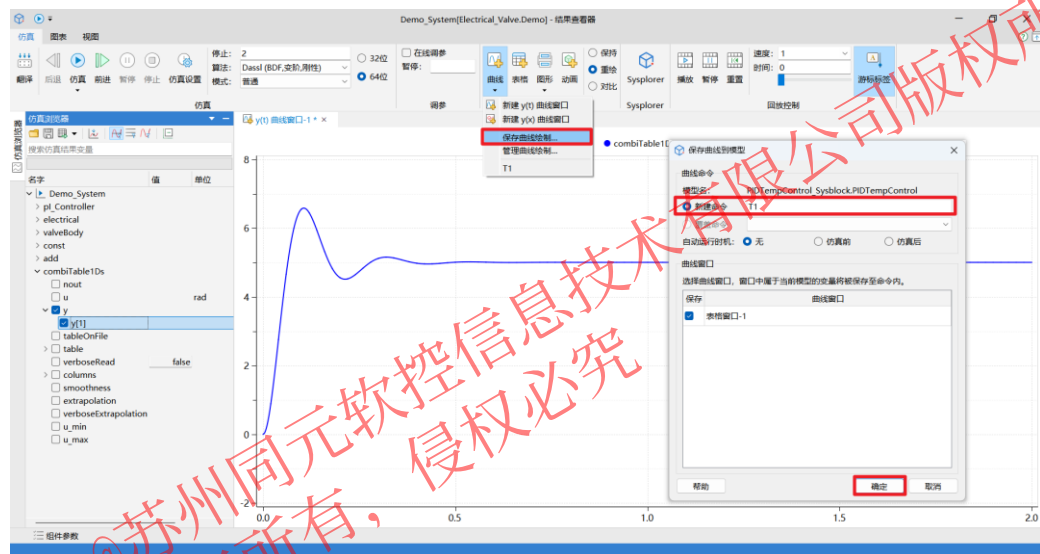
➤ 重绘（默认选项）

重新仿真后将当前窗口曲线根据重新仿真的结果再次绘制；

➤ 对比

重新仿真后会保持上一次仿真的结果，再将本次仿真结果进行绘制；

• 保存曲线绘制



CONTENTS

目录 ➔



01 MWORKS.Sysplorer软件概览



02 模型库加载



03 拖拽建模：电子阀流量控制系统



04 文本建模：电阻



05 模型发布



06 MWORKS.Sysplorer与Simulink之间的差异



07 Modelica语法概览

04

PART 04 ➔

文本建模：电阻

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

4.文本建模：电阻



@苏州同元软控信息技术有限公司
版权所有，侵权必究

4.文本建模：电阻-理论分析

如果想要构建一个电阻组件，我该如何做？

类



属性分析

- 电阻是导体本身的一种属性，因此导体的电阻与导体是否接入电路、导体中是否有电流、电流的大小等因素无关。
- 电阻是描述导体导电性能的物理量，用R表示。
- 电阻的单位是欧姆，简称欧，用希腊字母“Ω”表示

行为分析

- 电阻由导体两端的电压U与通过导体的电流I的比值来定义。
欧姆定律：

$$R = \frac{U}{I}$$

- 进入电阻电流为正值，离开电阻的电流为负值，电阻的电流的代数和等于零。
- 电阻的电压为电阻两端所加的电压

对象



1MΩ



2.74MΩ



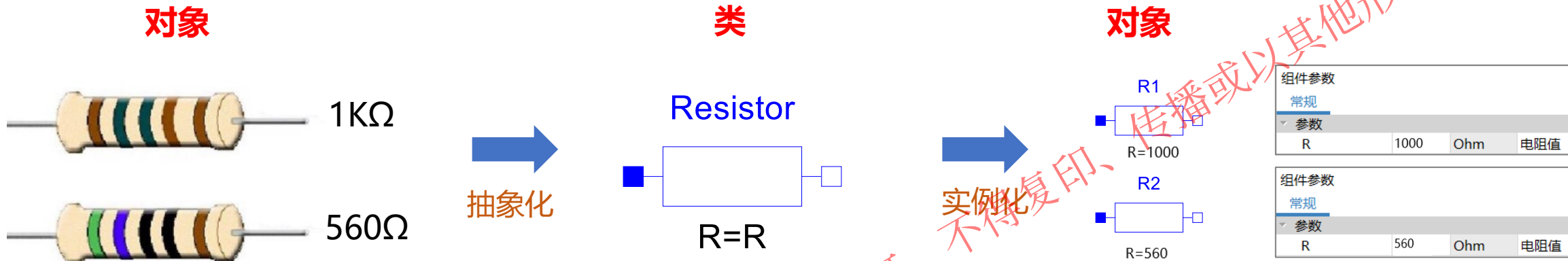
1KΩ



560Ω

以上的电阻每一个都要单独建模么？

4.文本建模：电阻-理论分析



- 接口：两个电学接口
- 属性：电阻值
- 行为：欧姆定律 $U = I \cdot R$

只需要构建一个电阻模型即可

```
model Object "对象"  
  Modelica.Electrical.Analog.Basic.Resistor R1(R = 1000) "电阻1"  
    annotation (...);  
  Modelica.Electrical.Analog.Basic.Resistor R2(R = 560) "电阻2"  
    annotation (...);  
annotation (...);  
end Object;
```

拖拽类至模型中并定义参数，即为实例化对象

4.文本建模：电阻-电阻模型开发

```
model 模型名称 "备注"  
//===参数定义===  
parameter 数据类型 参数名=参数值 "备注"  
  
//===变量定义===  
数据类型+变量 "备注";  
  
//===接口实例化===  
接口路径 接口1名称 "接口1"  
接口路径 接口1名称 "接口1"  
  
//===行为描述===  
equation  
定义方程或者算法  
.....  
  
end 模型名称;
```

Modelica代码一般结构

接口定义

接口实例化

参变量定义

行为描述

模型检查

图标绘制

模型测试

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

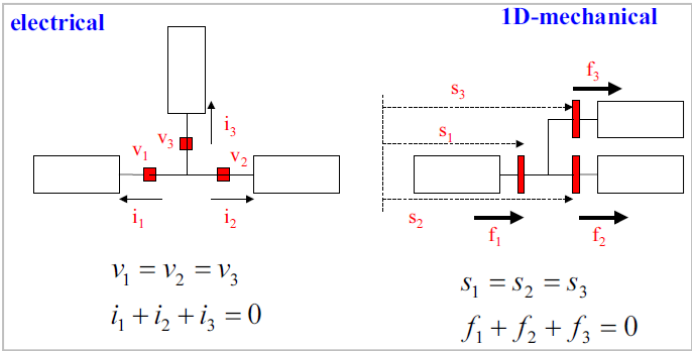
模型检查

图标绘制

模型测试

广义基尔霍夫定律

端口所有势变量相等；
端口所有流变量和零。



变量传递

连接器

势变量 (位移、压力、温度等等值传递的变量)



领域	势变量		流变量	
平移机械	s	位移	f	力
转动机械	φ	角度	τ	转矩
电子	v	电压	i	电流
液压	p	压力	$\dot{V}$	流速
热力学	T	温度	$\dot{Q}$	熵流
化学	μ	化学势	$\dot{N}$	粒子流

```
connector PositivePin "正极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end PositivePin;
```

```
connector NegativePin "负极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end NegativePin;
```

绘制
图标



- 新建接口
- 定义势变量v，流变量i
- 绘制图标

详见课程《Modelica语法-连接与连接器(非因果连接器)》

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

模型检查

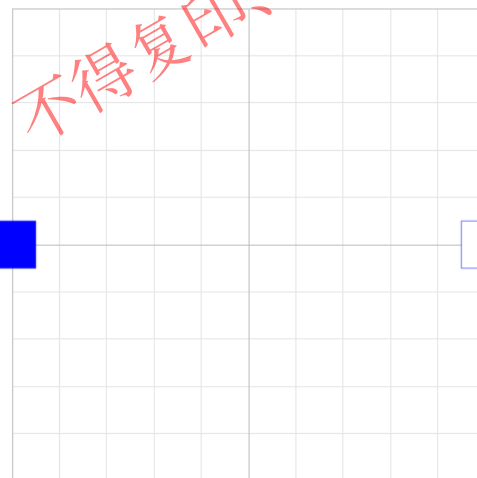
图标绘制

模型测试

```
connector PositivePin "正极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end PositivePin;
```

```
connector NegativePin "负极"  
  Modelica.SIunits.Voltage v "电势"; //势变量  
  flow Modelica.SIunits.Current i "电流"; //流变量  
end NegativePin;
```

接口实例化



```
model Resistance "电阻 "  
  //接口实例化  
  //形式：接口路径+接口名称  
  PositivePin positivePin;  
  NegativePin negativePin;  
  
end Resistance;
```

电阻"模型文本视图自动生成代码

将接口直接拖拽至"电阻"
模型图形视图

详见课程《Modelica语法-连接与连接器
(非因果连接器)》

@苏州同元软控信息技术有限公司版权所有，侵权必究

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

模型检查

图标绘制

模型测试

```
model 模型名称 "备注"  
//===参数定义===  
parameter 数据类型 参数名=参数值 "备注"  
//===变量定义===  
数据类型+变量 "备注";  
  
//===接口实例化===  
接口路径 接口1名称 "接口1"  
接口路径 接口1名称 "接口1"  
//===行为描述===  
equation  
    定义方程或者算法  
.....  
end 模型名称;
```

参数定义

变量定义

接口实例化

行为描述

参数:
电阻阻值: R = 10

变量:
电阻电压: v
电阻电流: i

变量定义方式:

数据类型+变量名(初值)+ "变量注解"

参数定义方式:

前缀 (参数、常数) + 数据类型 + 参数名=参数值+ "参数注解"

Modelica文本

```
//参数定义  
parameter Real R = 10 "电阻阻值";  
//变量定义  
Real v "电阻电压";  
Real i "电阻电流";
```

数据类型

Modelica语言提供了5大数据类型: **Real、Integer、Boolean、String、enumeration**

详见课程《Modelica语法-类与内置类型-基本类型(基础数据类型及其属性)》

前缀 (参数、常数)

前缀 (参数、常数) 用于定义组件值在分析过程中的可变性: **parameter、constant**

详见课程《Modelica语法-类与内置类型-基本类型(数据的前缀用法)》

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

模型检查

图标绘制

模型测试

参数：
电阻阻值：R = 10

变量：
电阻电压：v
电阻电流：i

有物理含义
Modelica文本

```
//参数定义
parameter Modelica.SIunits.Resistance R = 10 "电阻阻值" ;
//变量定义
Modelica.SIunits.Voltage v "电阻电压" ;
Modelica.SIunits.Current i "电阻电流" ;
```

无物理含义
Modelica文本

```
//参数定义
parameter Real R = 10 "电阻阻值" ;
//变量定义
Real v "电阻电压" ;
Real i "电阻电流" ;
```

Modelica标准库中已根据ISO 31-1992国际标准单位制，声明了450个左右的物理量类型

因此，建议在建模过程中直接引用Modelica标准库中已进行预定义的物理量
Modelica还可以自行执行单位推导计算用来方便用户检查变量定义

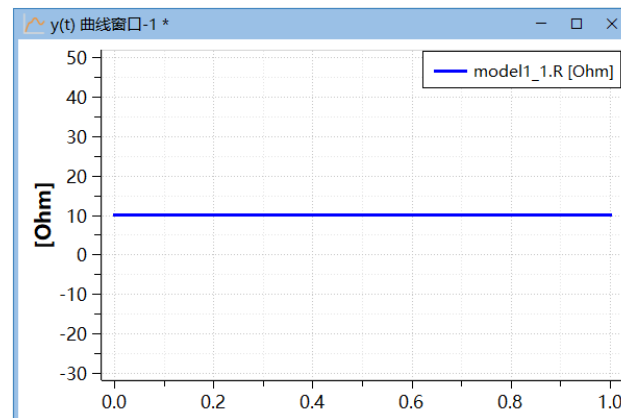
上述模型实现只反映了其数学特性，但是并没有明确模型中各个变量的物理属性，比如R、v、i等分别是什么？即：

- 各变量物理单位是什么？
- 更为重要的是：这些方程是否具有**物理一致性**？

注：物理一致性是方程等式两边是否具有相同的物理单位

那么，我们应该怎么做？

详见课程《Modelica语法-类与内置类型-基本类型(type)》



4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

模型检查

图标绘制

模型测试

构建一个模型，需要定义对象的属性和行为。

前面已经学习了对象属性的定义，那么**对象的行为**该如何定义？

特点：以**陈述式**方程表达模型的行为，模型行为即模型的数学方程或物理方程。

```
model Resistance "电阻"
parameter Modelica.SIunits.Resistance R = 10;
Modelica.SIunits.Voltage v;
Modelica.SIunits.Current i;
PositivePin positivePin;
NegativePin negativePin;
equation
v = R * i;
i = positivePin.i;
positivePin.i + negativePin.i = 0;
v = positivePin.v - negativePin.v;
end Resistance "电阻";
```

```
model 模型名称 "备注"
//===参数定义===
parameter 数据类型 参数名=参数值 "备注"
//===变量定义===
数据类型+变量 "备注";
//===接口实例化===
接口路径 接口1名称 "接口1"
接口路径 接口1名称 "接口1"

//===行为描述===
equation
定义方程或者算法
.....
end 模型名称;
```

参数定义

变量定义

接口实例化

行为描述

- 欧姆定律： $R = \frac{U}{I}$
- 进入电阻电流为正值，离开电阻的电流为负值，电阻的电流的代数和等于零
- 电阻的电压为电阻两端所加的电压

详见课程《Modelica语法-模型行为描述(方程)》

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

模型检查

图标绘制

模型测试

```
model Resistance "电阻"  
  parameter Modelica.SIunits.Resistance R = 10;  
  Modelica.SIunits.Voltage v;  
  Modelica.SIunits.Current i;  
  PositivePin positivePin;  
  NegativePin negativePin;  
equation  
  v = R * i;  
  i = positivePin.i;  
  positivePin.i + negativePin.i = 0;  
  v = positivePin.v - negativePin.v;  
end Resistance "电阻";
```



- 检查语法
- 检查方程数和变量数



6个变量:

- v
- i
- positivePin.v
- positivePin.i
- negativePin.v
- negativePin.i

6个方程:

- $v = R * i;$
- $i = \text{positivePin.i};$
- $\text{positivePin.i} + \text{negativePin.i} = 0;$
- $v = \text{positivePin.v} - \text{negativePin.v};$
- $\text{positivePin.i} = 0;$
- $\text{negativePin.i} = 0;$

只有方程数=变量数，模型才完整，
否则需要检查模型中的变量和方程

@苏州同元软控信息技术有限公司版权所有，侵权必究

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

行为描述

模型检查

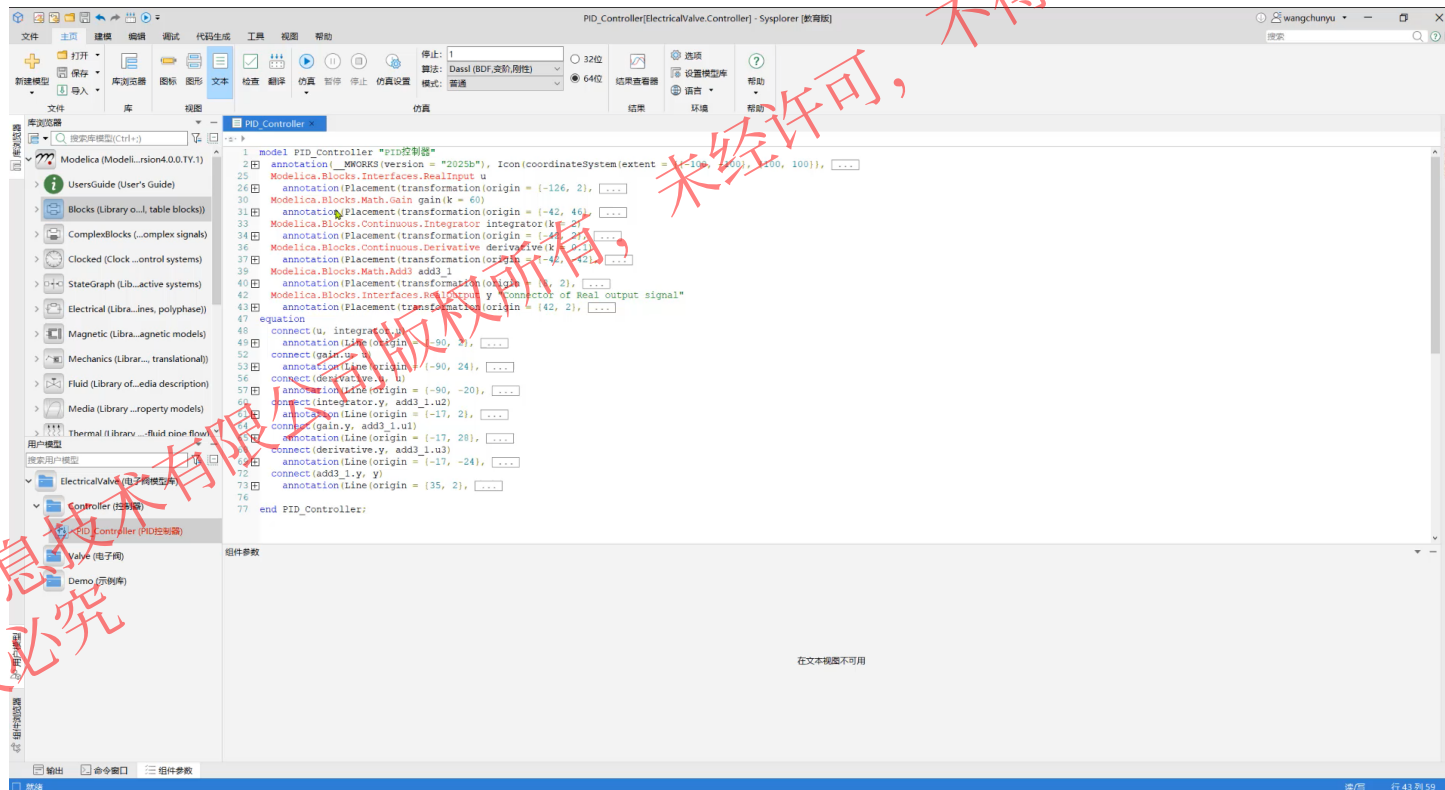
图标绘制

模型测试

功能修改：不再支持自动进行文本格式化

说明：在此版本中不会在检查时自动进行文本格式化，同时双击菜单栏文本按钮进行格式化的功能也被移除。

操作：在文本视图中，可以通过**右键进行格式化文本**的方式或通过快捷键 **Ctrl+Alt+L** 进行文本格式化。



文本格式化示例.mp4

4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

参变量定义

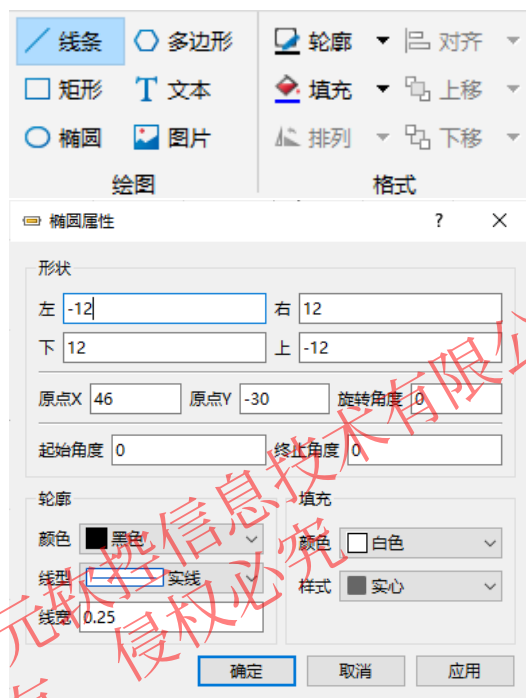
行为描述

模型检查

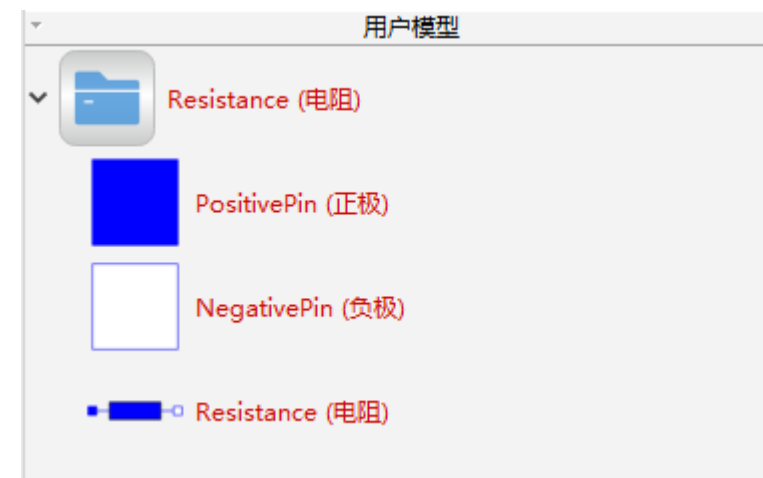
图标绘制

模型测试

- 选择图标界面
- 使用“建模”或“编辑”的基本图元进行绘制
- 在图形属性中，为图形添加填充颜色及样式



给模型穿上“漂亮的衣服”



4.文本建模：电阻-电阻模型开发

接口定义

接口实例化

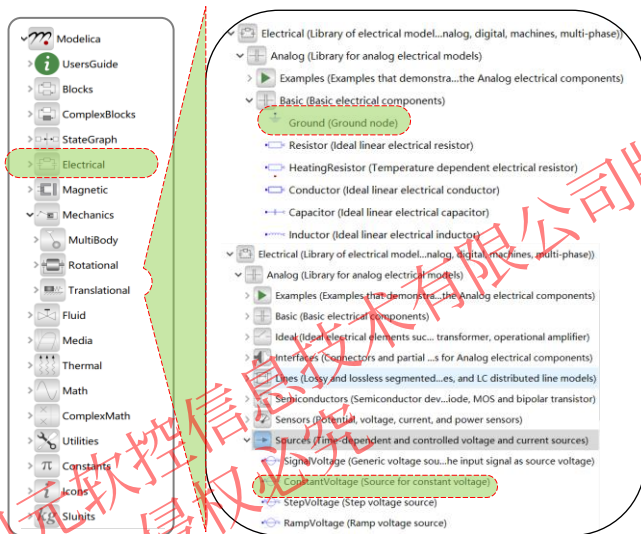
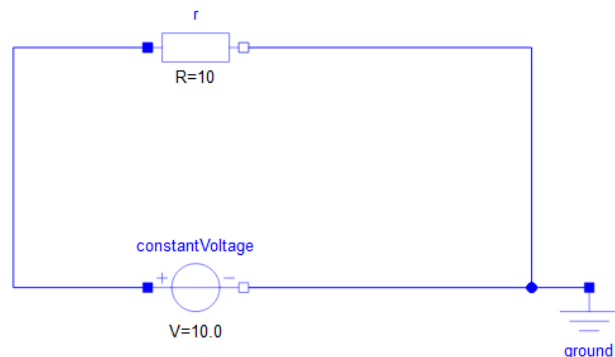
参变量定义

行为描述

模型检查

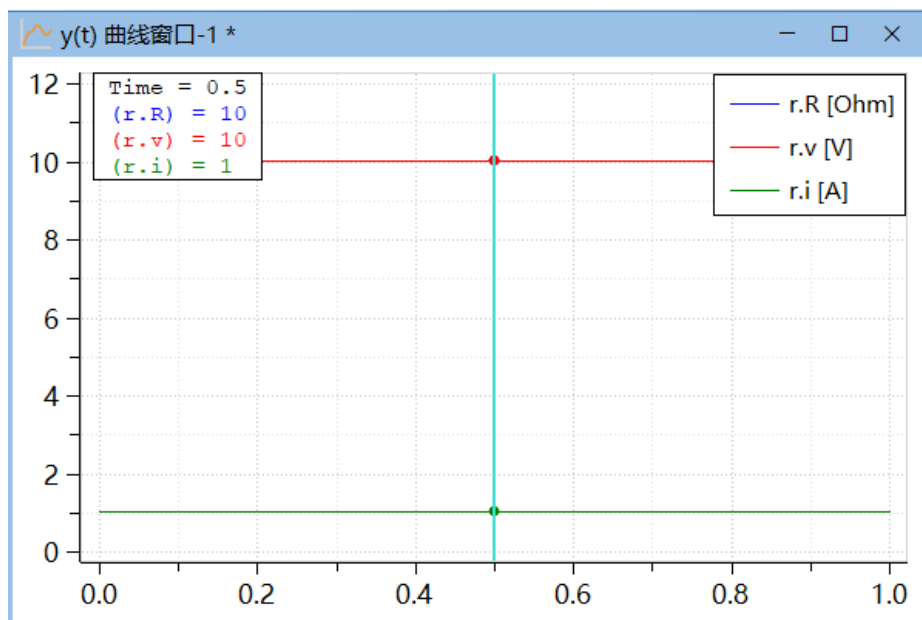
图标绘制

模型测试



名字	值	单位	描述
R	10	Ohm	电阻值
v	10	V	电阻两端电压
i	1	A	流过电阻的电流

结果查看



接地组件路径-Modelica. Electrical.Analog.Basic.Ground
恒压电源路径-Modelica. Electrical.Analog.Sources. ConstantVoltage

CONTENTS

目录 ➞



01 MWORKS.Sysplorer软件概览



02 模型库加载



03 拖拽建模：电子阀流量控制系统



04 文本建模：电阻



05 模型发布



06 MWORKS.Sysplorer与Simulink之间的差异



07 Modelica语法概览

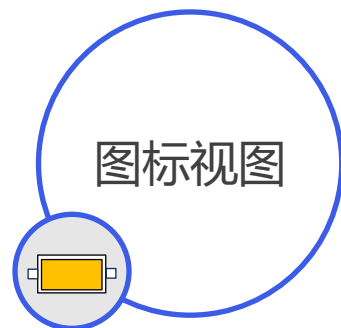
PART 05 ➔

模型发布

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

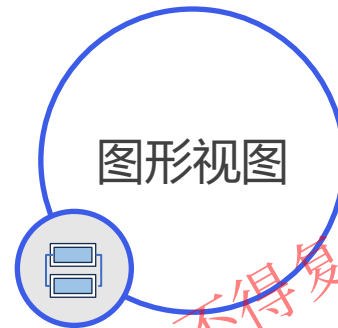
05

5.模型发布-为什么需要模型发布



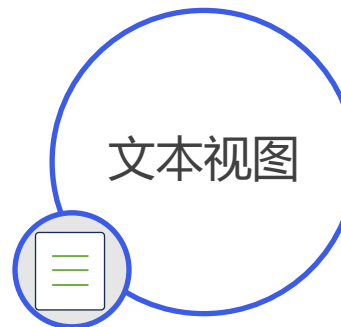
图标视图

图标是模型作为组件插入其他模型时的图形表示，在图标视图中可以绘制模型图标。



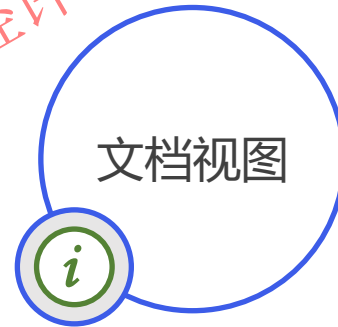
图形视图

图形视图是可视化建模时最重要的一个视图，显示了模型中声明的组件、连接器、连接关系等信息，主要在以拖放方式构建模型时使用。



文本视图

文本视图是一个文本编辑器，可以显示和编辑模型的Modelica文本。



文档视图

文档浏览器内显示模型的相关说明，同时可以编辑模型文档以丰富模型说明。

MWORKS.Sysplorer 提供多种可视化的形式支持进行模型细节浏览，包括图标、图形、文本、文档等视图窗口，其中文本视图，可能涵盖丰富的专业相关的知识与经验，甚至涉及开发者独有的且不希望公开的核心数据。此时需要模型发布支持为发布模型设置加密等级与许可特征，对Modelica 模型进行有效的保护。

5.模型发布-模型发布功能说明

• 加密等级

模型发布功能支持多种粒度、多种层次的模型保护级别，对模型使用、代码浏览、代码复制等显示和操作场景进行必要的控制。模型保护级别由模型库开发者指定，MWORKS.Sysplorer 确保外部用户不能对其进行非法访问或篡改

• 许可特征

模型发布功能支持逐层设置模型的使用许可，模型开发者可在发布模型时指定使用许可名称，当用户缺少使用许可时，将无法使用该模型

模型发布

模型名	加密等级	许可特征	模型路径
Electrical_Valve	Default		C:\Users\TR\Desktop\EI...rical_Valve\package.mo
> Controller	Default	继承于父模型	C:\Users\TR\Desktop\E...Controller\package.mo
> Valve	Default	继承于父模型	C:\Users\TR\Desktop\E...alve\Valve\package.mo
> Demo	Default	继承于父模型	C:\Users\TR\Desktop\...alve\Demo\package.mo

☒ 拷贝模型目录中的资源文件

发布路径 C:\Users\TR\Documents\MWORKS\Electrical_Valve_publish 浏览...

帮助 保存加密配置 确定 取消

注意：

- 1.发布路径选择的文件必须为**空文件夹**
- 2.发布模型时自动修正父模型与子模型的加密配置冲突
父模型不得高于子模型的加密等级

5.模型发布-模型加密等级限制说明

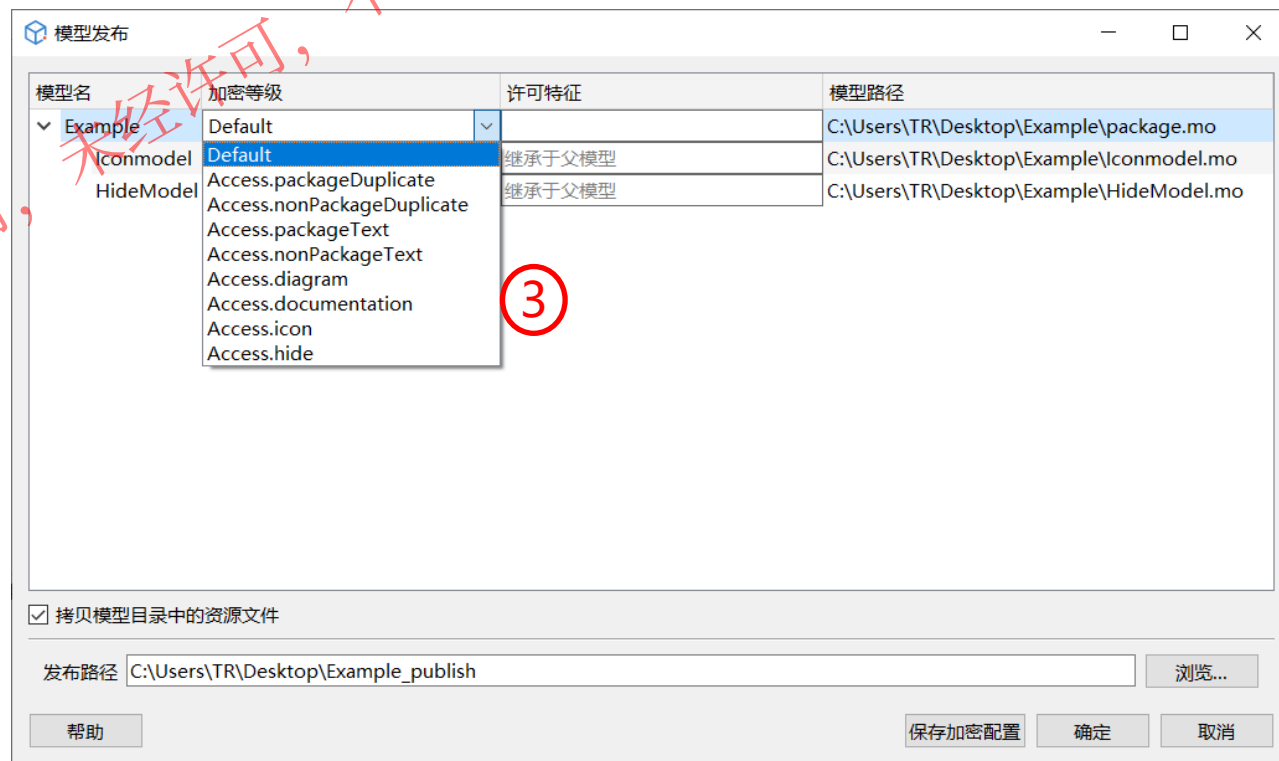
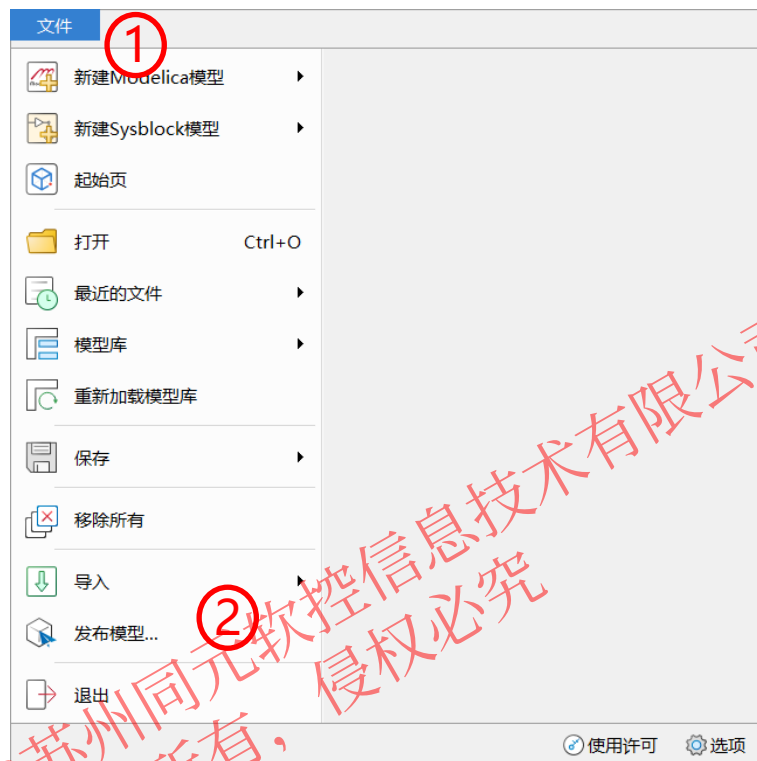
加密等级		复制模型	另存为模型	查看文本	查看图形	查看文档	查看图标	打开模型	在模型浏览器上显示
Access.packageDuplicate		√	√	√	√	√	√	√	√
Access.nonPackageDuplicate	非package模型	√	√	√	√	√	√	√	√
	package模型	×	×	×	√	√	√	√	√
Access.packageText		×	×	√	√	√	√	√	√
Access.nonPackageText	非package模型	×	×	√	√	√	√	√	√
	package模型	×	×	×	√	√	√	√	√
Access.diagram		×	×	×	√	√	√	√	√
Access.documentation		×	×	×	×	√	√	√	√
Access.icon		×	×	×	×	×	√	√	√
Access.hide		×	×	×	×	×	×	×	×

@苏州同元软控信息技术有限公司版权所有，侵权必究

5.模型发布-界面设置

模型发布界面设置流程如下：

- 建立模型库
- 菜单栏“文件”
- 发布模型，打开模型发布窗口
- 设置模型加密等级与许可特征



5.模型发布-文本设置

模型保护级别注解

模型的保护级别设置为Access.xxxx：

```
annotation( Protection( access = Access.xxxx ));
```

示例:保护级别Access.packageText

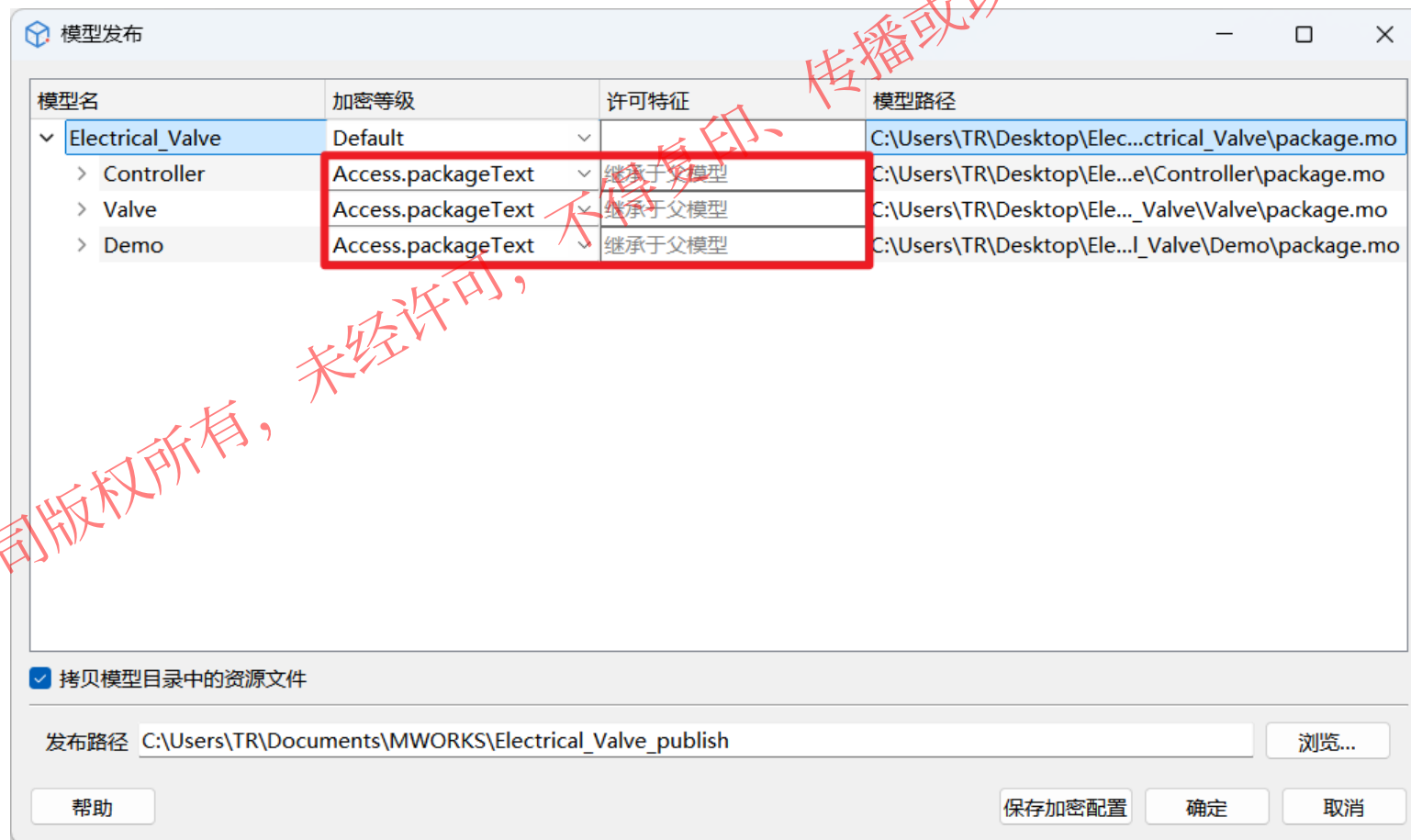
```
package ExampleLib
annotation (Protection(access =
Access.packageText));
end ExampleLib;
```

模型的许可特征设置为xxxx：

```
annotation (Protection(features = {"xxxx"}));
```

示例:许可特征为 “MW_Sysplr_Standard”

```
package ExampleLib
annotation (Protection(features =
{"MW_Sysplr_Standard"}));
end ExampleLib;
```



5.模型发布-规则

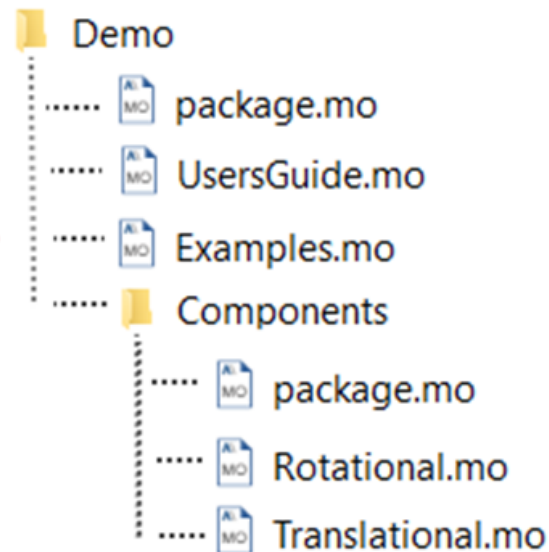
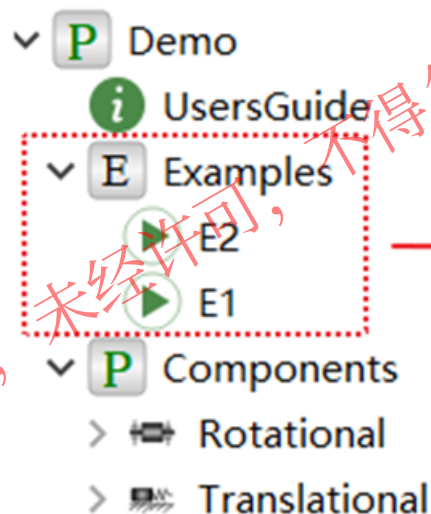
模型的保护级别由自身设置确定，但需要满足约束，约束以模型文件为单位

1.不同模型文件中的模型，保护级别互不影响。但 2. 中的情况除外。

2.无论是否在同一个模型文件，若父节点的等级为hide，子节点的等级将自动变为hide。

3.同一个模型文件中的模型：

- 子节点设置保护级别，父节点没有设置，此时系统默认父节点的保护级别为Access.documentation。
- 相反，父节点设置了保护级别，子节点没有设置，此时子节点自动继承父节点的保护级别。

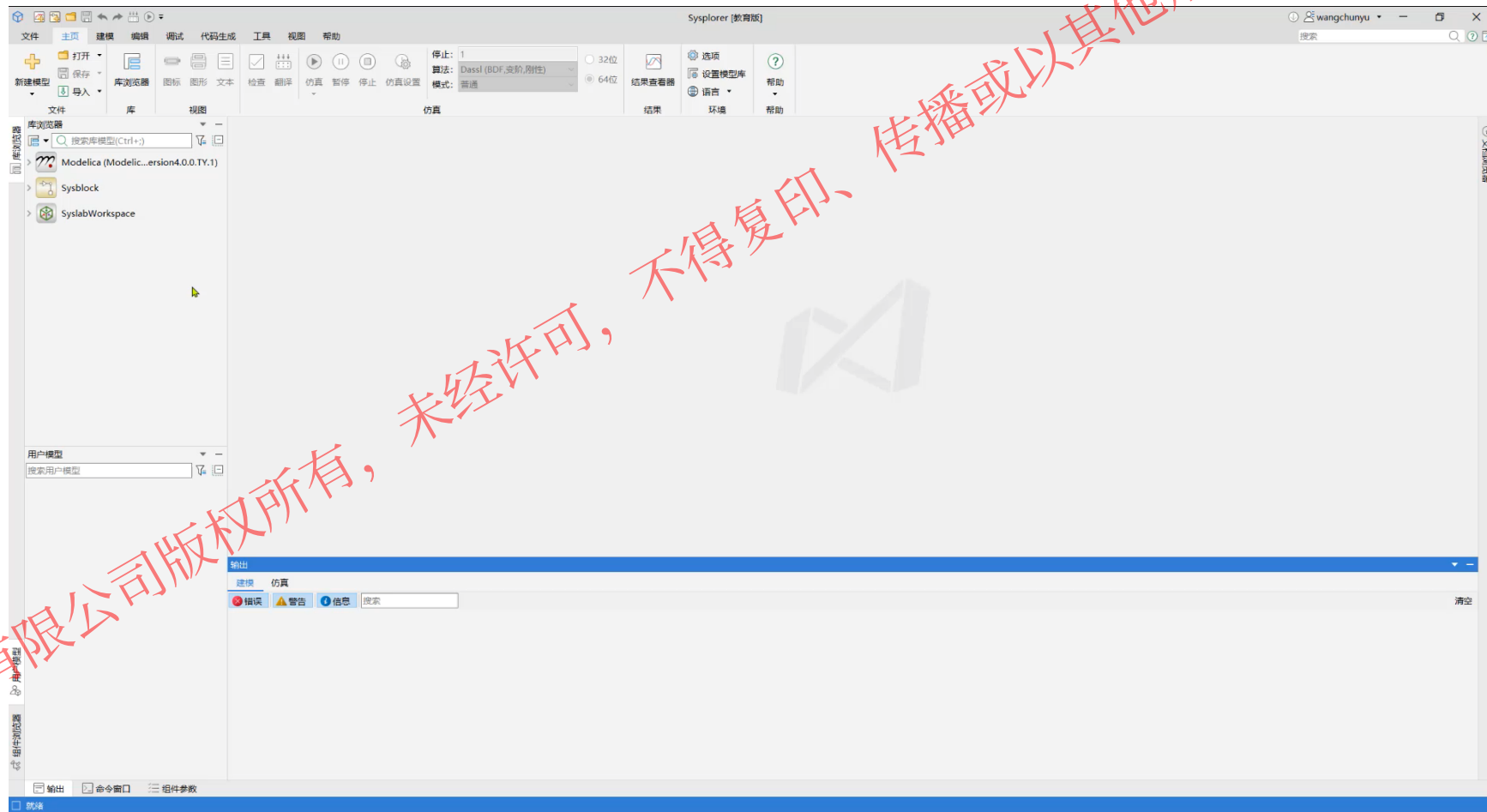


1.Demo.UsersGuide 和Demo.Examples 保护级别互不影响

2.若Demo.Components 的保护级别为hide，则Rotational 和Translational 保护级别也为hide

5.模型发布-示例演示

- 1.模型库准备
- 2.模型库发布
- 3.设置加密等级、许可特征
- 4.选择保存路径
- 5.打开发布模型查看发布特征



模型发布示例.mp4

@苏州同元软控信息技术有限公司
版权所有，侵权必究

CONTENTS

目录 ➔



01 MWORKS.Sysplorer软件概览



02 模型库加载



03 拖拽建模：电子阀流量控制系统



04 文本建模：电阻



05 模型发布



06 MWORKS.Sysplorer与Simulink之间的差异



07 Modelica语法概览

PART 06 ➔

MWORKS.Sysplorer与 Simulink之间的差异

06

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

6.1 因果建模与非因果建模

问题1：有哪些方式可以进行ODE方程组求解？

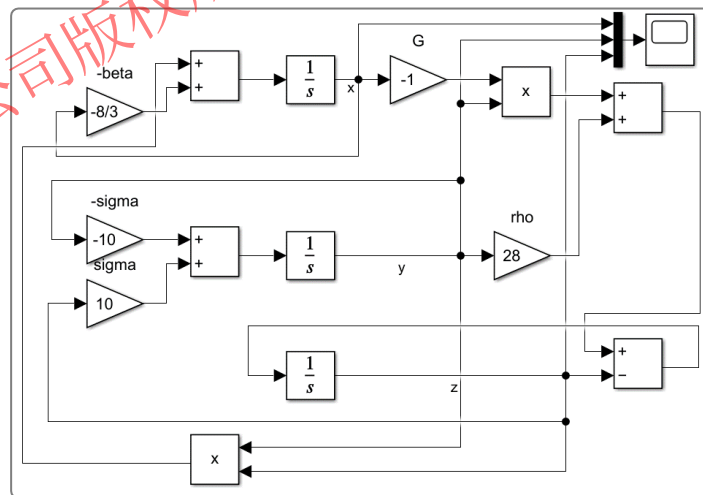
Lorenz方程的求解，

其中 $\sigma = 10, \rho = 28, \beta = 8/3$,

且初值为： $x_0 = 1, y_0 = z_0 = 0$

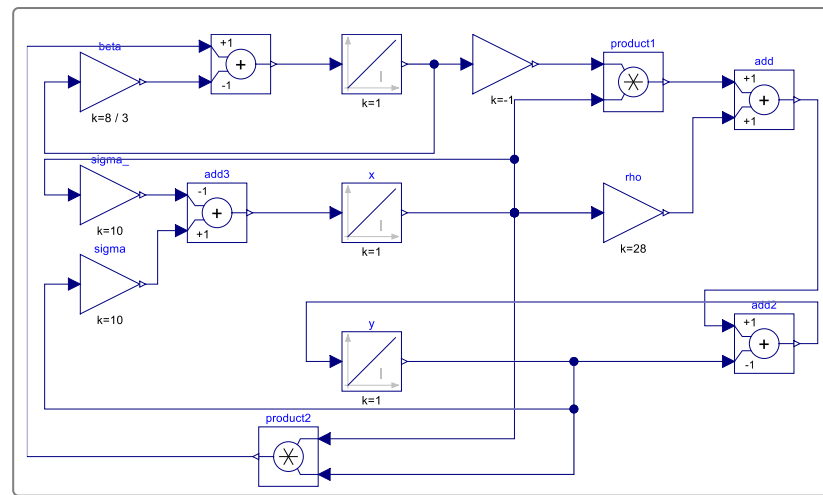
$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

```
clear;
f=@(t,x)[-8/3*x(1)+x(2)*x(3);
-10*x(2)+10*x(3);
-x(1)*x(2)+28*x(2)-x(3)];%描述微分方程系统
tf=100;%仿真结束时间
x0=[0;0;1e-9];%系统初值
options=odeset;
options.RelTol=1e-9;%仿真设置
[t,x]=ode45(f,[0,tf],x0,options);%调用算法求解
plot(t,x);%结果作图
legend('x1','x2','x3');
plot3(x(:,1),x(:,2),x(:,3));%相空间作图
```



#一阶显示微分方程组—洛伦兹方程

```
function Lorenz(du, u, p, t)
du[1] = 10.0 * (u[2] - u[1])
du[2] = u[1] * (28.0 - u[3]) - u[2]
du[3] = u[1] * u[2] - (8 / 3) * u[3]
end
u0 = [10, 0, 0]
tspan = (0, 100)
prob = ODEProblem(Lorenz, u0, tspan)
sol = solve(prob, DP5(), saveat = 0.1)
plot3(sol[1, :], sol[2, :], sol[3, :])
```



@苏州同元软控信息技术有限公司
版权所有，侵权必究

6.1 因果建模与非因果建模

方程的形式描述模型行为

Lorenz方程的求解，其中 $\sigma = 10, \rho = 28, \beta = 8/3$ ，且初值为： $x_0 = 1, y_0 = z_0 = 0$

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

```
model Lorenz
  parameter Real sigma = 10;
  parameter Real rho = 28;
  parameter Real beta = 8 / 3;
  Real x(start = 1.0);
  Real y(start = 0.0);
  Real z(start = 0.0);
equation
  der(x) = sigma * (y - x);
  der(y) = x * (rho - z) - y;
  der(z) = x * y - beta * z;
end Lorenz;
```

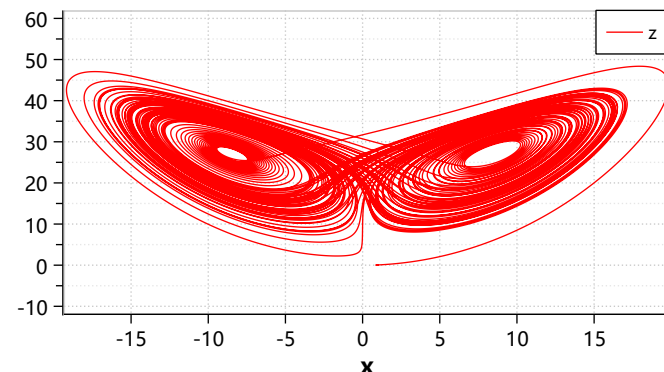
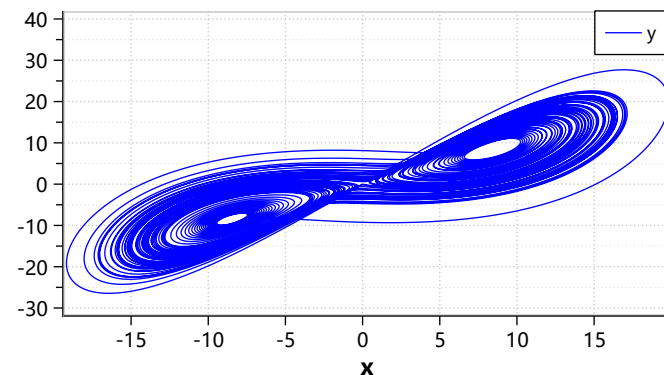
以方程的形式描述模型行为，不需要考虑计算步骤和赋值关系，可极大增加建模效率

定义模型名称: model+模型名称+“备注”

- 声明模型参数
- 声明模型变量
- 声明继承或被调用的模型 (如接口)

- 通过方程或算法，描述模型行为

结束模型定义: end+模型名称;

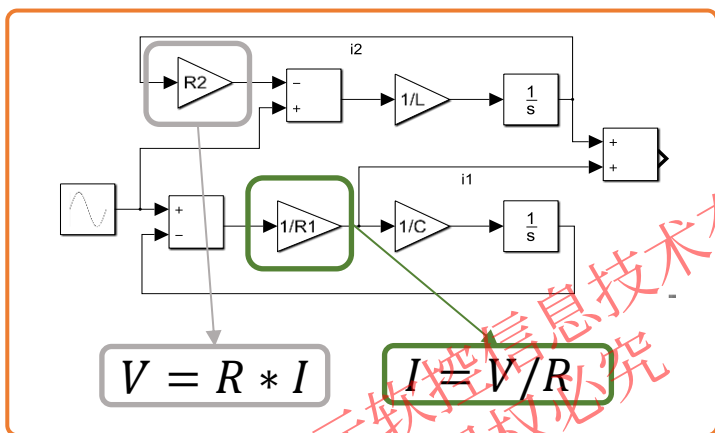
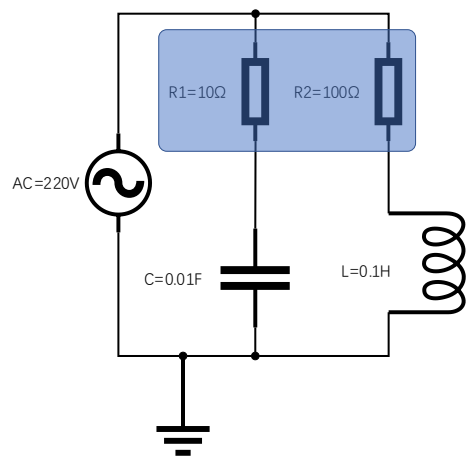


传播或以其他形式使用

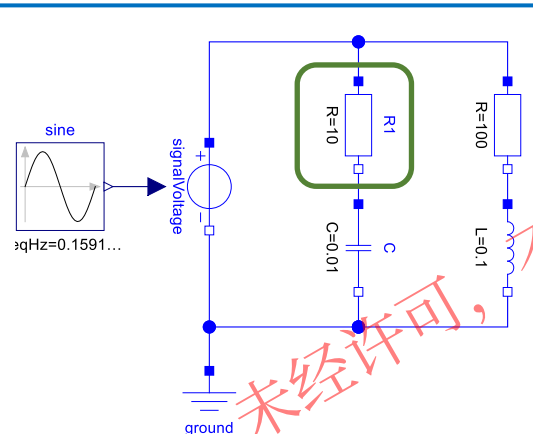
@苏州同元软件技术有限公司版权所有，侵权必究

6.1 因果建模与非因果建模

问题2：因果建模和非因果建模方式如何构建模型？



因果建模：根据模型的求解和编程实现的过程建模



欧姆定律：

$$V = R * I \text{ 或 } I = V/R$$

非因果建模：

Modelica语言可以根据
机理方程直接构建等式，
无需推导因果关系。

```
model Resistance1 "电阻"
  annotation ();
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";
  Modelica.SIunits.Voltage v "电阻两端电压";
  Modelica.SIunits.Current i "电阻两端电流";
  Interfaces.PositivePin positivePin "正极" annotation (...);
  Interfaces.NegativePin negativePin "负极" annotation (...);
equation
  v = positivePin.v - negativePin.v; //电阻两端电压
  i = positivePin.i; //流过电阻电流
  -i = negativePin.i;
  v = R * i; //欧姆定律
end Resistance;
```

```
model Resistance2 "电阻"
  annotation ();
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";
  Modelica.SIunits.Voltage v "电阻两端电压";
  Modelica.SIunits.Current i "电阻两端电流";
  Interfaces.PositivePin positivePin "正极" annotation (...);
  Interfaces.NegativePin negativePin "负极" annotation (...);
equation
  i = v / R; //欧姆定律
  v = positivePin.v - negativePin.v; //电阻两端电压
  i = positivePin.i; //流过电阻电流
  -i = negativePin.i;
end Resistance2;
```

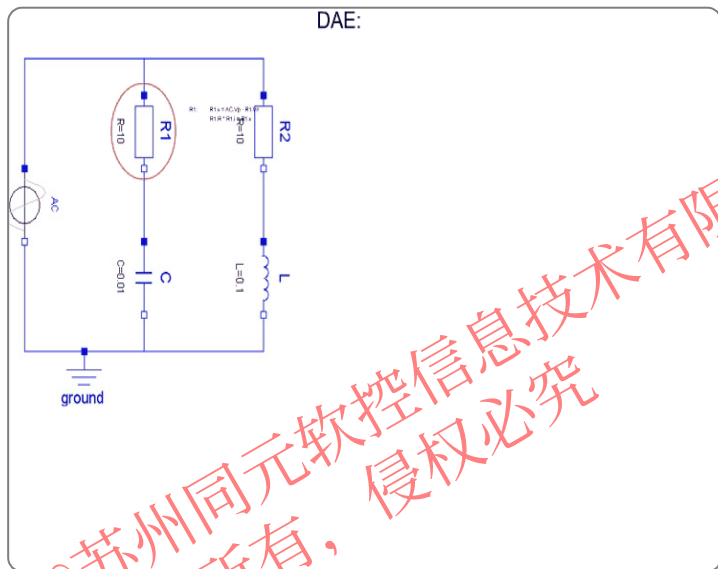
完全等价

6.1 因果建模与非因果建模

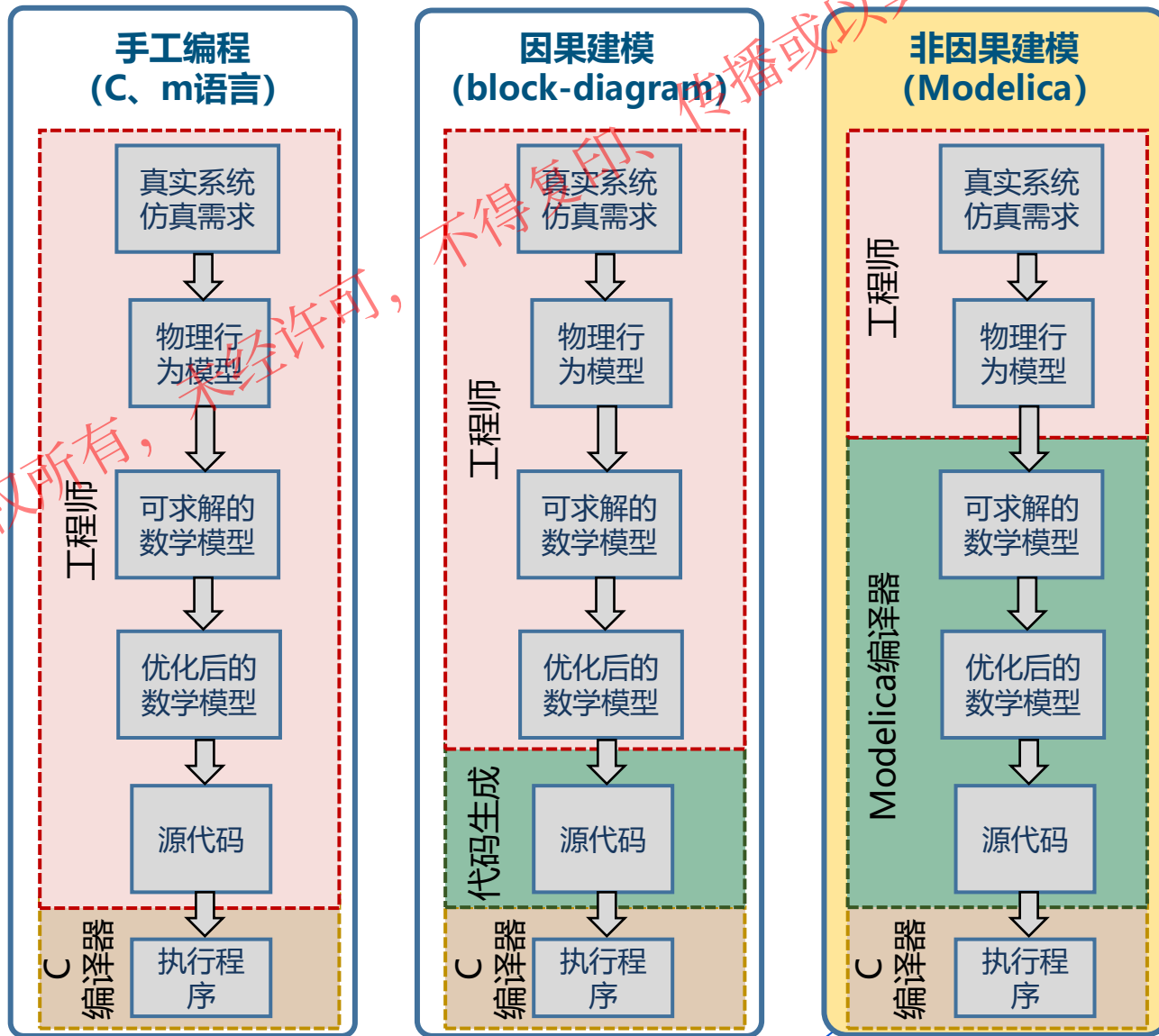
因果建模VS非因果建模

基于数学公式建模，不必关心求解与编程细节

Modelica建模仿真工具能够决定如何自动求解方程变量，无需手工处理。工程师集中精力于建立对象的数学模型，不必关心模型的求解和编程实现的过程，大大提高建模的效率以及模型的可重用性。



非因果建模演示.gif

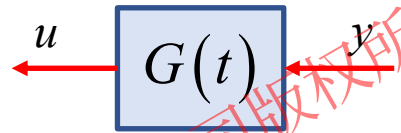


6.1 因果建模与非因果建模

非因果是否意味着：已知模型输出，可以反求其输入？



Modelica实现动态逆模型：

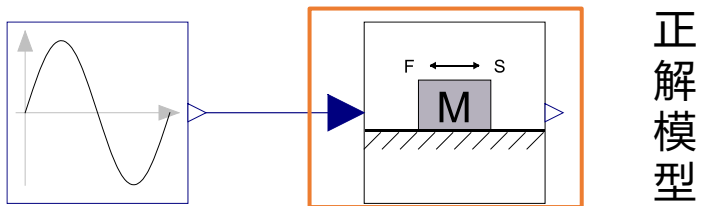
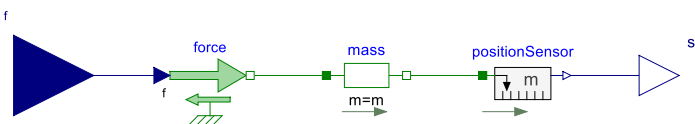


6.1 因果建模与非因果建模

基于非因果建模的动态逆



$$F = ma$$
$$a = \dot{v}$$
$$v = \dot{s}$$

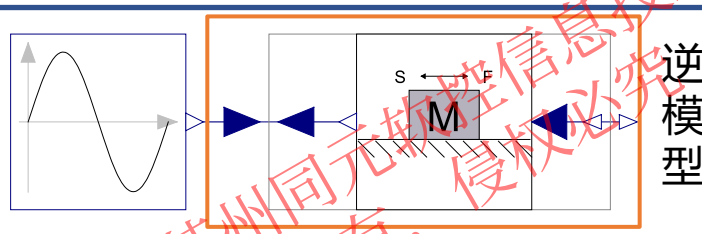


正解模型

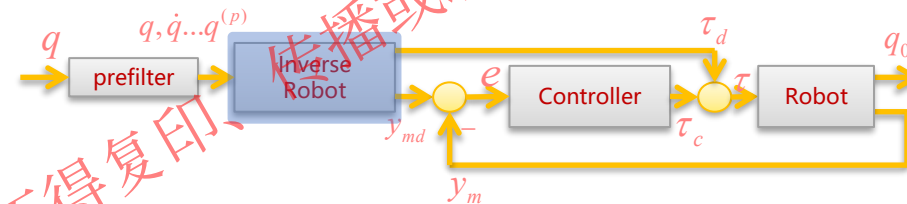
如何基于以上模型，
已知S求解F？



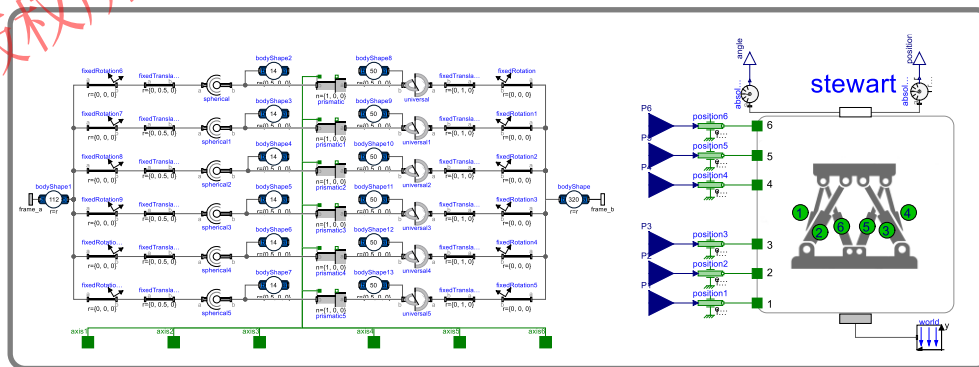
逆模块



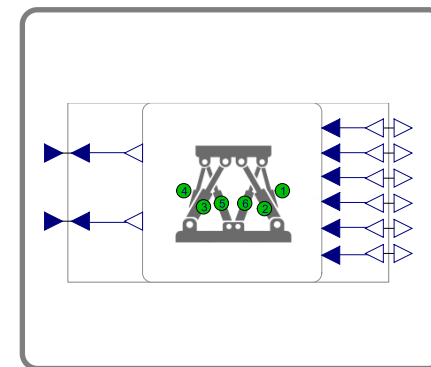
逆模型



动态逆模型表现为**高阶DAE方程**，手工推导十分繁杂，
利用Modelica非因果建模的方式则非常容易求解。



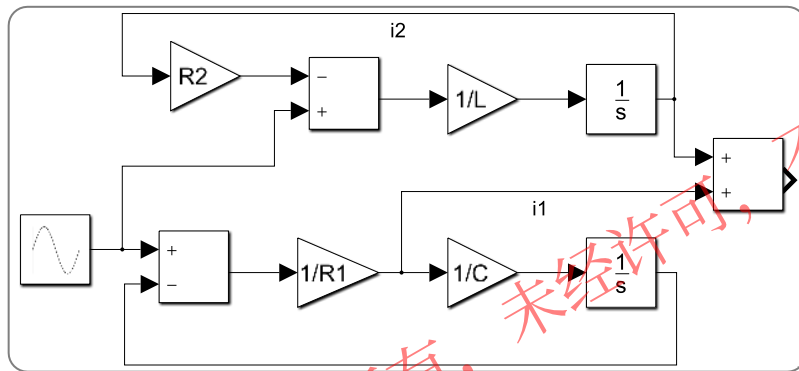
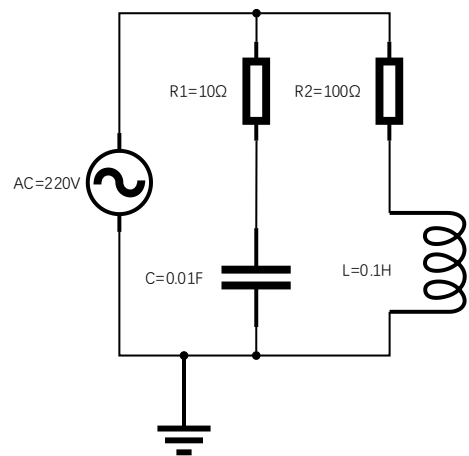
1. 根据物理拓扑关系形成正解模型



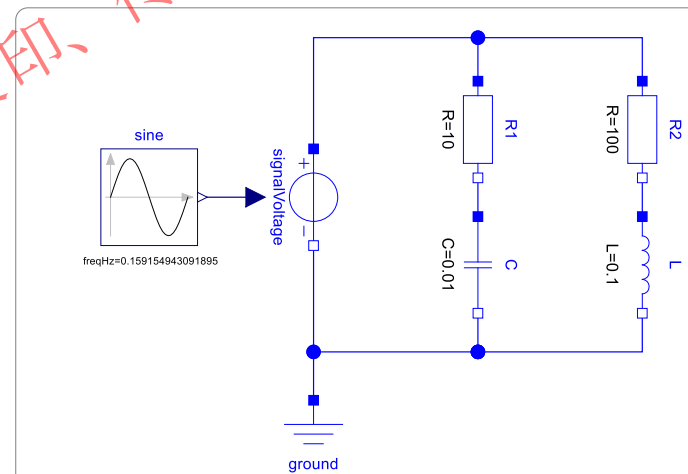
2. 使用逆模块封装

6.2 面向函数与面向对象

问题3：如何根据物理现象构建系统模型？



- 确定系统的输入与输出信号
- 建立方程组
- 将输入带入输出，推导输入输出函数
- 根据输入输出函数，拖放并连接组件

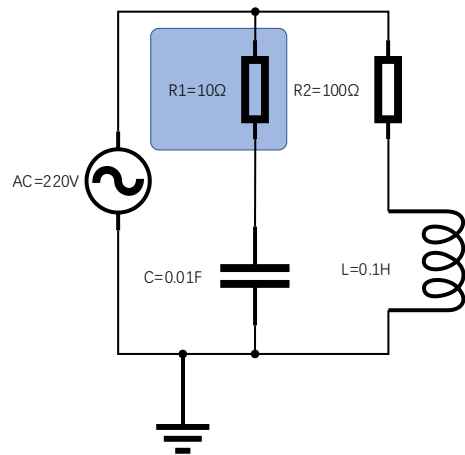


- 根据**物理拓扑关系**，拖放并连接组件

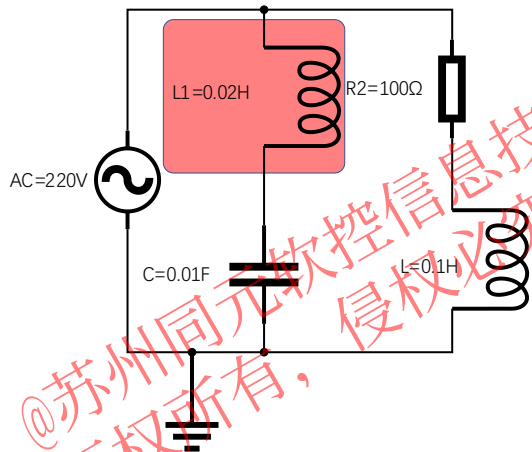
@苏州同元软控信息技术有限公司版权所有，侵权必究

6.2 面向函数与面向对象

问题4：系统架构不变的情况下，如何调整某些组件的型号？



电阻改为电感

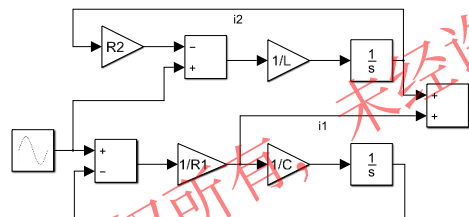


1. 确定系统的输入与输出信号

2. 建立方程组

3. 将输入带入输出，推导输入输出函数

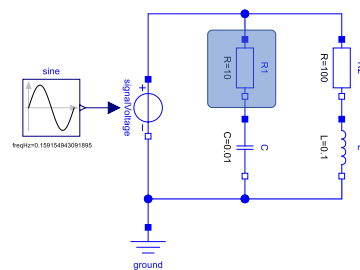
4. 根据输入输出函数，拖放并连接组件



$$\begin{aligned} u(t) &= R_1 i_1(t) + \frac{1}{C} \int i_1(t) dt \\ u(t) &= R_2 i_2(t) + L \frac{di_2(t)}{dt} \\ i(t) &= i_1(t) + i_2(t) \end{aligned}$$

$$\begin{aligned} i_1(t) &= \frac{1}{R_1} \left(u(t) - \frac{1}{C} \int i_1(t) dt \right) \\ i_2(t) &= \int \frac{u(t) - R_2 i_2(t)}{L} dt \end{aligned}$$

1. 根据物理拓扑关系，拖放并连接组件

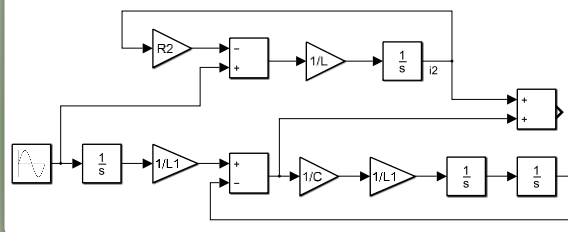


5. 确定系统的输入与输出信号

6. 重新建立方程组

7. 将输入带入输出，推导输出函数

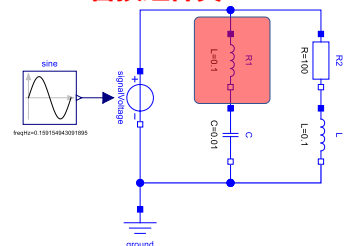
8. 根据输入输出函数，拖放并连接组件



$$\begin{aligned} u(t) &= L_1 \frac{di_1(t)}{dt} + \frac{1}{C} \int i_1(t) dt \\ u(t) &= R_2 i_2(t) + L \frac{di_2(t)}{dt} \\ i(t) &= i_1(t) + i_2(t) \end{aligned}$$

$$\begin{aligned} i_1(t) &= \frac{1}{L_1} \int u(t) dt - \frac{1}{CL_1} \iint i_1(t) dt \\ i_2(t) &= \int \frac{u(t) - R_2 i_2(t)}{L} dt \end{aligned}$$

2. 替换组件类

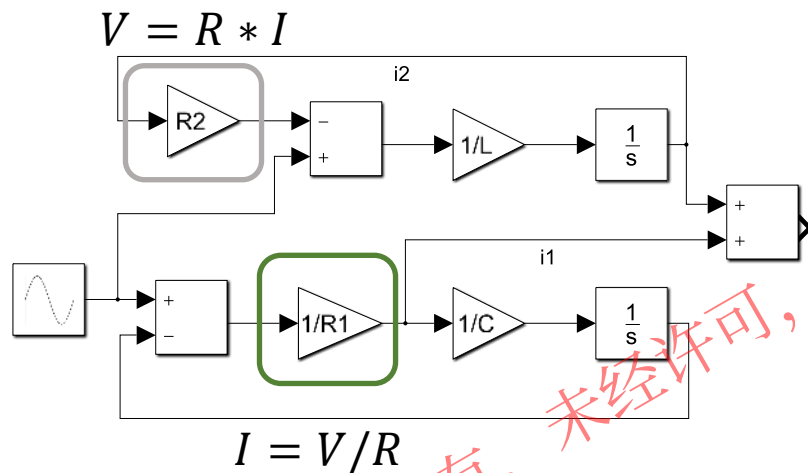
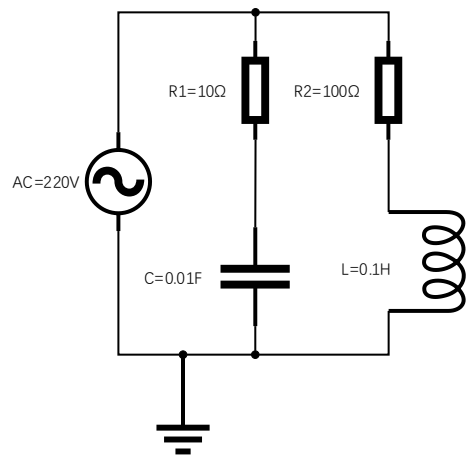


传播或以其他形式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

6.2 面向函数与面向对象

面向函数VS面向对象



面向函数-事务:

分析解决问题的步骤, 使用框图实现每个步骤的事务

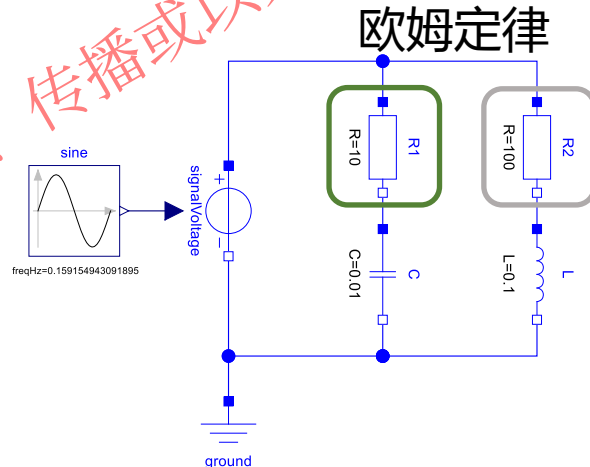
优点:

- 步骤清楚, 便于调试分析;

缺点:

- 需要深入推导, 耗费精力大;
- 重用性低;
- 扩展能力差;

不得复印、传播或以其他形式使用



面向对象-事物:

将事物分解为若干个组成, 根据特点描述事物的行为

优点:

- 具有封装、继承、多态的特性;
- 易重用;
- 易扩展;

缺点:

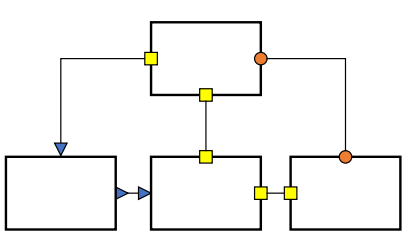
- 不易于调试;

@苏州同元软控信息技术有限公司
版权所有, 侵权必究

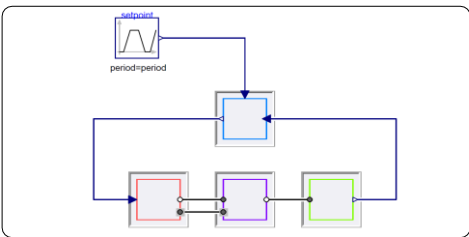
6.2面向函数与面向对象

基于面向对象的模型演进

考虑系统在不同阶段对同一系统粒度需求的差异性，结合Modelica语言面向对象的特征，在架构接口保持不变的情况下，可以通过重声明机制实现对模型的演进，以适用于不同阶段的仿真需求。

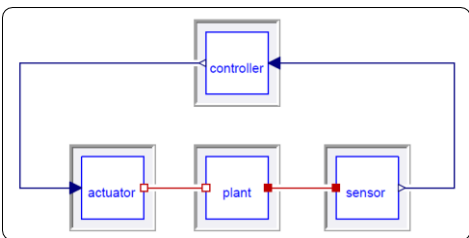


系统架构



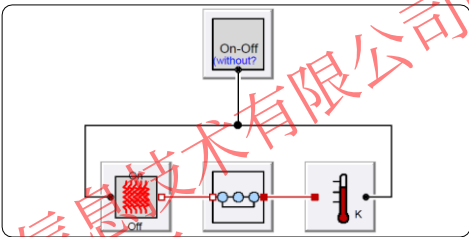
```
Partial Model BaseArchitecture
replaceable Interfaces.PlantModel plant;
replaceable Interfaces.ControlSystem controller;
replaceable Interfaces.Sensor sensor;
replaceable Interfaces.Actuator actuator;
```

架构接口基类模型



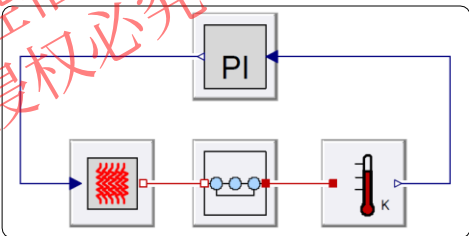
```
Model BaseSystem
extends BaseArchitecture(
  redeclare replaceable Implementations.BasicPlant plant,
  redeclare replaceable Implementations.BasicController controller,
  redeclare replaceable Implementations.IdealSensor sensor,
  redeclare replaceable Implementations.IdealActuator actuator);
```

实例化并重声明的仿真模型



```
Model BaseSystemI
extends BaseArchitecture(
  redeclare replaceable Implementations.Plant plant(C=2, G=1, h=2),
  redeclare replaceable Implementations.OnOffController controller(setpoint=300),
  redeclare replaceable Implementations.TempSensor sensor,
  redeclare replaceable Implementations.Actuator actuator);
```

重声明替换实现模型演进



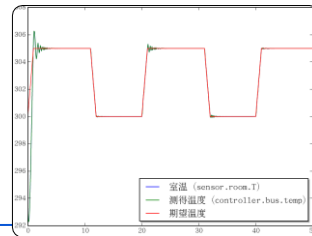
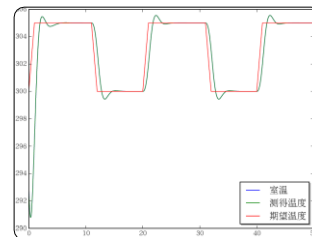
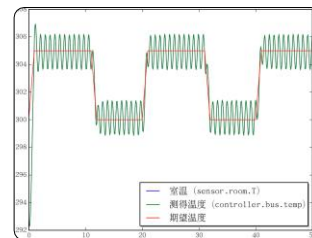
```
Model BaseSystemII
extends BaseArchitecture(
  redeclare replaceable Implementations.Plant plant(C=2, G=1, h=2),
  redeclare replaceable Implementations.PIDController controller(setpoint=300, T=1, k=200),
  redeclare replaceable Implementations.TempSensor sensor,
  redeclare replaceable Implementations.Actuator actuator);
```

根据场景、需求，模型粒度的进一步细化

Modelica多领域仿真模型库

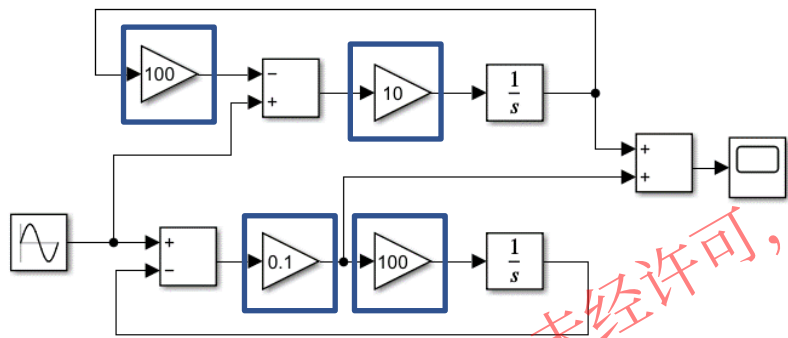
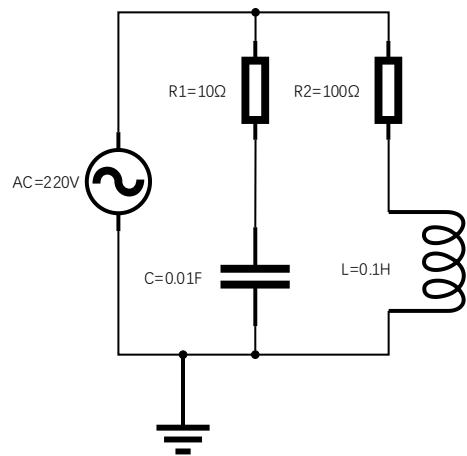
基于Modelica重声明机制的模型演进

动态仿真



6.3 信号建模与物理建模

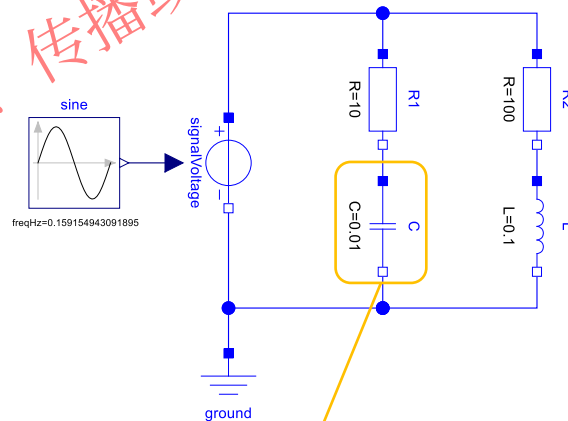
问题5：查看组件的参数，结果有何异同？



- 100: R2
- 0.1: 1/R1
- 100: 1/C
- 10: 1/L

均为无量纲数值，无实际物理意义

传播或以其他形式使用



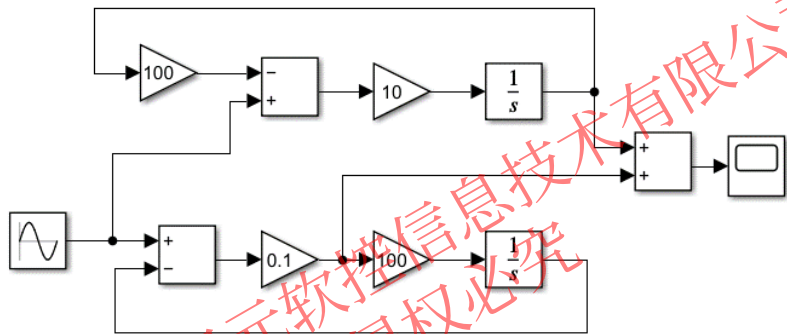
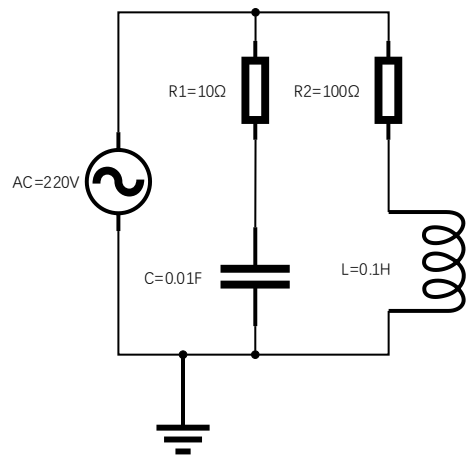
组件参数			
常规			
参数			
C	1	F	电容值
		F	
		uF	

每一个数值都有物理量纲，均具有物理含义，可自动进行单位换算。

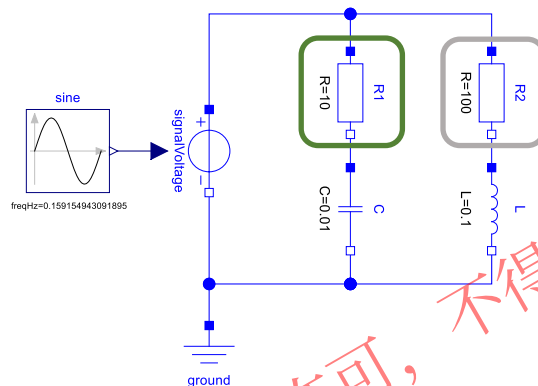
@苏州同元软控信息技术有限公司
版权所有，侵权必究

6.3 信号建模与物理建模

信号建模VS物理建模



只是数值，无实际物理意义



Modelica标准库中已根据ISO 31-1992
国际标准单位制，声明了450个左右的物
理量类型，可声明变量的物理量纲

```
model Resistance1 "电阻"
  annotation ();
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";
  Modelica.SIunits.Voltage v "电阻两端电压";
  Modelica.SIunits.Current i "电阻两端电流";
  Interfaces.PositivePin positivePin "正极" annotation (...);
  Interfaces.NegativePin negativePin "负极" annotation (...);
equation
  v = positivePin.v - negativePin.v; //电阻两端电压
  i = positivePin.i; //流过电阻电流
  -i = negativePin.i;
  v = R * i; //欧姆定律
end Resistance;
```

```
model Resistance1 "电阻"
  annotation ();
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";
  Modelica.SIunits.Voltage v "电阻两端电压";
  Modelica.SIunits.Current i "电阻两端电流";
  Interfaces.PositivePin positivePin "正极" annotation (...);
  Interfaces.NegativePin negativePin "负极" annotation (...);
equation
  v = positivePin.v - negativePin.v; //电阻两端电压
  i = positivePin.i; //流过电阻电流
  -i = negativePin.i;
  v = R * i; //欧姆定律
end Resistance;
```

---- 检查模型: ModelicaGrammar.RLC.BasicModel.Resistance ----

正在解析模型...

正在实例化模型...

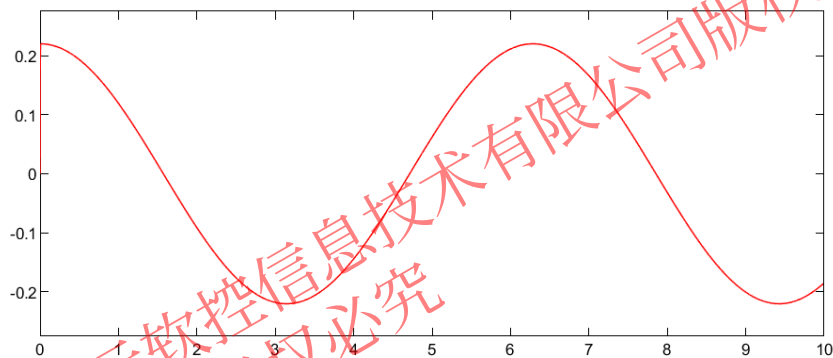
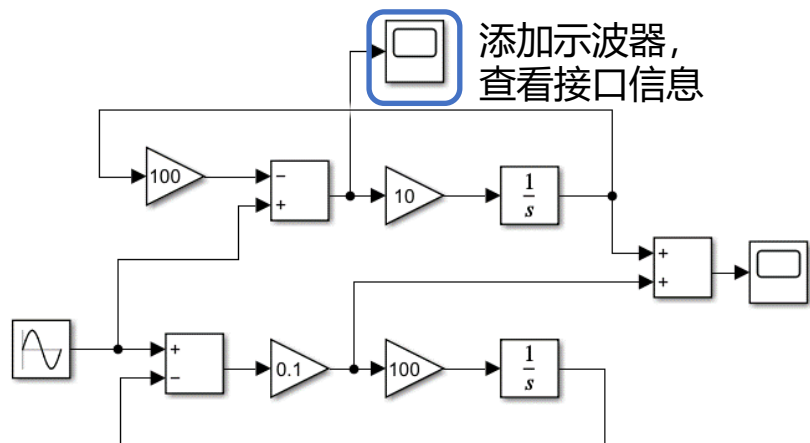
警告(5002): 方程 $v = \text{positivePin.v} - \text{negativePin.v}$ 的单位不匹配:
 v 的单位为 m/s ,
 $\text{positivePin.v} - \text{negativePin.v}$ 的单位为 V .

警告(5002): 方程 $v = R / i$ 的单位不匹配:
 v 的单位为 m/s ,
 R / i 的单位为 Ohm/A .

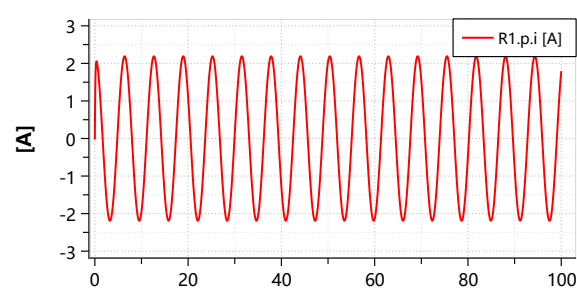
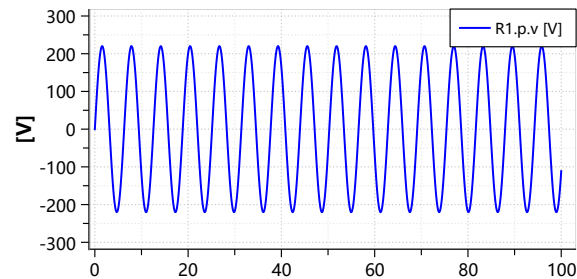
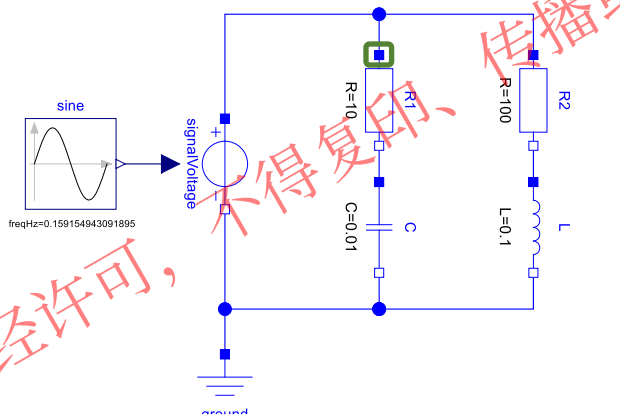
Modelica 还规定了复杂数学表达式单位计算的规则。以便检测模型中单位属性的错误或不一致。这对模型开发者来说是一个巨大的优势!

6.4接口机制

问题6：如何查看R1仿真结果信息？



接口中仅传递是数值信号，因此称为**信号流连接器**。



仿真浏览器

搜索仿真结果变量

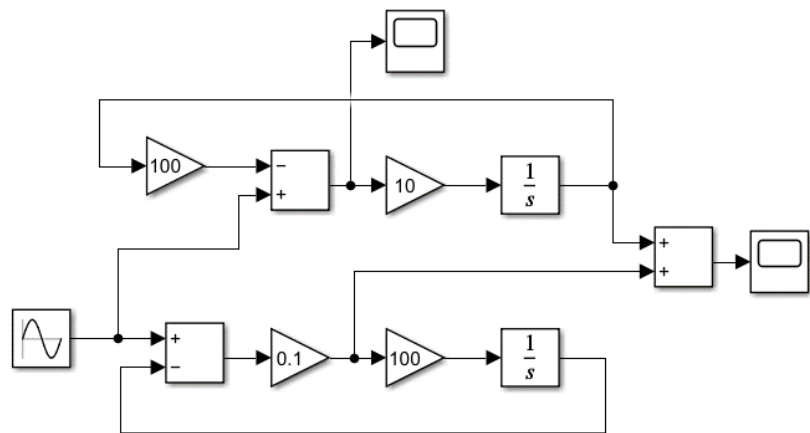
名字	值	单位	描述
Example			
R1			
☐ v		V	Voltage drop of the two ...
☐ i		A	Current flowing from pin ...
p			
☐ v		V	Potential at the pin
☐ i		A	Current flowing into the ...
n			
☐ v		V	Negative electrical pin
☐ i		A	Potential at the pin
☐ L	0.1	H	Current flowing into the ...
Inductance			
> C			
> L			
> R2			
> signalVoltage			
> ground			
> sine			

系统中任意组件中的所有变量均可查看结果

根据接口信息，电阻接口中传递是能量，因此称为**能量流连接器**。

6.4接口机制

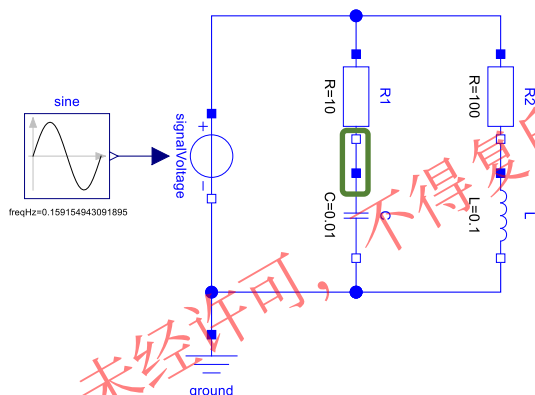
信号流连接器VS能量流连接器



信号流连接器:

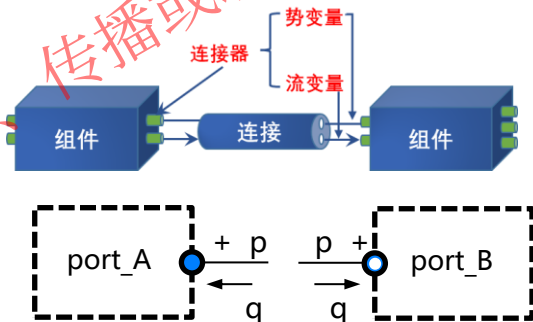
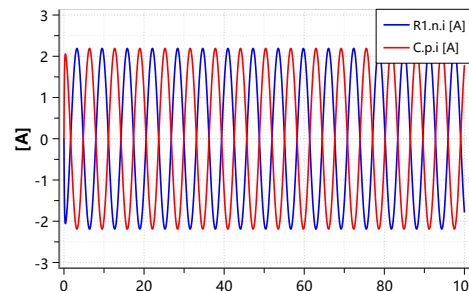
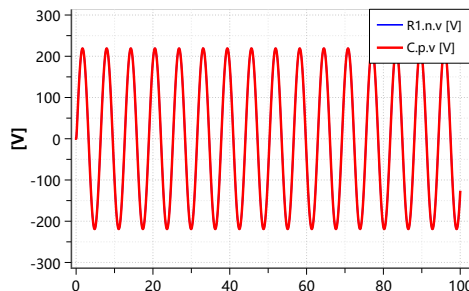
- 传递数值信号;
- 有固定流向, 输出接口→输入接口;
- 输出值→输入值;

能量流连接器更符合物理实际,
无需对子系统进行强行解耦,
极大减少工作量。



能量流连接器:

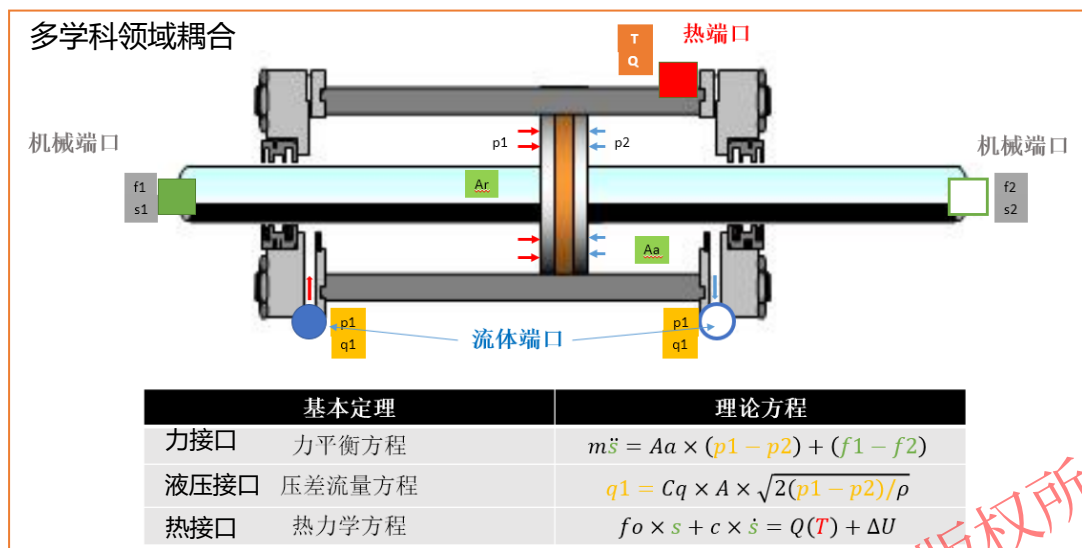
- 传递能量(根据广义基尔霍夫定律分解为势变量和流变量);
- 无固定流向, 根据边界条件和物理原理判定;
- 节点势变量相等, 流变量和为零;



领域	势变量		流变量	
平移机械	s	位移	f	力
转动机械	φ	角度	τ	转矩
电子	v	电压	i	电流
液压	p	压力	$\dot{V}$	流速
热力学	T	温度	$\dot{Q}$	熵流
化学	μ	化学势	$\dot{N}$	粒子流

6.4接口机制

基于能量流连接器的多领域统一建模



2. 实例化接口，定义参数和变量，根据机理方程描述模型行为，绘制图标

```
model Cylinder "液压缸"
parameter Modelica.SIunits.Mass m = 1 "活塞质量";
parameter Real Cq = 0.75 "流量系数";
...
Modelica.SIunits.Area A "活塞面积";
Modelica.SIunits.Density rho "密度";
...
Interfaces.Port_a port_a annotation (...);
Interfaces.Port_b port_b annotation (...);
...
```

```
equation
//力平衡方程
m * a = A * a * (p1 - p2) + (f1 - f2);
a = der(v);
v = der(s);
//压差流量方程
q1 = Cq * A * sqrt(2 * (p1 - p2) / rho);
//热力学方程
f0 * s + c * der(s) = Q(T) + deltaU;
end Cylinder;
```

基于物理原理方程构建的领域转换器可用于不同领域间的能量转换

1. 根据广义基尔霍夫原理构建不同领域接口

```
connector Port_A
Modelica.SIunits.Pressure p;
flow Modelica.SIunits.VolumeFlowRate q;
end Port_A;
```

port_A

```
connector Flange_A
Modelica.SIunits.Position s;
flow Modelica.SIunits.Force f;
end Flange_A;
```

flange_a

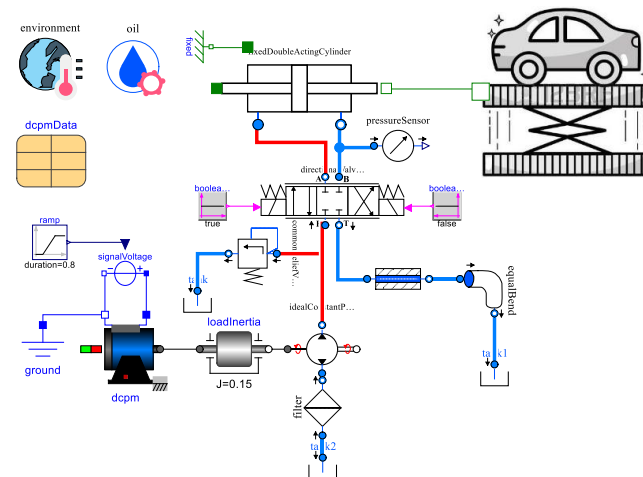
```
connector Port_B
Modelica.SIunits.Pressure p;
flow Modelica.SIunits.VolumeFlowRate q;
end Port_B;
```

port_B

```
connector Flange_B
Modelica.SIunits.Position s;
flow Modelica.SIunits.Force f;
end Flange_B;
```

flange_b

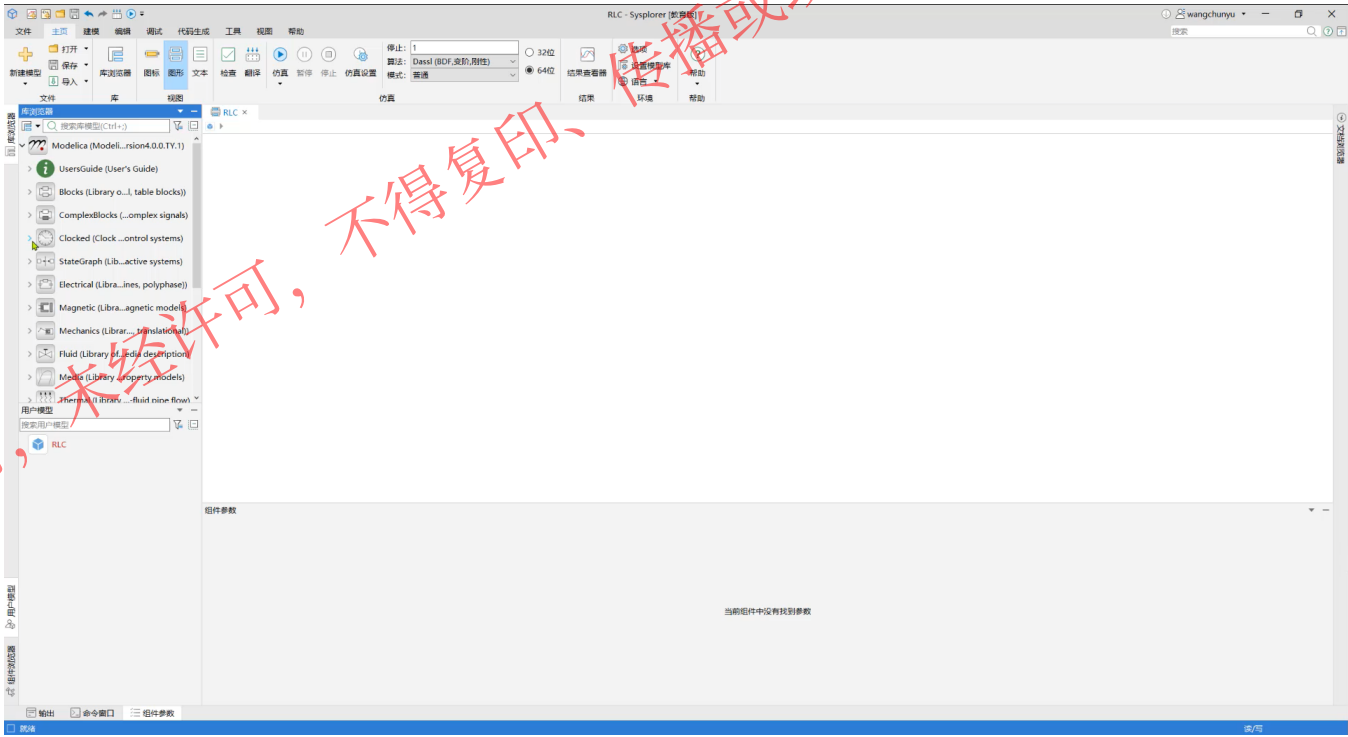
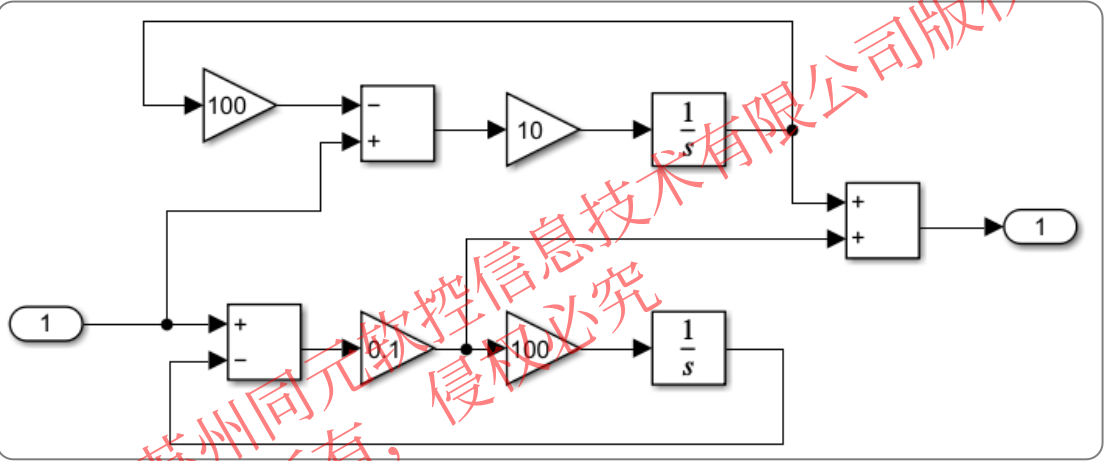
3. 根据物理拓扑关系构建多领域系统模型



6.5模型封装与集成

问题6：当模型库中的组件不满足需求时，如何解决？

Simulink	Sysplorer
框图式模块封装	元件级(框图和物理模型)模块封装 根据机理方程直接构建
S-Function封装	C/C++封装
M脚本封装	Julia脚本封装
FMU模型封装	FMU模型封装



基于物理拓扑直接封装模型.mp4

苏州同元软控信息技术有限公司版权所有，侵权必究

6.5模型封装与集成

外部模型集成

数据融合

- From/To Workspace

接口融合

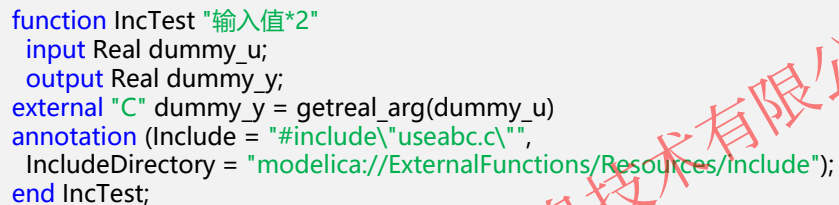
- Sysplorer API
- Syslab Function



外部模型集成.mp4

@苏州同元软控信息技术有限公司版权所有，侵权必究

外部模型集成



三种方式: 1. 调用c或h文件 2. 直接集成C源码 3. 调用动态链接库文件dll



同元MWORKS是中国唯一通过FMI3.0官方测试认证的自主软件，
列入FMI官网目录

CONTENTS

目录 ➞



01 MWORKS.Sysplorer软件概览



02 模型库加载



03 拖拽建模：电子阀流量控制系统



04 文本建模：电阻



05 模型发布



06 MWORKS.Sysplorer与Simulink之间的差异



07 **Modelica语法概览**

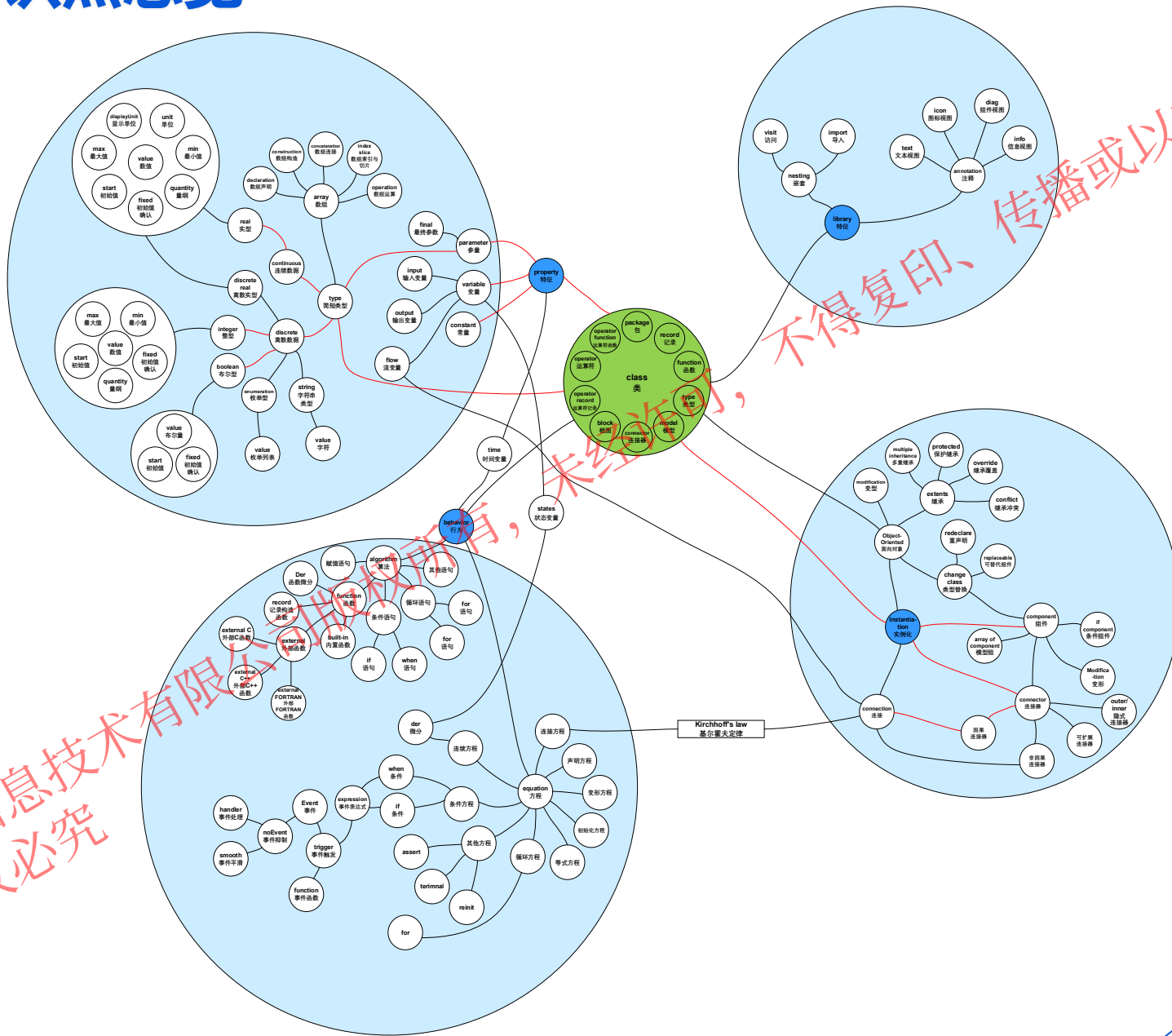
PART 07 ➔

Modelica语法概览

@苏州同元软控信息投
版权所有，侵权必究

07
传播或以其他方式使用

7. Modelica知识点总览



©苏州同元软控信息
版权所有，侵权必究

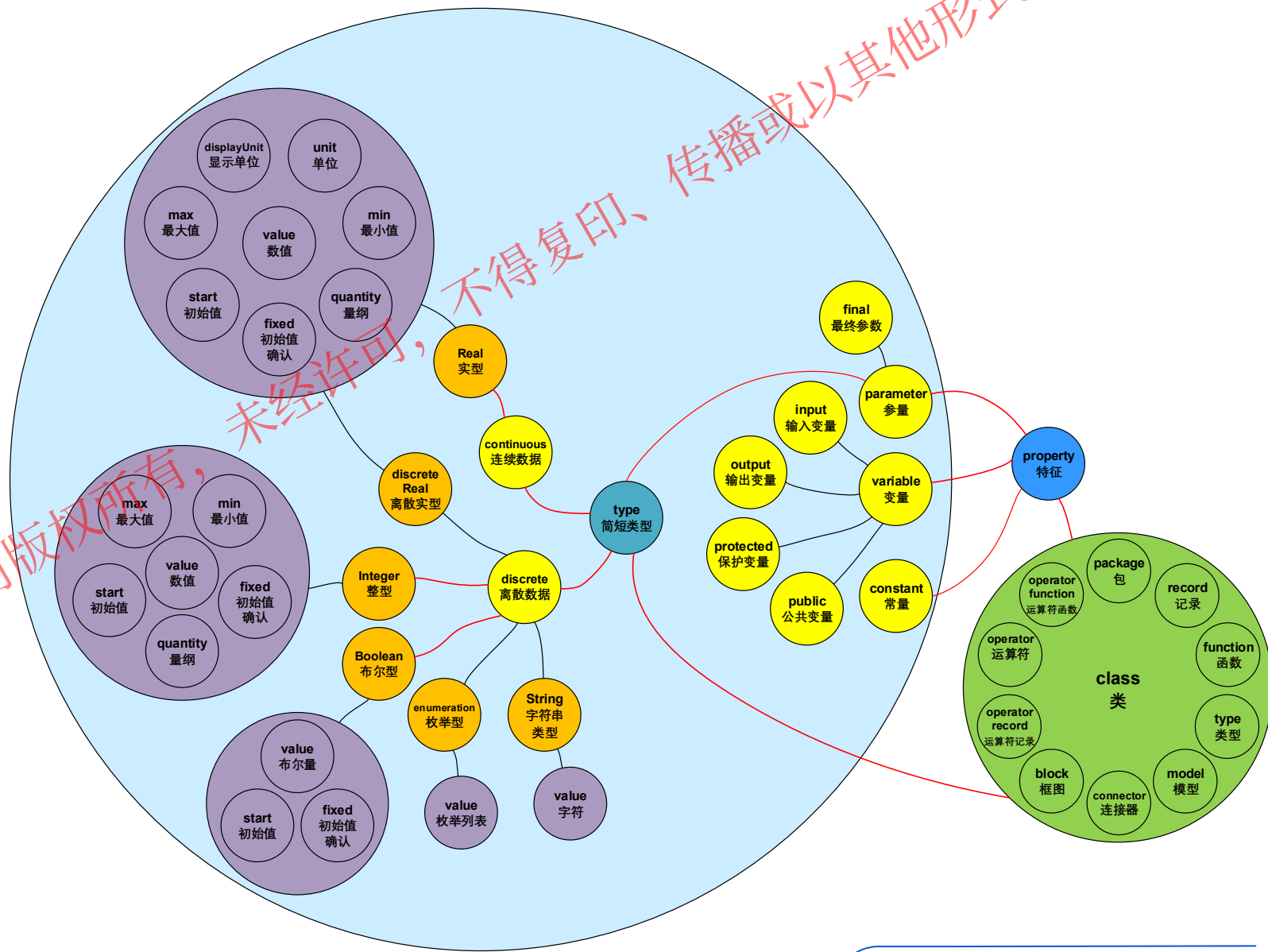
7. Modelica知识点总览-类及内置类型

类及内置类型—构成模型的最基本属性

Modelica语言定义了：

- 10种特化类
- 5种数据类型
- 多种前缀关键词
- 多种数据属性关键词

```
class Fall "自由落体"  
  final parameter Real g = 9.81;  
  Real h(start = 10);  
  Real v;  
equation  
  der(h) = v;  
  der(v) = -g;  
end Fall;
```



7. Modelica知识点总览-数组

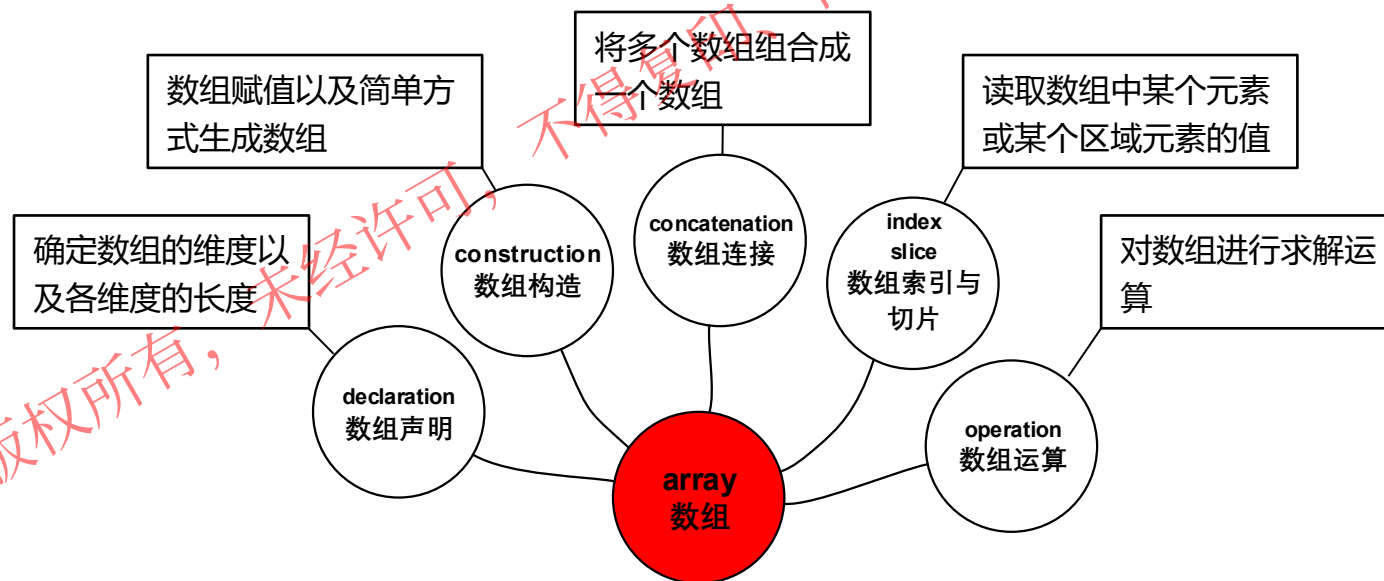
数组—多个同类型变量的集合

Modelica可定义多维数组：

- 0维数组：标量，如：3.14
- 1维数组：向量，如：
- 2维数组：矩阵，如： $\begin{Bmatrix} v_x & v_y & v_z \\ 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 2 & 1 \end{Bmatrix}$

.....

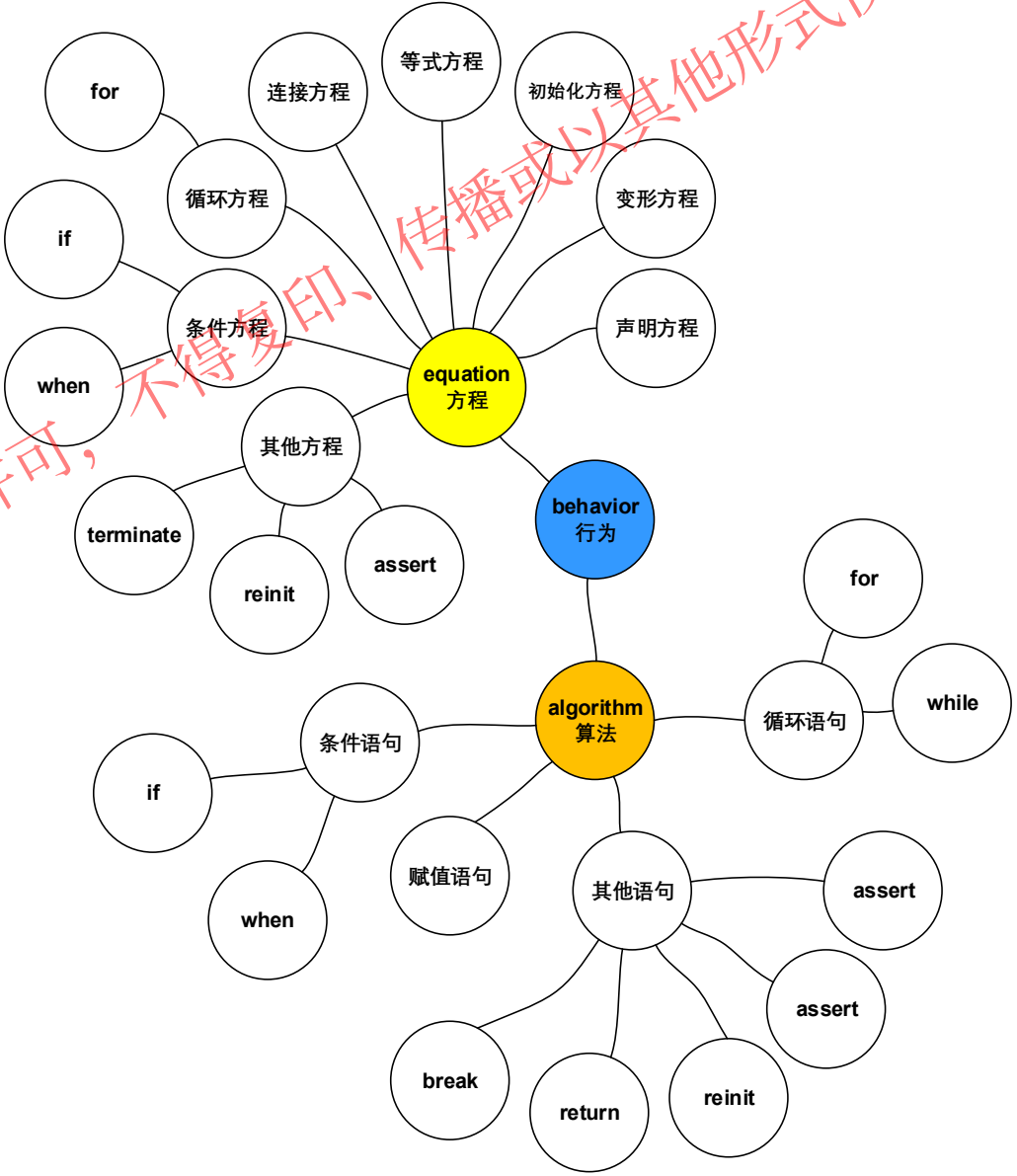
```
model FlatThrow "平抛运动"
  final parameter Real g = 9.81;
  parameter Real v0 = 1;
  Real x(start = 0);
  Real h(start = 10);
  Real position[2] = {x, h};
  Real v[2];
equation
  v[1] = v0;
  v[1] = der(x);
  v[2] = der(h);
  der(v[2]) = -g;
end FlatThrow;
```



7. Modelica知识点总览-模型行为描述

模型行为描述—通过方程或算法定义模型的行为

声明区域方程	声明方程		
	变形方程		
	初始化方程		initial equation
方程区域方程	常规方程	等式方程	=
		连接方程	connect
		循环方程	for
		条件方程	if
			when
		其他方程	assert()
			terminate
			reinit()
算法区语句	算法语句	赋值语句	“:=”
		循环语句	for
			while
		条件语句	if
			when
		其他语句	break
			return
			assert
			terminate
			reinit



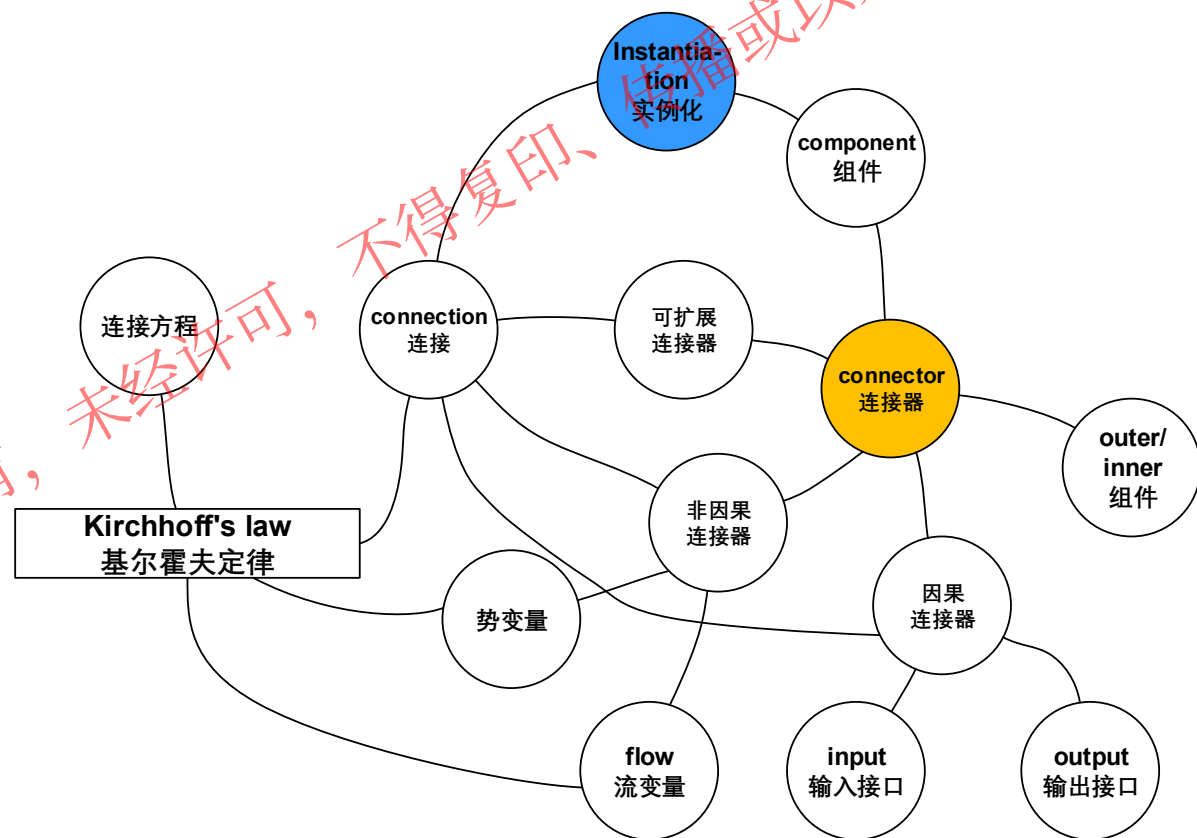
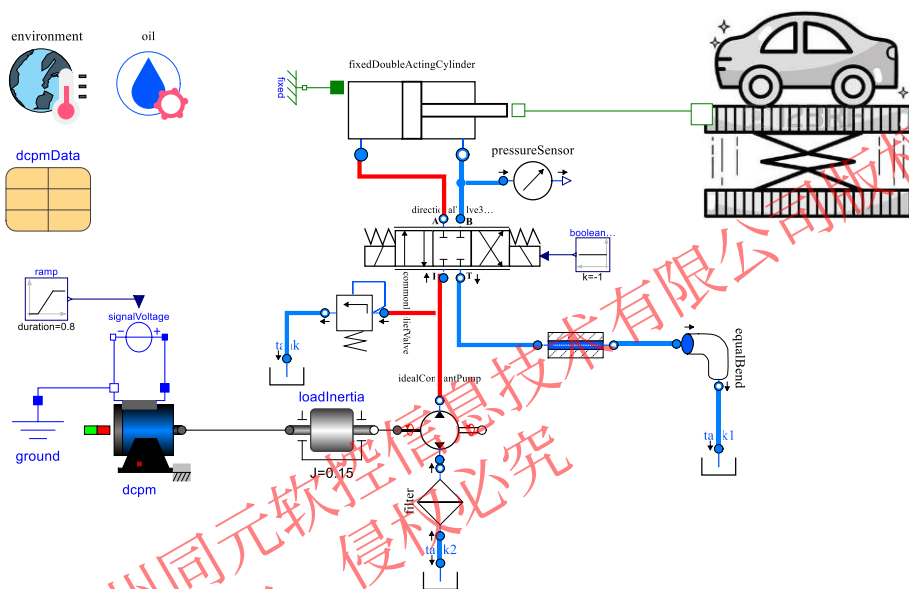
@苏州同元软控信息技术有限公司版权所有, 侵权必究

7. Modelica知识点总览-连接与连接器

连接与连接器—定义组件与外部的通讯

连接器：

- 因果连接器
- 非因果连接器
- 可扩展连接器
- 隐式连接器



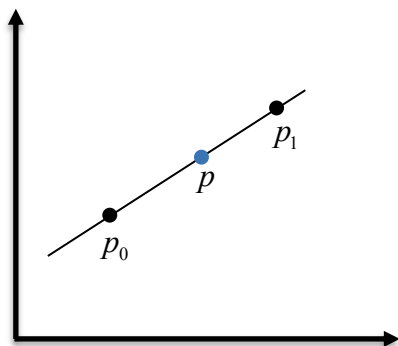
复杂系统上的多种连接器使用

7. Modelica知识点总览-函数

函数—数学模型的天然组成部分

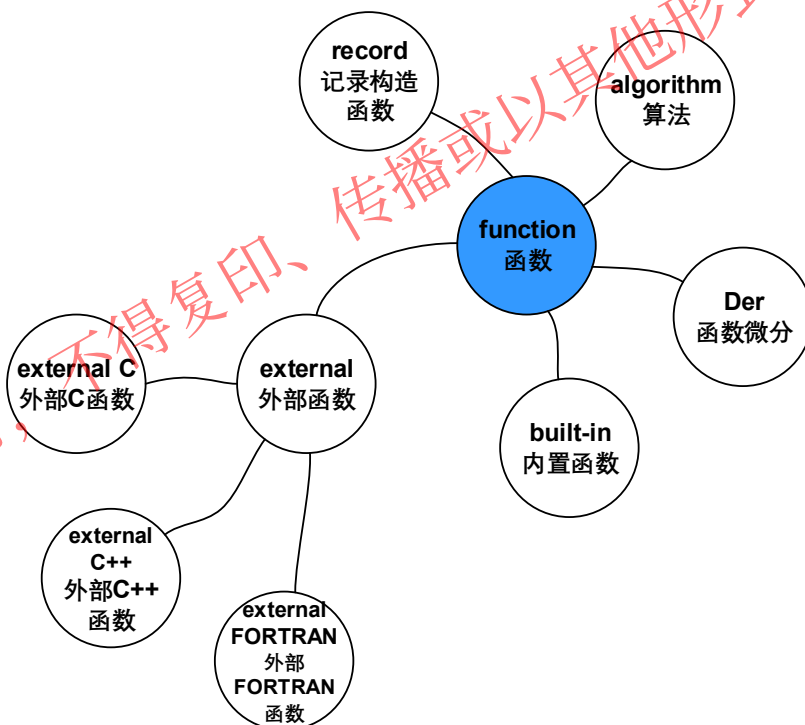
Modelica提供了：

- 多种内部函数
- 自定义函数
- C、C++、FORTRAN外部函数集成



如何计算一条直线上的点？

$$y(x, x_0, y_0, x_1, y_1) = x \frac{y_1 - y_0}{x_1 - x_0} - \frac{x_1 + x_0}{2(x_1 - x_0)} \frac{y_1 - y_0}{2} + \frac{y_1 + y_0}{2}$$



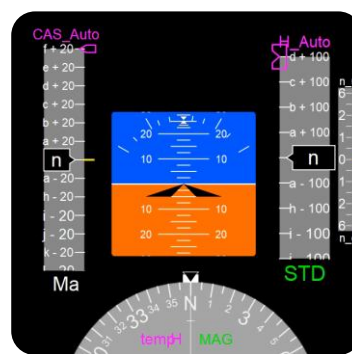
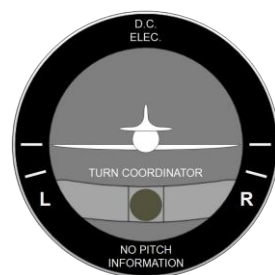
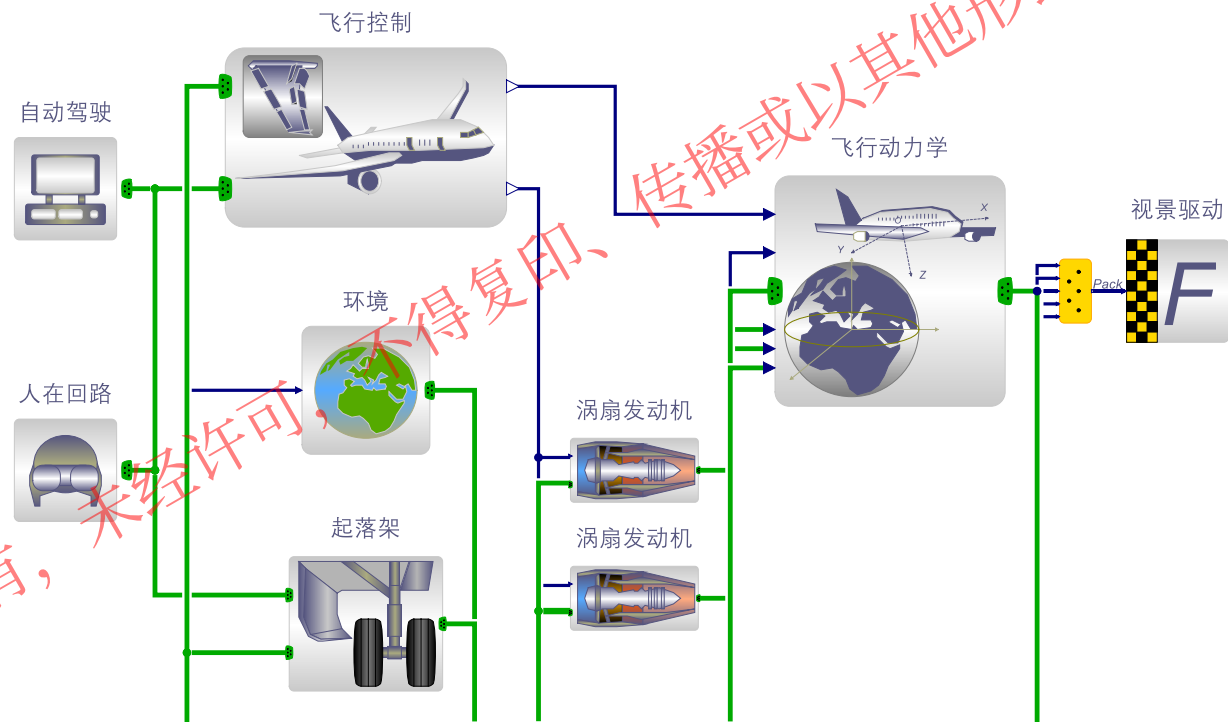
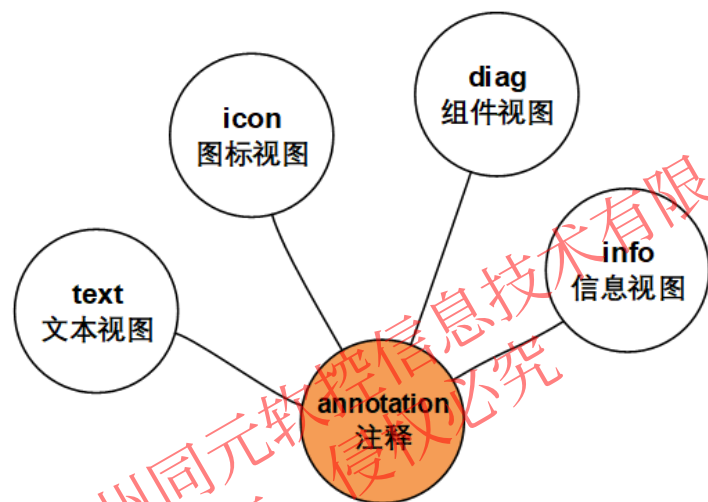
```
function LineWithProtected
input Real x;
input Real p0[2];
input Real p1[2];
output Real y;
protected
Real m = (p1[2] - p0[2]) / (p1[1] - p0[1]);
Real b = (p1[2] + p0[2] - m * (p1[1] + p0[1])) / 2.0;
algorithm
y := m * x + b;
end LineWithProtected;
```

7. Modelica知识点总览-注解

注解—定义与Modelica模型关联的附加信息

注解功能：

- 组件参数框设计
- 图标设计
- 模型帮助文档设计
- 动态显示设计
- 文本其他注释



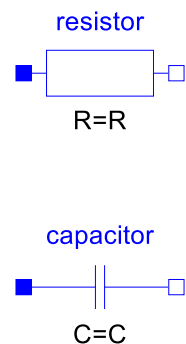
飞机仪表台.mp4

7. Modelica知识点总览-模型重用

模型重用—极大地提高建模效率

模型重用:

- 继承重用
- 实例化重用
- 重声明重用

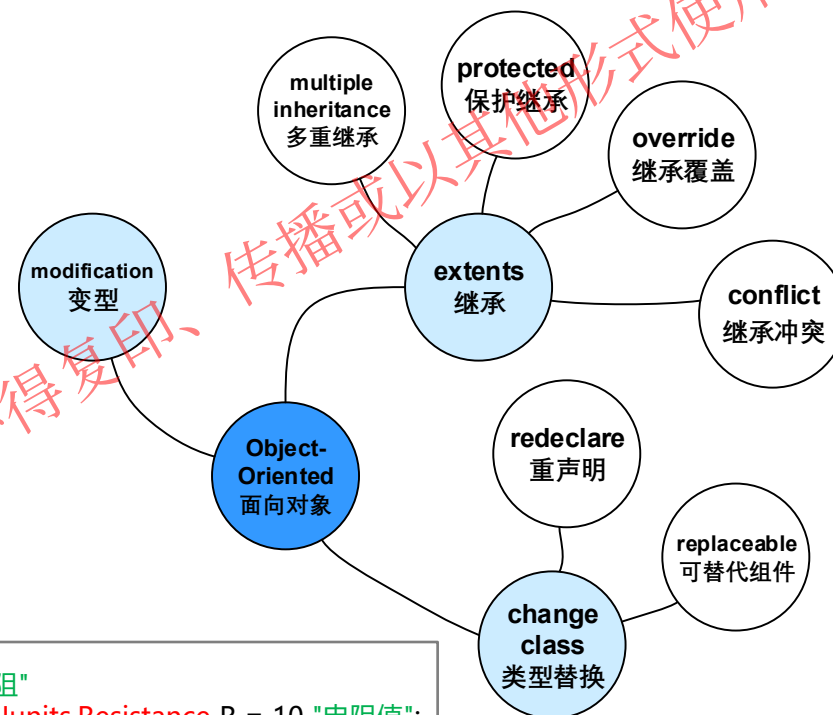


```
partial model PartialPort "基类"  
  Modelica.SIunits.Voltage v "两端电压";  
  Modelica.SIunits.Current i "两端电流";  
  Interfaces.PositivePin p "正极"  
  Interfaces.PositivePin n "负极"  
equation  
  v = p.v - n.v;  
  i = p.i;  
  -i = n.i;  
end PartialPort;
```

基类

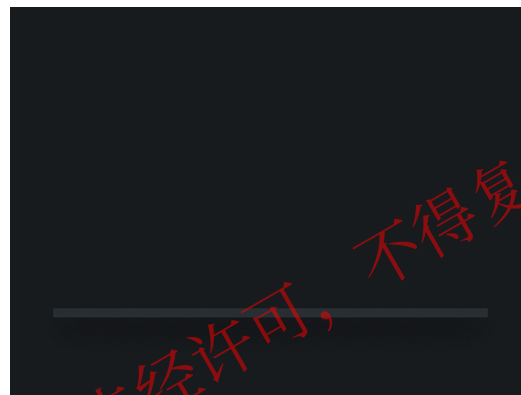
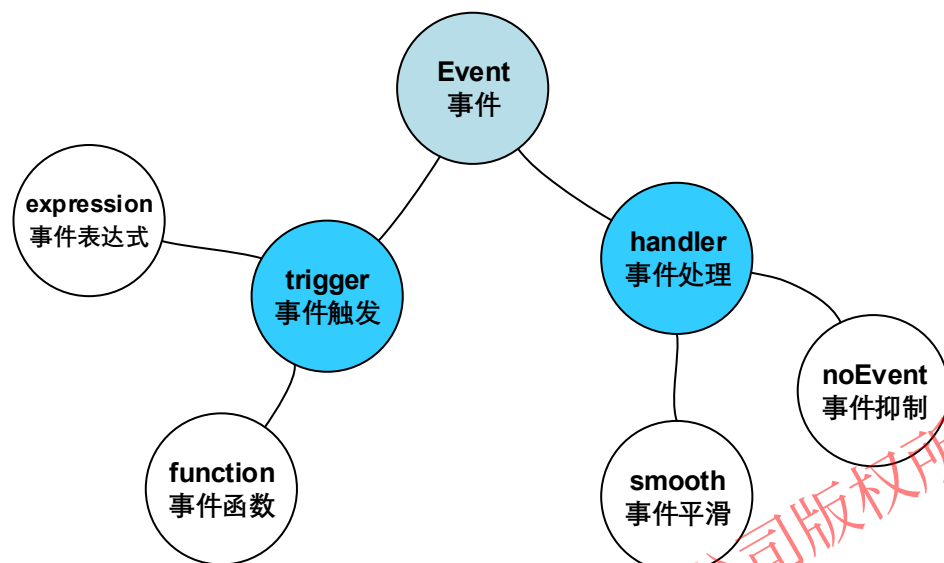
```
model Resistance "电阻"  
  parameter Modelica.SIunits.Resistance R = 10 "电阻值";  
  extends PartialPort;  
equation  
  //欧姆定律  
  v = R * i;  
end Resistance;
```

```
model Capacitor "电容"  
  parameter Modelica.SIunits.Capacitance C = 1 "电容值";  
  extends PartialPort;  
equation  
  //电容公式  
  i = C * der(v);  
end Capacitor;
```



7. Modelica知识点总览-事件

事件—系统变量在特定时间点上瞬时的、不连续的发生变化



弹跳小球.gif

```
model Ball "弹跳小球"
  final parameter Real g = 9.8 "重力加速度";
  parameter Real coef = 0.9 "弹性系数";
  parameter Real h0 = 10 "初始高度";
  Real h "小球高度";
  Real v "小球速度";
  Boolean flying "是否运动";
  initial equation
    h = h0;
  equation
    flying = not (h <= 0 and v <= 0);
    der(v) = if flying then -g else 0;
    v = der(h);
    when h <= 0 then
      reinit(v, -coef * v);
    end when;
  end Ball;
```

MWORKS.Sysplorer 2024a

输出

建模 仿真

Zero-Crossing of the 1th relational expression detected.
Time = 6.314285647, updating discrete events ...
Expression $h \leq 0$ became false, $(h) - (0) = 1.000060657285154e-010$
Expression $h \leq 0$ became false, $(h) - (0) = 1.000060657285154e-010$
Time = 6.314285647, event updating finished.

Zero-Crossing of the 1th relational expression detected.
Time = 8.397142853, updating discrete events ...
Expression $h \leq 0$ became true, $(h) - (0) = -1.000087790359316e-010$
Expression $v \leq 0$ became true, $(v) - (0) = -10.20600049959136$
Expression $h \leq 0$ became true, $(h) - (0) = -1.000087790359316e-010$
Iterating again to find consistent conditions ...
Expression $v \leq 0$ became false, $(v) - (0) = 9.185400449632224$
Time = 8.397142853, event updating finished.

Zero-Crossing of the 1th relational expression detected.
Time = 8.397142853, updating discrete events ...
Expression $h \leq 0$ became false, $(h) - (0) = 1.000061384784812e-010$
Expression $h \leq 0$ became false, $(h) - (0) = 1.000061384784812e-010$
Time = 8.397142853, event updating finished.

Simulation finished at time = 10 (stopTime = 10)
CPU time for Integration: 0.001s
Number of Time Events: 0
Number of State Events: 8
Number of Step Events: 0
Number of Grid Points: 500
Minimum integration stepsize: 1e-006
Maximum integration stepsize: 1.22803

Thanks.

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入工业创新，共创先进软件

